有界误差测向下无线电干扰源的鲁棒定位与滚动清除

摘要

无线电干扰源的快速自动定位与清除是无线电频谱管理中的一项重要任务。本文针对搭载于机器狗上的干扰源自动定位系统展开研究，建立了以“观测—保证—决策”为主线的鲁棒定位与滚动清除模型，使用了多种方法及模型求解定位区域的数学特征，并进一步给出干扰源快速定位与清除的路径规划算法。所有结论均按最坏情形给出，不依赖误差分布假设。

针对问题一：将每次测向给出的误差带转化为两个线性半平面，建立误差带可行域模型。首先采用按方向角排序的**双端队列半平面交**算法精确刻画可行域，并使用独立顺序裁剪算法校验，以严格区分空集、无界、退化和有界四种状态；接着采用**旋转卡壳**算法计算可行域的直径，并采用**随机增量法**计算最小包围圆；最后针对“直径圆”的两种解释分别做出对“是否可覆盖”的判定。

针对问题二：首先以最小接收半径构造必接收候选域，并选择最小化最坏最小包围圆半径作为优化目标，外层采用**差分进化**，内层对最坏场景连续搜索，并用 **SHGO**、活跃圆弧降维与边界扫描复核。求解得对称测点 $Q^{*} = (804.895, \pm 600.052)$ m，最坏包围圆半径 **67.03 m**。进一步，考虑接收边界余量，最终给出带 5 m 裕度的工程测点 $Q_{\mathrm{eng}} = (798.078, \pm 600.788)$ m，最坏半径 67.61 m，移动距离减少 5.01 m。通过与最大交会角、D-最优、E-最优等方法对比，本文目标最贴近 20 m 光学清除条件。

针对问题三：采用“圆心加正六边形”**七点确定性覆盖**保证发现所有干扰源，并采取顺路第二次测向的思想，把各频道的左右候选入口组织成**滚动广义旅行商问题**，用**贪心**初始化并以 **2-opt**、**Or-opt(1)** 与左右分支动态规划求解，再按粒子后验选择测点。500 局同种子配对实验中，6415 个源均全部清除；5 局官方演练中，66 个源均全部清除，平均定位清除时间为 **219.27s/源**。三次正式测试分别清除 **16**、**12**、**11** 个源，单源时间为 **205.11s**、**230.02s**、**267.68s**。

针对问题四：引入定向干扰源后，仅靠七点覆盖不足以保证发现，因此针对性地设计“中心 1 点、内环 12 点与外环 12 点”的 **25 点双环布局**，使圆盘内任一源点的 1000 m 邻域凸包包含该点，对任意未知辐射朝向都能命中检测点。在两批各 350 局的混合场景实验中，所有源均全部清除：原批次按源汇总为 488.11 s/源，独立复跑批次为 487.95 s/源；20 局官方演练的 275 个源全部清除，平均 **490.67s/源**。三次正式测试分别清除 **11**、**12**、**12** 个源，单源时间为 **590.08s**、**503.03s**、**493.76s**。

关键词：双端队列半平面交；旋转卡壳；差分进化；确定性覆盖；滚动广义旅行商；鲁棒路径规划

1 问题重述

目标区域是半径 $R = 1800$ m 的圆盘。机器狗从圆心出发，测向机初始工作在频道 1，需要在 20 个互异频道中搜索无线电干扰源。每局有 10–16 个源，其数量、频道和位置事先均未知。机器狗在移动过程中可以切换频道并调用测向接口：当测点处于源的有效接收范围内时，接口返回 `direction` 及测得方位；当测点距源不超过 5 m 且位于源的辐射范围内时，接口返回 `near`。机器狗还可调用光学定位与清除接口，其中清除距离为 20 m。

测向值与真实方位之间存在不超过 1° 的误差，有效接收半径则只知位于 1000–1500 m 之间。因此，一次测量只能把源限制在一个角度和距离共同确定的区域内，不能直接给出唯一坐标。另一方面，移动、频道切换、测向、光学定位和清除都会增加虚拟时间。所求方案首先要在最不利的接收半径和测向误差下保证源能够被发现并清除；满足这一条件后，才比较路线和动作安排造成的总耗时。正式运行还受到测试窗口 25 min、程序最长 20 min 的限制。

四个问题对应同一求解过程的不同阶段。问题一要求由多次带误差测向恢复源的可行位置范围，并计算其直径及覆盖圆；问题二已知第一次测向结果，需要选择一个必能收到第二次信号的复测位置，使两次测量后的最坏定位范围尽量小；问题三将单源定位嵌入多频道搜索，在保证全向源不漏检的同时安排巡视、复测和清除路线；问题四进一步考虑辐射方向未知的定向源，需要在距离覆盖之外补充方向命中条件。

2 问题分析

四问之间存在明确的依赖关系。一次测向首先产生一个连续的候选区域；多次测向后，需要对这些区域求交，才能判断源还可能位于何处。只有能够稳定计算该交集，第二测点的优劣才有统一的评价标准。得到单源复测规则后，问题三、四才进一步考虑两个实际问题：在哪里测才能保证发现，以及以什么顺序执行测量和清除才能减少总时间。

据此，全文把可行性和效率分开处理。对问题三，先检查保证点能否以最小接收半径覆盖整个目标圆盘；对问题四，还要检查源附近保证点的凸包能否排除所有背向情形。覆盖条件不成立的布点，即使路线很短也不参与时间比较。覆盖成立后，再按式 (1)分解移动、切换、测向、光学定位和清除耗时，并利用同种子配对、消融实验、参数扫描及正式日志判断改动是否确实减少了时间。

2.1 问题一的分析

每次测向给出的不是一条精确射线，而是测得方向左右各偏转 1° 形成的闭扇形。真实源点必须同时满足所有测量约束，所以多次测向的结果应取这些扇形的交集。纯方位定位对测站布局和交会角较为敏感 [1, 2]；常见最小二乘或约束加权最小二乘方法主要给出一个位置估计 [3]，却不能说明其周围还有多大的位置不确定性。题目要求计算定位区域、直径和覆盖圆，因此需要保留整个可行集合。

每个误差扇形可以写成两个线性半平面的交。多次测量后，问题便转化为凸半平面交。若交集有界，其直径表示两个可能源点之间的最大距离，最小包围圆半径则表示从某一位置实施清除时需要覆盖的最坏距离。于是，本问依次求解“误差带构造–半平面求交–直径计算–最小包围圆”四个环节 [4]。交集为空、无界或退化为点和线段时，后续动作不同，算法也分别保留这些状态，而不强行输出普通多边形。

$$\mathcal{P}=\bigcap_{i=1}^{n}\mathcal{W}_i,\qquad \big(D(\mathcal{P}),R^{*}(\mathcal{P})\big)=\left(\max_{X,Y\in\mathcal{P}}\|X-Y\|_2,\ \mathrm{MECrad}(\mathcal{P})\right),$$

其中 $\mathcal{W}_i$ 为第 i 次测向对应的误差扇形。求出 $\mathcal{P}$ 后，题目所需的范围、直径和清除半径便可在同一个凸集上计算。

两次测向形成的误差带、可行多边形与最远点对如图 1所示。

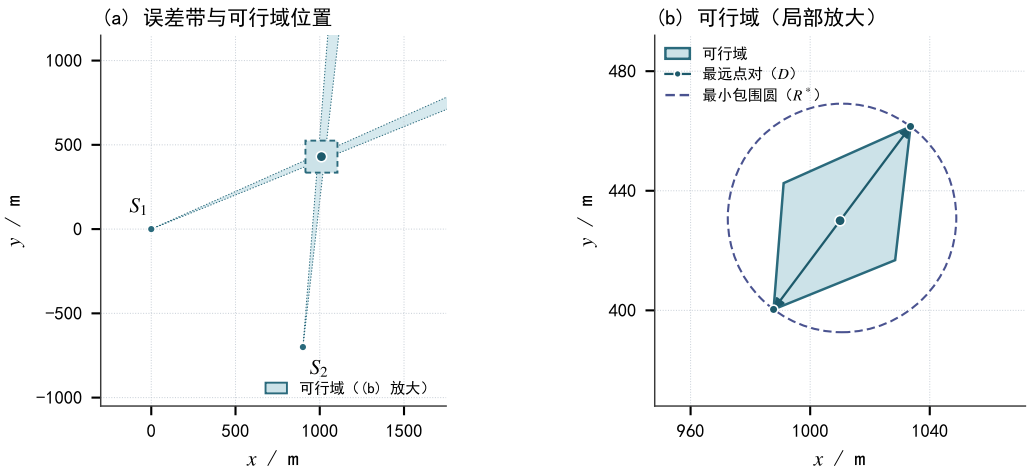


图 1 测向误差带与定位可行域

图中虚线表示测向误差边界，右图说明了可行域、最远点对与最小包围圆之间的几何关系。

2.2 问题二的分析

第一次测向后，源点仍可能位于距离与角度共同限定的扇形区域内。选择第二测点时存在两个先后约束：该点必须对扇形内任一可能源点保持接收，否则无法保证取得第二条方位；在满足接收的点中，再比较两次测向后的最坏定位范围。单纯追求接近 90° 的交会角，可能把测点推到最小接收半径之外，因此不能把交会角直接作为唯一目标。

清除动作关心的是整个定位区域能否被半径 20 m 的圆覆盖，而不是某个估计点的平均误差。故先由最小接收半径构造必接收候选域，再以定位区域的最小包围圆半径作为外层目标。最坏源位置和活跃边界会随测点变化，目标函数并不光滑；计算中先用差分进化搜索，再用 SHGO、连续角度搜索和边界扫描复核数值结果。

$$\min_{Q\in\mathcal{C}_{\mathrm{safe}}}\ \max_{G\in\mathcal{K}} R^{*}\big(\mathcal{P}\big(\theta_1(G),\theta_2(G;Q)\big)\big),$$

其中 $\mathcal{K}$ 表示首次测向后的候选源点集，$\mathcal{C}_{\mathrm{safe}}$ 表示能对其中任一点保持接收的第二测点集合。外层选择测点，内层取所有可能源点中的最坏值，所得位置不依赖某个预设真值。

2.3 问题三的分析

全向源在检测点与源的距离不超过最小接收半径时必能被发现，因此首先要用有限个保证点覆盖整个目标圆盘。该点集只负责保证不漏检，不能因路线较长而随意删减。机器狗访问保证点时逐频道测量；某频道返回方位后，任务随即由“搜索该频道”变为“复测并清除该频道”。如果仍沿固定巡视路线走完后再逐一返回处理，已走过的路段会被重复经过。

因此，本问先确定具有几何保证的七点构型，再在尚未走完的巡视线段上寻找顺路复测点。没有合格顺路点时，才使用问题二得到的工程测点。随着方位、无信号和清除结果不断到来，未访问保证点和待处理频道都会变化，路线不宜一次算定。本文把每个频道的候选测点或清除入口视为一组可选服务点，与剩余保证点共同组成滚动广义旅行商问题，每执行一个动作便更新一次任务集合。

$$\max_{G\in\Omega}\ \min_{v\in V}\|G-v\|_2\le L,\qquad \min_{\pi}\mathrm{Len}(\pi),$$

前一式是保证发现的距离覆盖条件，后一式才是在覆盖成立后比较的路线代价。二者分开能够避免以缩短路径为由牺牲漏检风险。

2.4 问题四的分析

定向源只向某个未知朝向的半平面辐射。此时，距离源点不超过 1000 m 只是接收的必要条件；若附近检测点恰好全部位于辐射半平面的另一侧，仍会漏检。为了消除未知朝向，需要寻找一个不含方向参数的等价条件。若源点位于附近检测点的凸包内，这些检测点就不可能全部处在经过源点的同一开半平面中，因而任意辐射朝向至少命中一个检测点。

由此，问题四的布点要同时满足两项条件：参与凸包的检测点距源不超过最小接收半径，且其凸包包含源点。本文先在对称双环点集中搜索满足这两个条件的参数，再比较保证点数量和开放巡视路线。执行阶段仍沿用问题三的滚动规划，但第二测点可能因定向遮蔽返回 `no_signal`；此时只把当前频道转入局部光学搜索，其余保证点和频道状态保持不变。

$$\max_{G\in\Omega}\min_{v\in V}\|G-v\|_2\le L,\qquad G\in\mathrm{conv}(V_G),$$

其中 V_G 为源点附近用于形成凸包的保证点。前一条件控制接收距离，后一条件保证任意辐射半平面至少命中一个检测点，二者共同构成问题四新增的方向性约束。

3 模型假设与符号说明

3.1 基本假设

建模假设汇总于表 1。

文中的“鲁棒”限于确定性有界误差：只要每次测向误差都落在 $[-\delta,\delta]$ 内，无论各次误差是否相关、是否同号，真实源点都应保留在可行域中。这里不使用误差分布或置

表 1 建模假设

编号	假设内容
A1	源点均位于半径 1800 m 的目标圆盘内，频道互异；源的有效接收半径在 1000–1500 m 之间。
A2	每次测向的真实误差绝对值不超过 1°，对同一源在不同测站的误差不作独立同分布假设。
A3	测站坐标和机器狗返回的位置按题面及接口值使用，不另加入平台定位误差；机器狗沿欧氏直线移动，动作时间按题面规则计入，网络重试复用同一 `request_id`。
A4	“不漏检”采用最小接收半径 1000 m 和定向半平面最坏朝向进行验证；随机实验只用于比较效率。
A5	离线与官方演练结果不替代正式隐藏实例的成绩，正式测试表保留原始记录位置。

信概率，也不把平台定位误差、系统偏置、多径传播和移动目标纳入保证范围。后文给出的最坏半径和覆盖结论均在这一范围内成立。

3.2 符号与时间目标

主要符号见表 2。

表 2 主要符号

符号	含义
Ω	半径为 R 的目标圆盘。
$S_i=(x_i,y_i)$	第 i 个测站及其平面坐标，单位为 m。
G	干扰源的真实位置，单位为 m。
X	定位可行域中的候选源点，单位为 m。
$Q=(a,b)$	第二测点及其局部坐标，单位为 m。
Q^*	最坏定位圆半径最小时的理论第二测点。
Q_{eng}	加入接收安全裕度后的工程第二测点。
m	第二测点的接收安全裕度，本文取 5 m。
θ_i	第 i 次测向所得方位角。
δ	测向误差上界，本文取 1°。
$\mathcal{W}_i$	第 i 次测向对应的闭误差扇形。
$\mathcal{P}$	全部测向误差扇形的公共可行域。
a_j	第 j 个半平面约束中 x 坐标的系数。
b_j	第 j 个半平面约束中 y 坐标的系数。
c_j	第 j 个半平面约束的常数项，单位为 m。
n	参与定位的测向次数。
n_P	可行多边形 $\mathcal{P}$ 的顶点数。
U_i	可行多边形 $\mathcal{P}$ 的第 i 个顶点。
$D(\mathcal{P})$	可行域 $\mathcal{P}$ 的直径，单位为 m。
$R^*(\mathcal{P})$	可行域 $\mathcal{P}$ 的最小包围圆半径，单位为 m。
C^*	可行域最小包围圆的圆心。
$J(Q)$	第二测点 Q 对应的最坏定位圆半径，单位为 m。
$\mathcal{K}$	首次测向后所有可能源点构成的集合。
$\mathcal{C}_{\mathrm{safe}}$	对 $\mathcal{K}$ 内任一点均保持接收的第二测点集合。
R	目标圆盘半径，本文取 1800 m。

续下页

续表 2：主要符号

符号	含义
L	保证接收所采用的最小接收半径，本文取 1000 m。
r_c	光学清除半径，本文取 20 m。
ρ	首次测站至候选源点的径向距离，单位为 m。
ϕ	候选源点相对首次测向中心线的偏角。
d_s	敏感性分析中测站至目标的距离，单位为 m。
P_0	双环保证布局的中心点。
P_k^{in}	双环布局中第 k 个内环保证点。
P_k^{out}	双环布局中第 k 个外环保证点。
r_{in}	双环布局的内环半径，单位为 m。
r_{out}	双环布局的外环半径，单位为 m。
φ	外环相对内环的角度偏移。
V	尚未访问的保证点集合。
V_G	距源点 G 不超过 L 的保证点集合。
Γ	全部 20 个频道构成的集合。
C_t	第 t 步已检出但尚未清除的频道集合。
$\mathcal{Z}_t$	第 t 步结束后的系统状态。
Φ	根据动作及其返回值更新系统状态的转移函数。
X_t	第 t 步结束时机器狗的位置，单位为 m。
$\mathcal{P}_{c,t}$	第 t 步频道 c 对应的定位可行域。
$s_{c,t}$	第 t 步频道 c 的处理状态。
a_t	状态 $\mathcal{Z}_t$ 下执行的动作。
y_t	动作 a_t 执行后获得的接口返回值。
$\mathcal{A}$	一局任务的完整动作序列。
$\mathcal{G}(\mathcal{A})$	动作序列满足覆盖与凸包命中条件的指示量。
$\mathcal{S}(\mathcal{A})$	动作序列完成全部源清除的指示量。
T	一局任务的总虚拟时间，单位为 s。
T_{move}	一局中的移动时间，单位为 s。
T_{switch}	一局中的频道切换时间，单位为 s。
T_{measure}	一局中的测向时间，单位为 s。
T_{locate}	一局中的光学定位时间，单位为 s。
T_{clear}	一局中的清除时间，单位为 s。
T_3	问题三单局总虚拟时间，单位为 s。
d_{move}	问题三单局累计移动距离，单位为 m。
v_{dog}	机器狗移动速度，本文取 5 m/s。
n_{meas}	一局中的测向次数。
n_{switch}	一局中的频道切换次数。
t_{meas}	单次测向耗时，本文取 5 s。
t_{switch}	单次频道切换耗时，本文取 1 s。
n_s	一局中被清除的干扰源数量。
n_{clear}	一局中调用清除接口的总次数。
t_{fail}	单次清除失败耗时，本文取 3 s。
t_{succ}	单次清除成功耗时，本文取 5 s。
$h(r)$	七点构型在环半径 r 下的最坏覆盖距离。
$r_{\min}$	七点构型满足 $h(r) \le L$ 的最小环半径。
$F_i(k)$	第 i 个频道选第 k 个入口时的最短前缀路线长度。
$S(q)$	候选测点 q 的距离型综合评分，单位为 m。
$p(z \mid \mathcal{O})$	与历史反馈 $\mathcal{O}$ 一致的源点粒子后验。
$\overline{R}(q)$	在候选点 q 测向后的预计定位圆半径，单位为 m。
λ	定位半径在评分中的无量纲权重。
C_{fail}	接收失败后返回保证测点的预计绕行距离，单位为 m。
$\varepsilon_{\parallel}$	近似平行判定的无量纲容差。
$\varepsilon_{\mathrm{hp}}$	半平面可行性判定容差，单位为 m。
$\varepsilon_{\mathrm{cov}}$	圆覆盖判定容差，单位为 m。
N_{run}	统计实验的总局数。
$T_{\mathrm{scan}}^{\mathrm{lb}}$	仅扫描全部保证点所需时间的下界，单位为 s。
N_{probe}	保证点巡视阶段实际执行的“保证点–频道”测量次数。

一局任务的总虚拟时间按式 (1)分解为五类耗时：

$$T = T_{\mathrm{move}} + T_{\mathrm{switch}} + T_{\mathrm{measure}} + T_{\mathrm{locate}} + T_{\mathrm{clear}}. \tag{1}$$

离线日志中移动时间占比最高，因而先满足发现与清除条件，再在可行方案中压缩 T；

定位范围直接作为后续路线的输入。

4 模型建立与求解

问题一、二先解决单源定位，问题三、四再安排覆盖点和访问路线。后两问以全部源被发现并清除为硬约束。令 $\mathcal{A}$ 为一局动作序列，$\mathcal{G}(\mathcal{A})$ 表示覆盖与凸包命中条件成立，$\mathcal{S}(\mathcal{A})$ 表示所有干扰源均已清除，则目标写为

$$\min_{\mathcal{A}} \ T(\mathcal{A}) \quad \text{s.t.} \quad \mathcal{G}(\mathcal{A}) = 1, \qquad \mathcal{S}(\mathcal{A}) = 1. \tag{2}$$

式(2)不允许以漏检换取更短时间，目标函数计入移动、切换、测量、定位和清除五类耗时。各问所用方法及对照方案列于表 3。

表 3 各问题模型与算法的选择依据

问题	主方法	对比方法	选择依据
问题一	半平面交、旋转卡壳与 MEC	最小二乘交会、矩形裁剪	保留连续误差带及空集、无界等状态，可分别检查定位直径和清除圆。
问题二	差分进化与 SHGO 相互复核	最大交会角、D-最优、E-最优	直接优化最坏清除圆，处理非光滑接收约束，并用连续角度搜索检查边界。
问题三	七点覆盖与滚动 GTSP	均匀网格、一次性 TSP、纯最近邻	覆盖条件可以证明；局部搜索规模小，每执行一步即可吸收新的测向和清除反馈。
问题四	双环凸包覆盖与滚动联合规划	单环采样、纯距离覆盖	同时处理未知辐射方向和接收距离，能把定向源的不漏检要求写成可检查的几何条件。

4.1 问题一：测向误差下的可行域、直径与覆盖判定

4.1.1 从方位误差到半平面交

采用平面右手直角坐标系，方位角从 x 轴正向起逆时针增加。程序先把角度归一化到 $[0, 2\pi)$，若误差区间跨越 0°，则按模 2π 的短弧处理。题设 $\delta = 1^\circ < 90^\circ$，每次测向对应一个凸扇形；下式只在 $0 < \delta < 90^\circ$ 的范围内使用。

设第 i 个测站为 $S_i = (x_i, y_i)$，测得方位角为 θ_i。定义 $\boldsymbol{d}(\varphi) = (\cos\varphi, \sin\varphi)$，二维叉积为 $\boldsymbol{a} \times \boldsymbol{b} = a_x b_y - a_y b_x$。点 X 落在误差扇形内，当且仅当

$$\boldsymbol{d}(\theta_i - \delta) \times (X - S_i) \geq 0, \qquad \boldsymbol{d}(\theta_i + \delta) \times (X - S_i) \leq 0. \tag{3}$$

两式均可改写为关于 $X=(x,y)$ 的线性不等式 $a_jx+b_jy\le c_j$，于是 n 次测向的公共可行域为

$$\mathcal{P}=\bigcap_{j=1}^{2n}\{(x,y):a_jx+b_jy\le c_j\}. \tag{4}$$

这是凸集，可能为空、无界、退化为点或线段，也可能是有界凸多边形。与人为设置大矩形后裁剪相比，式(4)保留了无界性信息，适合自动判断异常输入。

4.1.2 半平面交与退化判断

将每条边界定向为“可行侧在左侧”，按方向角排序，并用双端队列维护当前边界。加入新边界时，从队尾和队首删除不满足新约束的交点；处理完后进行首尾闭合清理。若最终约束矛盾，返回空集；若存在非零可行无穷方向，返回无界；否则相邻边界交点按逆时针排列成多边形。该方法的排序和扫描复杂度为 $O(N\log N)$，其中 $N=2n$，与前述半平面交算法一致。

4.1.3 直径与最小包围圆

对顶点按逆时针排列的凸多边形 $U_0,\dots,U_{n_P-1}$，凸集上的最远点可在顶点处取得。旋转卡壳只需沿边界同步推进对踵点，即可在 $O(n_P)$ 时间内得到

$$D=\max_{i,j}\|U_i-U_j\|_2,\qquad (U_a,U_b)=\arg\max_{i,j}\|U_i-U_j\|_2. \tag{5}$$

式 (5)由旋转卡壳在 $O(n_P)$ 时间内求得；随后用随机增量法计算顶点的最小包围圆 $\mathcal{B}(C^*,R^*)$[5]，以判断定位结果是否已经满足 20 m 光学清除要求。

4.1.4 “直径圆”覆盖判定

题意中的直径圆有两种可能解释。若圆的直径端点必须取最远点对 A,B，圆心 $M=(A+B)/2$，半径 $D/2$，只需检查所有顶点是否满足式 (6)

$$\|U-M\|_2\le D/2\quad\Longleftrightarrow\quad(U-A)\cdot(U-B)\le 0. \tag{6}$$

若圆心可以任意选择，则存在直径为 D 的覆盖圆当且仅当 $R^*\le D/2$。等边三角形给出反例：其直径为边长 D，但最小包围圆半径为 $D/\sqrt{3}>D/2$，故不能由直径为 D 的圆覆盖。程序同时输出两种判定，避免把“最远点对圆”和“任意圆心圆”混为一谈。

两种判定的区别见图 2。

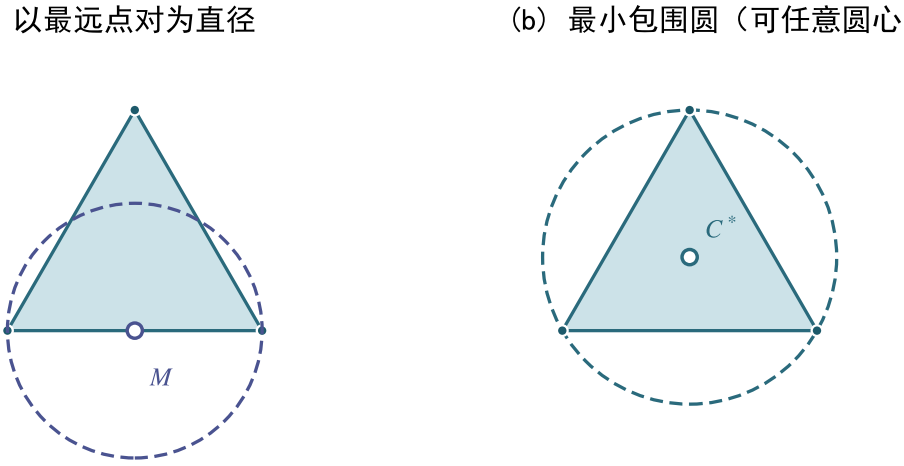


图 2 直径圆与最小包围圆的覆盖比较

4.1.5 敏感性与算法可靠性

在两测站到目标距离均为 1000 m、测角误差为 1° 时，交会角越小，定位直径越大（表 4）。以 d_s 表示测站至目标的距离，对称正交构型可推出

$$D = \frac{2\sqrt{2}\, d_s \tan\delta}{1-\tan^2\delta} \approx 2\sqrt{2}\, d_s \delta, \tag{7}$$

其中 δ 取弧度，说明固定测角误差下定位直径近似与测站距离成正比。

表 4 交会角对定位直径的影响（$d_s = 1000$ m，$\delta = 1°$）

交会角/°	3	5	10	20	45	60	90
直径/m	2400.03	952.59	417.15	203.03	91.39	69.88	49.39

图 3把上述关系画成三条曲线，并标出 20 m 清除半径对应的直径阈值。

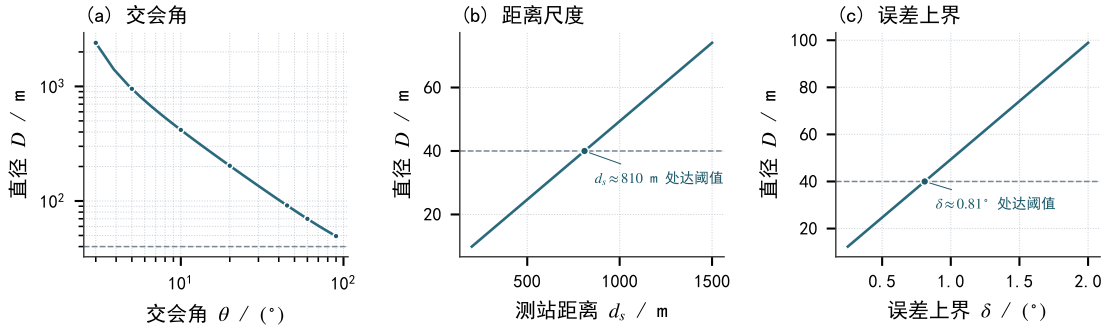


图 3 定位直径的参数敏感性

4.1.6 正确性、退化情形与数值实现

命题 1（误差带等价性）. 对任意测站 S_i，满足式(3)的点集恰为方位角落在 $[\theta_i - \delta, \theta_i + \delta]$ 内的闭扇形。证明如下：叉积符号给出向量 $X - S_i$ 相对两条边界射线的左右

关系；两条不等式同时成立当且仅当向量位于两射线之间。由于边界取闭集，误差恰为 δ 时仍被保留。对多个测站取交集即得式(4)，故算法不会因把角度离散化而漏掉边界源点。

命题 2（直径计算）. 有界凸多边形的直径在顶点对中取得。若某个最远点位于边的内部，则沿该边方向移动至端点不会减小其与另一点的距离，故可将最优点推到顶点；对所有顶点对应用旋转卡壳的对踵点遍历即可在线性时间内求得直径。该证明也解释了为何不直接在多边形内部做网格采样。

实现时采用三类容差：平行判定 $\varepsilon_{\parallel} = 10^{-10}$、半平面可行性 $\varepsilon_{\mathrm{hp}} = 10^{-8}$ m、圆覆盖判定 $\varepsilon_{\mathrm{cov}} = 10^{-6}$ m。容差只用于浮点比较，不改变输入角度和接收半径。表 5列出自动化测试覆盖的边界类型；每类均同时由双端队列半平面交和独立的顺序裁剪器复核。

表 5 问题一退化与异常输入的处理策略

状态	判定条件	输出与降级动作
空集	加入一条约束后双端队列首尾矛盾	返回状态码 `EMPTY`，不进入清除阶段
无界	存在共同可行方向，边界交点无法闭合	返回 `UNBOUNDED`，限制到题目圆盘后再计算
点/线段	顶点面积低于 10^{-8}	直接以端点直径和 MEC 半径作为定位结果
近似平行	方向差小于 $\varepsilon_{\parallel}$	保留更严格的半平面，记录被合并约束编号
重复测向	同站同角度重复出现	不重复增加几何边界，但保留测量次数用于时间统计

这些状态会改变下一步动作：`EMPTY` 触发重新测量，`UNBOUNDED` 先与目标圆盘求交，`DEGENERATE` 转入近距离光学定位。异常结果不会继续作为普通坐标参与路线计算。

4.2 问题二：必接收第二测点的鲁棒选择

4.2.1 局部坐标与必接收候选域

以第一次测站为原点，以测得方位为前向轴，设第二测点为 $Q = (a, b)$。第一次测量返回 `direction` 时，源点属于

$$G(\rho, \phi) = (\rho\cos\phi, \rho\sin\phi), \quad 5\ \mathrm{m} < \rho \le 1500\ \mathrm{m}, \quad |\phi| \le \delta. \tag{8}$$

第二测点须在最坏情况下仍能接收信号，即

$$\max_{G \in \mathcal{K}} \|Q - G\|_2 \le L, \qquad \mathcal{K} = \{G(\rho,\phi)\ \text{满足式(8)}\}. \tag{9}$$

距离关于 ρ 的最大值出现在区间端点，关于 ϕ 出现在角端点或与 Q 相反的方向，可用有限个极值场景精确检查式(9)，而不依赖粗糙网格。

必接收候选域与两侧推荐第二测点见图 4。

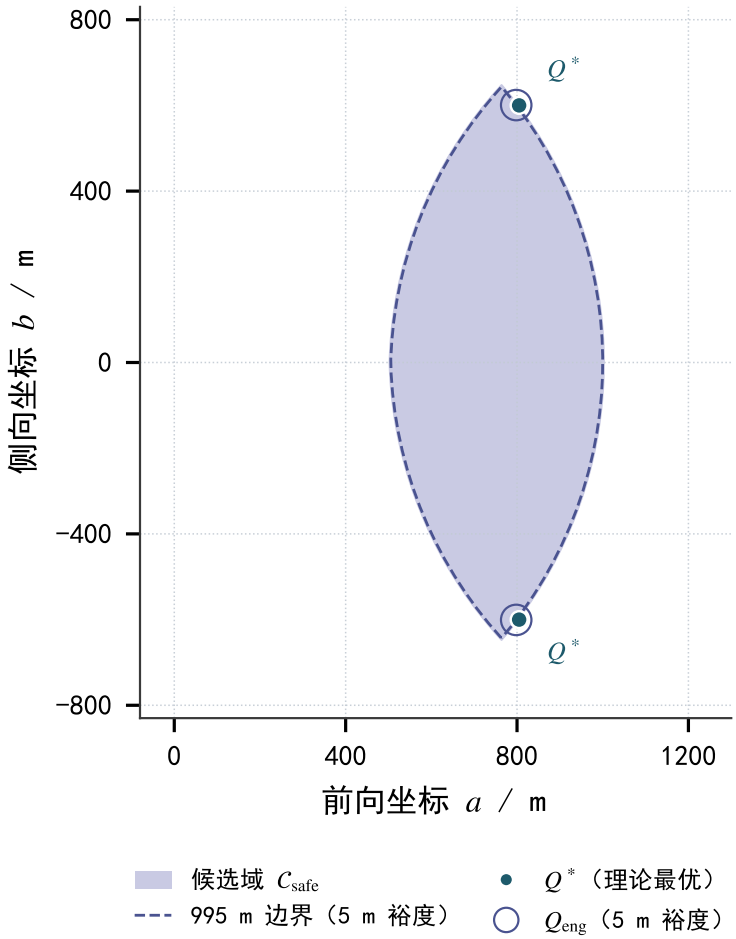


图 4 必接收候选域与推荐测点

4.2.2 以最坏清除圆为目标

只最大化交会角会忽略接收约束：第二测点过远可能失去最小接收半径为 1000 m 的保证，而清除动作真正关心的是定位集合的最小包围圆。对每个候选 Q，令 $\theta_i(G,e_i)=\operatorname{atan2}(G-S_i)+e_i$，其中 $|e_i|\le\delta$，并记由两次带误差方位得到的可行域为 $\mathcal{P}(\theta_1,\theta_2)$。对 $\mathcal{K}$ 中的角点场景和连续对抗场景求半平面交，定义

$$J(Q)=\max_{\substack{G\in\mathcal{K}\\ |e_1|,|e_2|\le\delta}} R^*\big(\mathcal{P}(\theta_1(G,e_1),\theta_2(G,e_2))\big),\qquad Q^*=\arg\min_{Q\in\mathcal{C}_{\text{safe}}} J(Q), \tag{10}$$

其中 $\mathcal{C}_{\text{safe}}$ 为满足式(9)的安全候选域，式 (10)即问题二的极小极大目标。由于清除条件是定位圆半径不超过 20 m，直接优化 R^* 比优化交会角更贴近题意。外层采用差分进化 [6]，再用 SHGO[7]、连续角度搜索和活跃约束的一维扫描核对结果。

4.2.3 求解步骤与收敛检验

求解时先处理接收约束，再计算定位目标。差分进化在矩形 $a \in [0,1000], b \in [-800,800]$ 内产生第二测点候选。对每个候选点，先计算式 (9)左端的最远距离；超过 L 的候选按越界量平方加罚，不再进入耗时较高的定位计算。只有候选点满足必接收条件时，才对源位置和测角误差构造对抗场景，逐一求半平面交和最小包围圆，并把其中最大的 R^* 作为 $J(Q)$。

内层场景先检查扇形的径向端点与角端点，再把偏角 ϕ 作为连续变量搜索内部极值，最后沿当前活跃圆弧做一维加密。这样安排是因为粗网格适合快速排除普通场景，而最坏值常出现在接收边界或几何约束切换处，只靠固定网格可能低估 $J(Q)$。

差分进化设置种群 36、迭代 160 代、交叉率 0.80、缩放因子 0.70，独立运行 5 次；另用 SHGO 复核 1 次。连续对抗搜索使用 12 个初值、80 步并重复 3 次，活跃圆弧先以 1° 步长扫描，再用黄金分割细化。五次差分进化得到的最坏半径极差为 3.9×10^{-8} m，SHGO 给出相同的五位小数结果；最优点均成对出现在 $b \approx \pm 600.05$ m 处。将活跃参数 $\tau = 36.879883^\circ$ 向两侧改动 $0.5^\circ, 1^\circ, 2^\circ$ 后，目标半径均增大，说明该解并非一次随机搜索产生的孤立低值。

多算法复核的一致性见图 5。

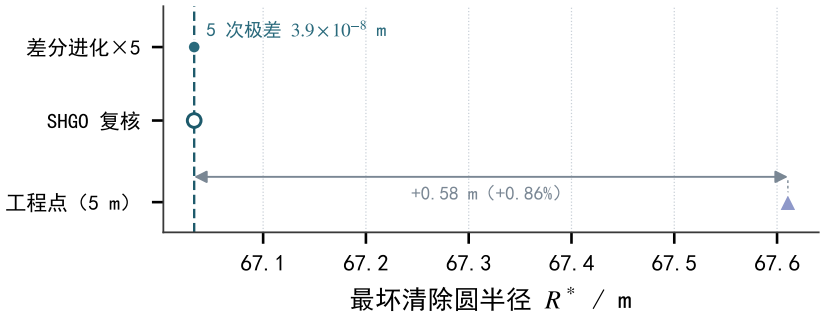


图 5 问题二数值解的一致性检验

4.2.4 数值结果、算法比较与工程点

上述求解过程得到

$$Q^* = (804.895, \pm 600.052)\ \text{m}, \qquad J(Q^*) = 67.032926\ \text{m}, \tag{11}$$

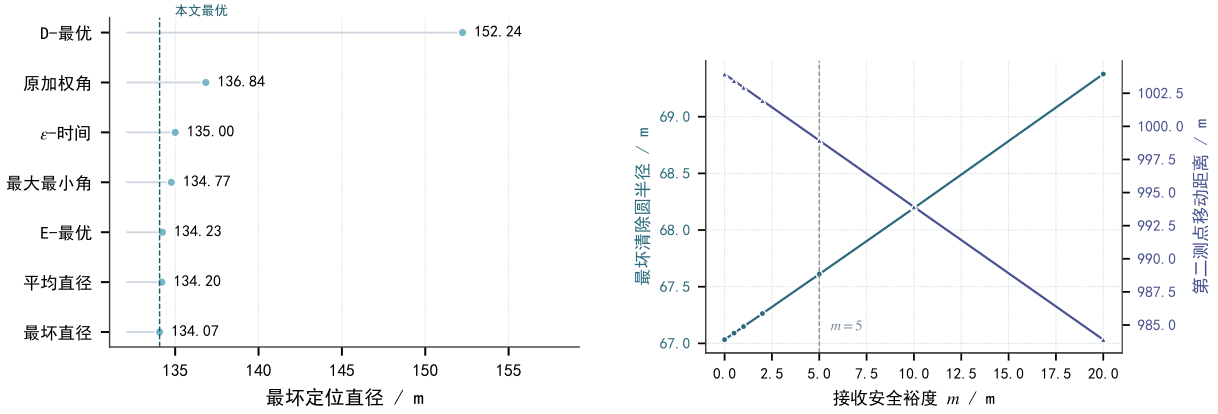
对应最坏定位直径 134.065852 m。b 的正负解关于首次测向中心线对称，二者目标值相同，实际使用时可选择离当前路线较近的一侧。最优点处有两个限制同时起作用：继续向外移动虽能增大部分场景的交会角，却会逼近近端源的接收边界；向内移动则会扩大远端源的定位区域。因此，该点不是 90° 交会点，而是接收可靠性与最坏定位范围平衡后的结果。

不同目标函数的最坏定位直径对比见表 6。

目标函数对比与接收裕度折中见图 6。

表 6 问题二不同目标的最坏定位直径比较

目标函数	最坏直径/m	解释
原加权角	136.838	同时兼顾角度与距离，但并非针对最坏清除圆
最大最小角	134.771	只优化交会角，忽略近距离必接收约束
D-最优	152.237	最大化近似信息矩阵行列式，未直接控制最坏定位直径
E-最优	134.235	最大化近似信息矩阵的最小特征值，仍未直接约束最坏圆
平均直径最小化	134.199	以均值最优代替最坏情形，尾部风险更大
ε-时间折中	135.000	用少量精度换移动距离，最坏直径略升
最坏直径最小化（本文）	**134.066**	直接最小化最坏清除圆对应的定位直径



(a) 目标函数对比　　**(b)** 移动距离–定位精度折中

图 6 问题二算法选择与工程折中

纯数学最优点的接收约束几乎贴在 1000 m 边界上。它是在原始约束下求得的理论解，并非实际部署点。加入安全裕度 m 后，工程约束改为

$$\max_{G\in\mathcal{K}} \|Q-G\|_2 \le L-m.$$

计算中取 $m=5$ m。该数值不是由样本拟合得到，而是在缺少平台定位误差统计时人为保留的保守量，用于隔开接收临界点和坐标舍入、浮点判定造成的微小偏差；若正式接口能给出坐标精度，应据此重新设定 m。在 $m=5$ m 下得到工程点

$$Q_{\mathrm{eng}} = (798.078, \pm 600.788)\ \mathrm{m}, \tag{12}$$

最坏半径为 67.610529 m，仅比纯最优增加 0.86%，而移动距离减少 5.01 m。故问题二分别报告理论点 Q^* 和工程点 Q_{eng}，问题三、四只使用后者，并在在线规划中根据当前位置选择左右对称一侧。上述六位小数用于复核数值优化，不代表传感器达到同等测量精度；实际下发坐标按接口允许精度取舍。

表 7 接收安全裕度的敏感性

裕度 m/m	0	1	5	10	20
前向坐标 a/m	804.895	803.531	798.078	791.276	777.699
侧向坐标 $\lvert b\rvert$/m	600.052	600.203	600.788	601.468	602.686
最坏半径/m	67.033	67.148	67.611	68.194	69.377
移动距离/m	1003.951	1002.948	998.937	993.922	983.893

接收安全裕度的敏感性见表 7。

表 6中的差异说明，目标函数会直接改变测点位置。最大交会角只描述两条中心射线的夹角；D-最优和 E-最优依赖局部协方差近似，没有直接约束误差扇形交集的覆盖圆；平均直径目标还可能用多数普通场景掩盖少数较差场景。本文采用 R^{*} 作为目标，是因为它和 20 m 清除条件使用同一几何量。理论最优点随后向必接收域内部移动 5 m，用少量定位精度换取接收边界附近的实施裕度。

4.3 问题三：全向源的覆盖–定位–路径联合优化

问题三需要同时回答 “能否发现全部源” 和 “如何少走路”。本文先构造保证点，使机器狗即使不知道源数和频道分布，也能在完整巡视后发现所有全向源。巡视过程中一旦某个频道返回方位，便立即利用后续线段寻找顺路复测点；若定位区域已缩小到可清除范围，则把清除位置加入待办任务。未访问保证点、复测任务和清除任务随后统一交给滚动路线规划。每次只执行当前路线的第一个动作，收到反馈后重新计算余下部分。这样，几何覆盖负责不漏检，滚动规划负责减少重复路程，二者的作用不会混在一起。完整动作顺序见附录 A。

4.3.1 七点确定性覆盖

全向源只要与检测点距离不超过 1000 m 即可被发现。取圆心和半径 r 的正六边形六个顶点。由六折对称性，最坏覆盖距离为

$$h(r)=\max\left\{\frac{r}{2\cos 30^{\circ}},\sqrt{R^{2}+r^{2}-2Rr\cos 30^{\circ}}\right\}. \tag{13}$$

令 $h(r)\le L$ 得

$$r_{\min}=R\cos 30^{\circ}-\sqrt{L^{2}-R^{2}\sin^{2}30^{\circ}}=1122.955832\ \text{m}. \tag{14}$$

式 (13)可由一个 60° 基本扇区推出。正六边形内部只需检查圆心与相邻两个环点围成的等边三角形，其外心到三个检测点的距离为 $r/(2\cos 30^{\circ})$；六边形外部的最远位置在目标圆边界与扇区中线的交点处，到最近环点的距离为式 (13)第二项。两部分的最大值不超过 L，才能覆盖整个圆盘。分别解这两个不等式并取可行区间的最小端点，即得式 (14)。因此，$r_{\min}$ 是正六边形七点构型下的最小可行环半径，不是把式 (13)两项强行取等得到的数值。

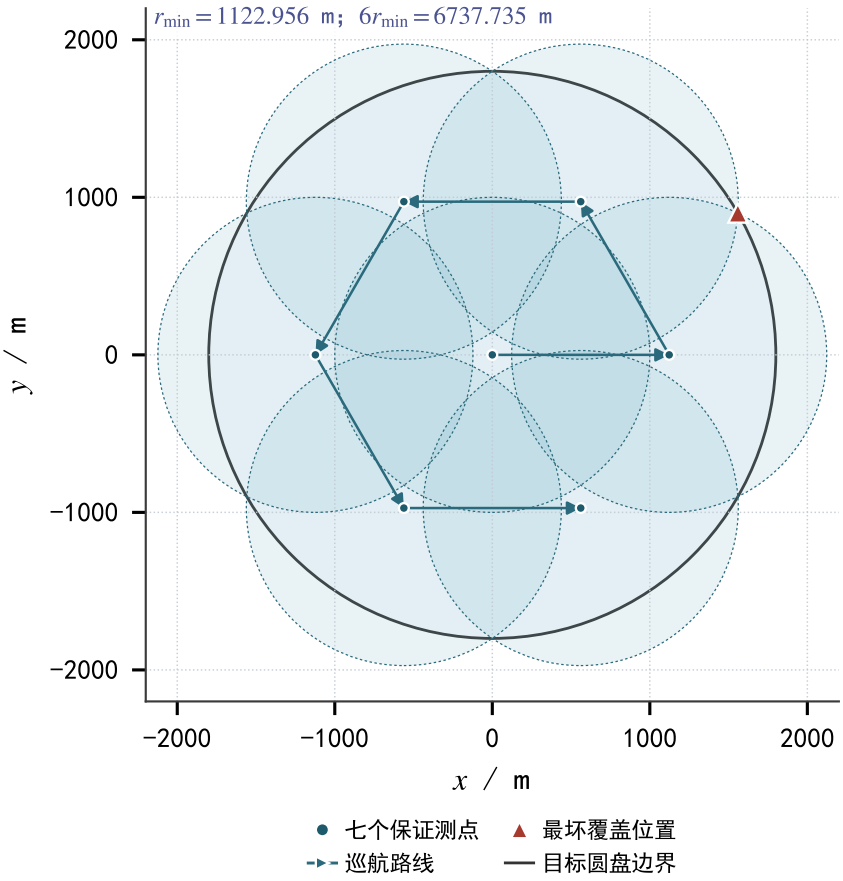


图 7 全向源的七点覆盖构型

从圆心出发依次访问六个环点时，开放巡视路线长度为

$$d_{\text{tour}} = 6r_{\min} = 6737.735 \text{ m}.$$

该式计算的是距离，换算移动时间时还要除以机器狗速度。式 (14)只给出正六边形七点构型中的最小环半径，不据此断言任意布点下七点都是全局最少。

七点覆盖构型与最坏覆盖位置见图 7。

4.3.2 顺路第二测向

机器狗在保证点 A 获得首次后，下一段原定路线为 $A \to B$。若立即转向问题二的专用工程点，定位质量较稳定，但会增加一次往返。为利用原本就要经过的路段，先在线段上写出候选点 $P(t) = (1-t)A + tB$，其中 $t \in [0,1]$。候选点必须同时满足三项条件：距所有可能源点不超过 $L - 5$ m，保证第二次接收；两次中心方位的夹角不低于 $10°$，排除近似平行交会；重算后的最坏包围圆半径不超过 110 m，避免把过大的定位区域留给光学搜索。每段均匀取 41 个点，先用前两项快速筛选，再对不超过 12 个候选计算完整半平面交和 MEC。若选中的点位于原线段上，则

$$|AP(t)| + |P(t)B| = |AB|, \tag{15}$$

故把复测插入该线段不会改变移动距离，只增加一次 5 s 测量。若整条线段没有合格点，再调用问题二的工程点。这个回退规则保证顺路策略只减少路程，不以牺牲接收和定位

条件为代价。

顺路测点的筛选见图 8。

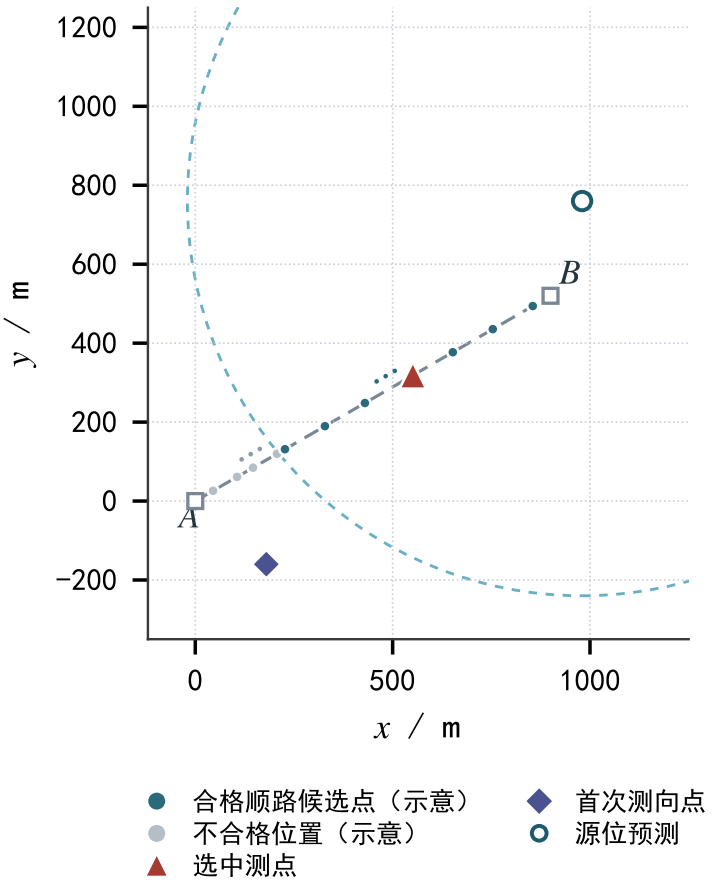


图 8 顺路第二测向的候选点筛选

4.3.3 非顺路测点的粒子评分

顺路筛选失败后，需要从二维区域内另选测点。此时若只取定位可行域的几何中心，虽然位置直观，却没有考虑机器狗当前位置、候选源点的分布以及测点失去信号后的绕行。本文用与全部历史反馈一致的粒子集合近似源点的可能分布。粒子只用于比较候选测点和路线，不改变问题一得到的完整半平面可行域。

对候选点 q 计算

$$S(q)=\underbrace{\|X_t-q\|}_{\text{到达}}+\underbrace{\mathbb{E}_{z\sim p(z\mid\mathcal{O})}\|q-z\|}_{\text{预计后续移动}}+\underbrace{\big(1-P(\text{可接收}\mid q)\big)\,C_{\text{fail}}}_{\text{失联代价}}+\lambda\,\overline{R}(q),\tag{16}$$

其中第一项是从当前位置到测点的路程，第二项近似复测后靠近源点所需的距离，第三项惩罚可能失去信号的测点，第四项控制预计定位范围。C_{fail} 按失联后返回保证测点的绕行距离估计，λ 无量纲，因此四项均为距离量，可以直接相加。

候选点由粒子均值附近的稠密网格和“当前位置–预测源点”连线上的插值点组成。先按接收概率粗筛，再在得分最低的 4 个候选邻域内作 ± 80 m 精筛。最初该评分只用于恰有一次测向的频道；离线日志表明，两次测向后虽然定位区域已经缩小，服务入口

仍会影响后续路线，故最终把适用范围扩展到不超过两次测向的频道。粒子数量有限可能影响候选排序，但不会删去七点保证点，也不会缩小半平面交，因而不影响最坏情况下的发现与定位保证。

4.3.4 滚动广义旅行商

得到各频道的候选入口后，还需决定访问顺序。只完成一次测向的频道通常有左右两个复测入口，完成两次测向的频道则有一个或多个清除入口；同一频道只需选择其中一个服务分支。这正是广义旅行商问题（GTSP）[8]。若每次反馈后都精确求解 GTSP，任务集合尚未稳定时会反复求解一个很快就会变化的问题；纯最近邻计算快，却容易产生交叉边和局部折返。考虑到每次规划的频道数不超过 20，本文采用“贪心初解–2-opt–Or-opt(1)–入口动态规划”的组合：距离贪心先给出可行次序，2-opt 消除交叉边，Or-opt(1) 移动位置不当的单个任务；在任务次序固定后，再用动态规划从每组入口中选出路程最短的一支：

$$F_i(k) = \min_j \{F_{i-1}(j) + \|X_{i-1,j} - Q_{ik}\| + \|Q_{ik} - X_{ik}\|\}. \tag{17}$$

其中 Q_{ik} 和 X_{ik} 分别是第 i 个频道第 k 个服务分支的入口与出口；$F_i(k)$ 及右端各距离项均以米计。一次规划只决定当前最先执行的动作。动作完成后，当前位置、未访问保证点和频道状态都可能改变，程序据此重新建立剩余 GTSP。这样做不追求一次得到整局的全局最优路线，而是避免旧计划忽略新方位、`no_signal` 或清除失败。覆盖路径规划和移动机器人规划中的相关方法可参见 [9]。

为使几何计算与路线更新使用同一组信息，记第 t 步结束后的状态为

$$\mathcal{Z}_t = (X_t, V_t, \{\mathcal{P}_{c,t}, s_{c,t}\}_{c\in\Gamma}), \qquad \mathcal{Z}_{t+1} = \Phi(\mathcal{Z}_t, a_t, y_t), \tag{18}$$

其中 $\Gamma = \{1, \ldots, 20\}$ 为全部频道，X_t 是当前位置，V_t 是尚未访问的保证点，$\mathcal{P}_{c,t}$ 和 $s_{c,t}$ 分别记录频道 c 的定位区域与处理阶段。处理阶段依次取“未检出、已示向、可清除、兜底、已清除”，C_t 则由已检出但尚未清除的频道组成。执行动作 a_t 并收到返回值 y_t 后，转移函数 Φ 只更新受该动作影响的状态。

具体地，移动动作更新 X_t；到达保证点后将该点从 V_t 删除；测向动作把新的误差带与 $\mathcal{P}_{c,t}$ 求交。若更新后 $R^*(\mathcal{P}_{c,t}) \le r_c$，下一服务入口取最小包围圆圆心并安排清除，否则继续生成复测入口。清除成功后，该频道改为“已清除”；失败则保留原定位区域并转入兜底状态。算法随后重建剩余路线。当 V_t 和 C_t 同时为空时，既没有未巡视区域，也没有待处理频道，一局任务结束。

4.3.5 单源补充测量与异常处理

获得至少两次方位后，按式 (4)更新定位多边形。若其最小包围圆半径不超过 r_c，圆内任一可能源点都距圆心不超过清除半径，可直接前往圆心清除；若半径仍然过大，则在当前定位区域附近补充测量，每得到一条新方位都重新求交，最多补测 5 次。

第二测点无信号时，原方位并未失效，但不能再假设该测点处于接收范围内；半平面交退化时，也不宜把数值不稳定的圆心交给路线规划。两种情形均保留已有误差带，

并在其局部坐标系中使用旋转蛇形网格搜索。网格步长不超过 $r_c\sqrt{2}$，使每个网格单元都被某个半径 r_c 的清除圆覆盖。清除失败时从相邻网格点继续。兜底过程耗时较长，但只改变当前频道的状态，其他频道和未访问保证点不被删除。

4.3.6 时间目标的展开

前述规则最终都要通过总虚拟时间评价。对问题三，一局耗时可以按动作次数展开为

$$T_3 = \underbrace{\frac{d_{\mathrm{move}}}{v_{\mathrm{dog}}}}_{\text{移动}} + \underbrace{t_{\mathrm{meas}} n_{\mathrm{meas}} + t_{\mathrm{switch}} n_{\mathrm{switch}}}_{\text{测向与频道切换}} + \underbrace{T_{\mathrm{locate}}}_{\text{光学定位}} + \underbrace{t_{\mathrm{fail}}(n_{\mathrm{clear}} - n_s) + t_{\mathrm{succ}} n_s}_{\text{清除}}, \tag{19}$$

其中 $v_{\mathrm{dog}} = 5$ m/s，$t_{\mathrm{meas}} = 5$ s，$t_{\mathrm{switch}} = 1$ s，T_{locate} 按实际光学定位动作累计。按附件 1 表 1，清除失败和成功分别耗时 $t_{\mathrm{fail}} = 3$ s 和 $t_{\mathrm{succ}} = 5$ s。七点完整巡视至少移动 $6r_{\min} = 6737.735$ m，因此移动项本身已有约 1347.55 s 的固定成本。顺路复测、滚动 GTSP 和粒子评分都不减少保证点，而是尽量让复测与清除共用这段必经路线；参数选择和消融实验也围绕式 (19)的各项展开。

4.3.7 参数与消融实验

参数分为几何保证参数和效率参数。七点环半径由式 (14)直接确定，5 m 接收裕度沿用问题二的工程设置；这两项不通过随机实验调节。顺路交会角、定位半径上限、候选点数、粒子数和权重只影响路线效率，先在小批量场景中确定可用范围，再用未参与选择的种子做参数扫描。采用的数值见表 8。

表 8 问题三主要参数

参数	数值	作用
七点环半径	1122.955832 m	覆盖条件下的下界
顺路接收裕度	5 m	抑制边界失效
顺路最小交会角	10°	排除近似平行交会
顺路最坏半径上限	110 m	控制后续细化成本
每段候选点数	41	在线计算量与精度折中
预计源距离	625 m	仅用于路径排序
测点不确定度权重 λ	0.4	平衡到达路程与定位圆半径
粒子数	48	后验采样规模
测点寻优作用域	≤2 次测向	覆盖首测点与二次测向入口

逐级消融结果见表 9。

消融顺序与实际求解过程一致。先加入七点覆盖和顺路复测，单源时间由 438.93 s 降至 346.51 s，说明避免专程往返已经产生主要收益；再把巡视与频道任务合并为滚动

表 9 问题三算法结构的逐级消融（60 局同批种子，共 764 个源全部清除）

方案（逐级加入）	单源时间/s	平均移动/m	平均检测	平均清除尝试
两阶段最近邻（基线）	438.93	24348.6	100.88	12.73
七点覆盖＋顺路复测	346.51	18416.4	100.72	12.73
＋滚动联合路线	303.04	15674.7	100.28	12.73
＋归航细化与提前清除	264.26	12951.2	113.18	12.73
＋粒子后验测点	243.20	11775.9	103.22	16.38
＋测点作用域扩展（完整方案）	**237.56**	**11476.9**	101.32	15.80

路线，时间降至 303.04 s。归航细化和提前清除减少了已定位频道长期滞留在任务队列中的情况，虽然测向次数有所增加，移动距离仍继续下降。最后加入粒子评分并扩大其适用范围，时间降至 237.56 s。各阶段都清除了同一批 764 个源，因此这些差异可以解释为路线与测点规则的影响，而不是成功样本数量变化造成的偏差。

参数敏感性见图 9。四条曲线分别对应顺路最小交会角 $5°, 10°, 15°, 20°$。默认参数 $(10°, 110\ \mathrm{m})$ 位于较平坦的区域，局部扰动不会明显改变单源时间。

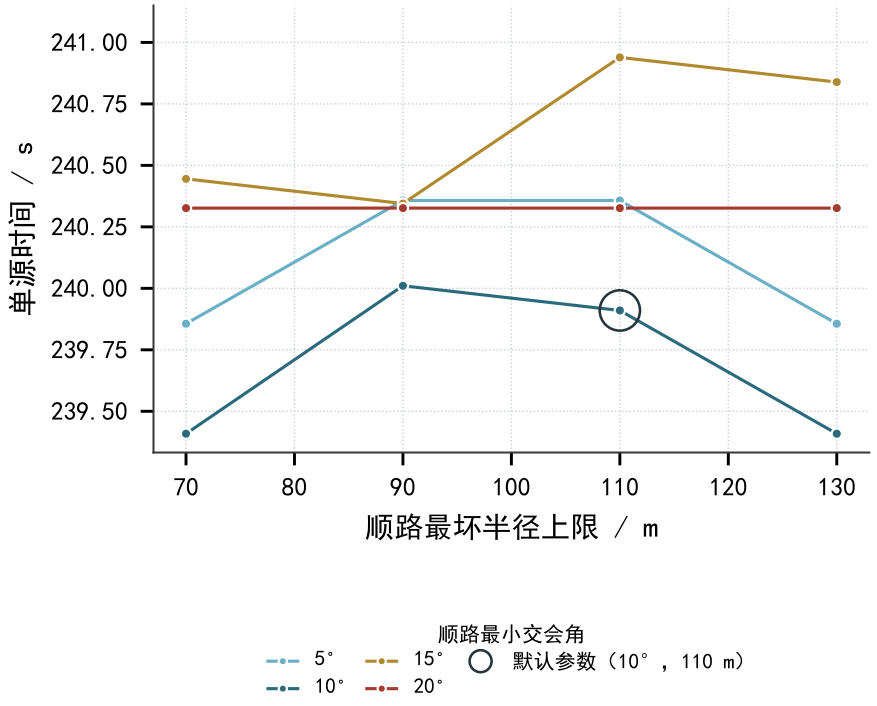


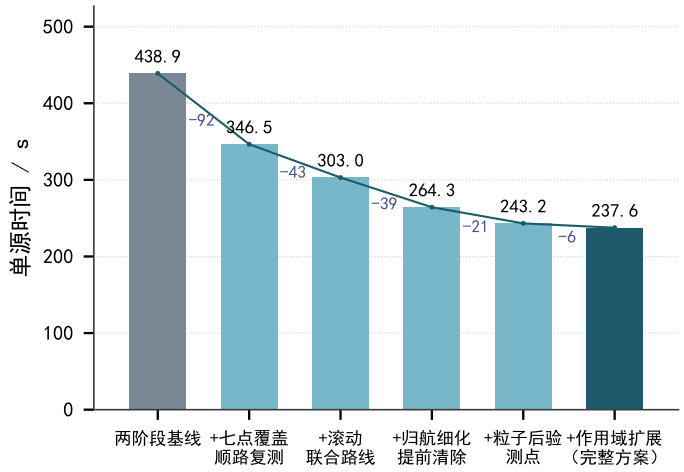
图 9 问题三参数敏感性

表 10使用同一批随机种子分别运行两种方案，使每一局的源位置和频道组成完全相同。只有两种算法都清除全部源时才比较时间；失败样本不会从统计中删去。

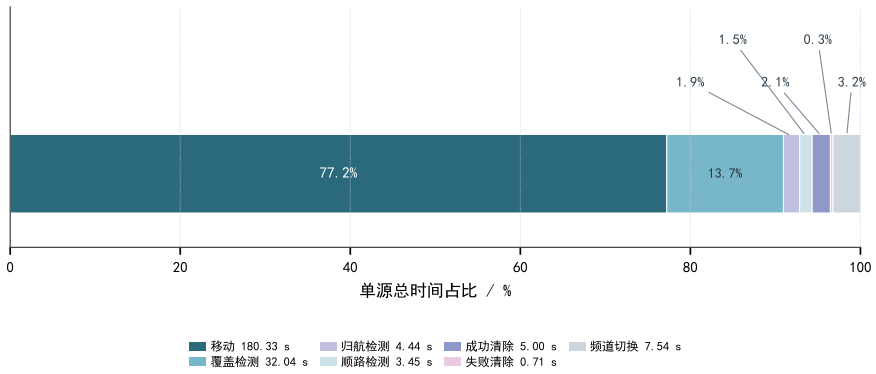
逐级消融与本文方案的动作时间结构见图 10。

表 10 问题三 500 局配对实验（同一批种子，6415 个源全部清除）

方案	单源时间/s	中位数/s	P95/s	最坏/s	均值移动/m	最坏移动/m	平均检测
两阶段基线	441.51	438.41	518.26	568.55	24638.1	32460.6	102.05
本文方案	**238.97**	**236.28**	**293.99**	**318.89**	**11633.2**	**15065.1**	101.94
相对变化	-45.87%	-46.10%	-43.27%	-43.91%	-52.78%	-53.59%	-0.11%



(a) 结构消融阶梯



(b) 本文方案的时间结构

图 10 问题三：逐级消融（60 局）与本文方案的动作时间结构（200 局）

在 500 局配对实验中，两种方案均清除全部源（6415 个源）。本文方案把单源时间由 441.51 s 降至 238.97 s （−45.87%），均值移动距离由 24638.1 m 降至 11633.2 m（−52.78%），最坏单源时间由 568.55 s 降至 318.89 s；500 局配对差全部为负（500 胜 0 负）。检测次数基本不变（102.05 与 101.94 次），说明改进来自路线缩短与测点后验选择，而不是少测频道。原始数据、绘图程序与逐局日志一并保存在支撑材料中。

4.3.8 模拟器演练测试统计（按题面口径）

题目要求通过模拟器演练测试给出“被清除干扰源个数的比例”与“平均定位清除时间”。表 11 汇总三批演练的原始日志，三批均使用同一套几何保证，差别只在测点选择策略。

表 11 问题三演练测试统计（原始日志：代码/logs）

方案	局数	源数	清除数	比例	单源时间/(s/源)	最差局/(s/源)
稳健版（七点 + 顺路 + 滚动 GTSP）	22	299	299	100%	329.23	460.58
竞速版（+ 粒子后验首测点）	10	125	125	100%	249.70	332.53
本文方案（+测点作用域扩展）	5	66	66	100%	**219.27**	255.98

表中“平均定位清除时间”按题面定义计算，即各局定位清除总时间之和除以被清除干扰源总数；若改为“每局先求单源时间、再对局等权平均”，三批分别为 332.69、254.76 与 221.86 s/源，中位数分别为 328.23、243.63 与 213.50 s/源。源数越多，七点覆盖与频道扫描的固定成本被摊得越薄，低源数局的单源时间明显偏高（最低为 10 源局）。

正式阶段又独立运行 3 次，分别清除 16、12 和 11 个干扰源，单源时间为 205.11、230.02 和 267.68 s，按 39 个已清除源汇总为 230.43 s/源，程序墙钟时间均不超过 2.08 s。正式结果未再用于调参，因此可作为离线与演练结果之外的一组独立记录；各次测试的原始统计见表 23。

三批演练的逐源结果见图 11。每个点表示同一局内相邻两次成功清除之间的虚拟时间；每局首个点还包含约 1700–2250 s 的初始巡视成本，因此超出图中的纵轴范围。

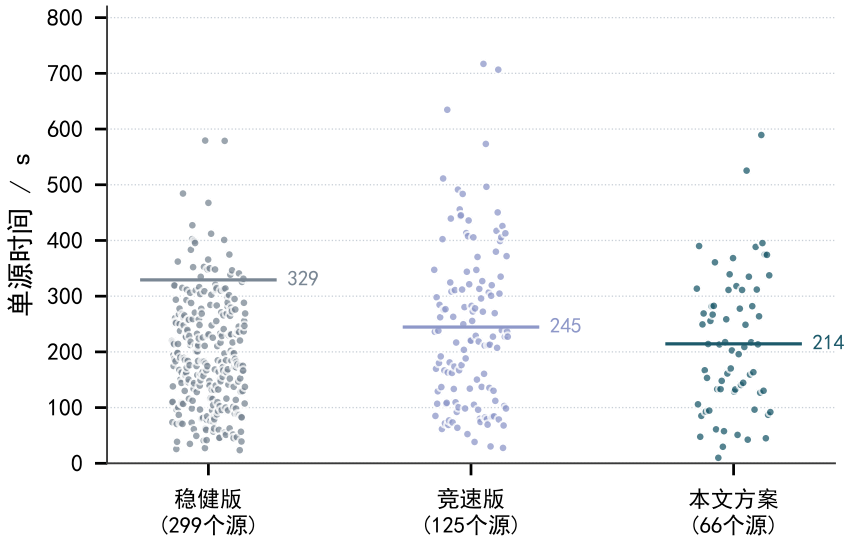


图 11 问题三三批演练的逐源边际清除时间

4.4 问题四：全向/定向混合源的保证发现与滚动清除

问题四保留问题三“保证点巡视–频道定位–滚动清除”的执行方式，但不能直接沿用七点构型。全向源只需有一个保证点落入接收圆，定向源还要求该点位于辐射半平面内；辐射方向未知时，一个邻近点无法提供这种保证。因此，先把方向命中要求转化为局部凸包条件，再按该条件搜索新的保证点布局。布局确定后，测点候选和滚动路线仍按问题三的方法生成；若第二次测向因背向遮蔽而无信号，则保留第一次误差带，在其内部启动局部光学搜索。具体状态转移见附录 A。

4.4.1 定向源的凸包命中定理

设距候选源点 G 不超过 L 的保证点集合为 V_G。定向源的辐射区域是边界经过 G 的某个闭半平面。若 $G \in \mathrm{conv}(V_G)$，则无论该半平面朝向何处，其中至少含有一个 V_G 中的点。证明可用反证法：若存在一个辐射半平面不含任何保证点，则全部 V_G 都位于其对侧的开半平面内；该开半平面是凸集，V_G 的任意凸组合也位于其中，不可能等于边界上的 G。反之，若 G 不在凸包内，由凸集分离定理可以找到一条过 G 的直线，使所有邻近保证点严格落在同一侧；把另一侧取为辐射方向，即得到一个无法接收的朝向。

所以，问题四的发现条件可以写成：对每个 $G \in \Omega$，先取 $L = 1000$ m 邻域内的保证点，再检查 $G \in \mathrm{conv}(V_G)$。距离限制保证这些点处于最小接收半径内，凸包限制排除了所有保证点同时落在背向一侧的可能。相关的方向传感器覆盖问题见 [10]。

4.4.2 25 点双环保证布局

为兼顾圆心附近和圆盘边界，先考虑“中心点 + 内环 + 外环”的对称布局。内、外环各放 12 点，角度间隔均为 30°；外环相对内环偏转 φ，以避免两环点落在同一径向线上。待选参数为内环半径 r_{in}、外环半径 r_{out} 和偏角 φ。对每组参数，在目标圆盘上逐点建立 V_G，先检查距离，再计算 G 到局部凸包各支持边的最小角裕度。只有最小裕度为正的布局才进入路线长度比较。

粗网格筛选后，最短的稳定可行区域出现在 $(r_{\mathrm{in}}, r_{\mathrm{out}}, \varphi)$ 约为 $(950\ \mathrm{m}, 1875\ \mathrm{m}, 15^\circ)$。据此采用中心 1 点、内环 12 点和外环 12 点，共 25 点，其坐标写为

$$\begin{aligned} P_0 &= (0\ \mathrm{m}, 0\ \mathrm{m}), \\ P_k^{\mathrm{in}} &= (950\ \mathrm{m})(\cos(30^\circ k), \sin(30^\circ k)), \\ P_k^{\mathrm{out}} &= (1875\ \mathrm{m})(\cos(15^\circ + 30^\circ k), \sin(15^\circ + 30^\circ k)). \end{aligned} \tag{20}$$

其中 $k = 0, \ldots, 11$。按径向 5 m、角度 0.25° 加密检查，共检验 518401 个采样源点；所有采样点均满足凸包条件，最小角裕度为 2.623°。为判断能否少用一个点，又以相同口径检查中心 + 11 + 12、中心 + 10 + 13 和无中心 12 + 12 三类 24 点对称布局，所得最小角裕度分别为 -10.695°、-3.734° 和 -20.909°。负值表示存在一条分离直线，因而能构造出漏检朝向；这些 24 点方案不能仅凭路线较短进入时间比较。25 点布局见图 12。

外环半径略大于目标圆盘半径并非作图上的外扩，而是由边界源点的凸包条件决定。若外环恰取 1800 m，靠近边界的源点只能看到向圆内收缩的邻点集合，容易落在其

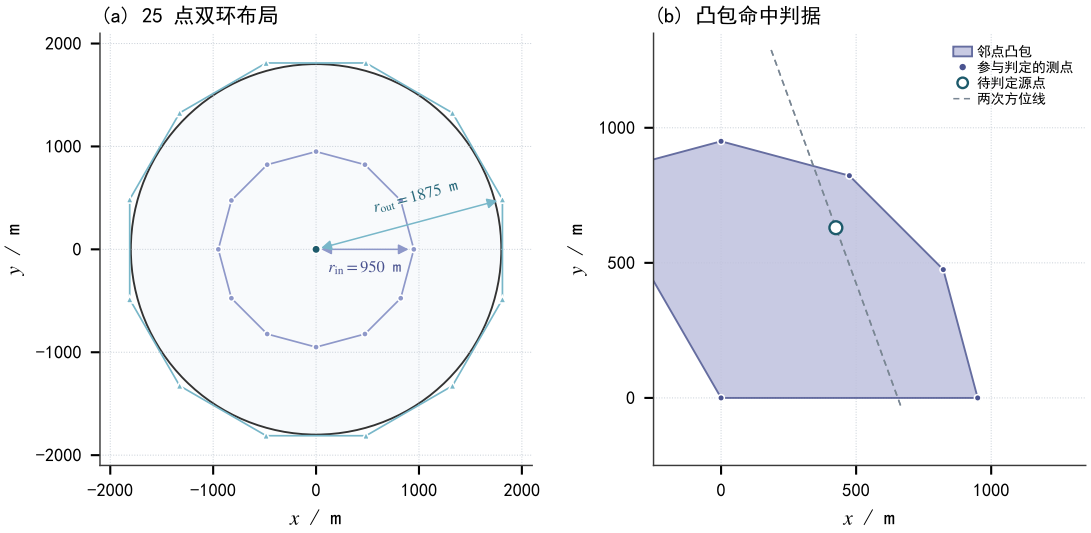


图 12 问题四的 25 点双环保证布局

凸包外。固定 $(r_{\mathrm{in}},\varphi)=(950\ \mathrm{m},15^\circ)$ 后，再按径向 2 m、角度 0.1° 检查约 324 万个采样源点：$r_{\mathrm{out}}=1800$ m 和 1850 m 时的最小角裕度分别为 -12.576° 和 -3.119°，到 1875 m 时才变为 $+2.623^\circ$，与前一轮扫描一致。题面只限制坐标分量绝对值不超过 2×10^6 m，允许机器狗到目标圆盘之外检测；20 局接口演练中，外环点也未触发坐标或边界限制。

4.4.3 滚动联合规划

25 个保证点给出完整巡视任务，但执行过程中没有必要先走完全部保证点再处理频道。未访问保证点始终保留在任务集中；某频道取得方位后，为它生成复测入口；定位圆足够小时，再把圆心改为清除入口。程序用不同的最近邻起点和同距离择优规则生成 5 条初始开放路线，分别经 2-opt 和 Or-opt(1) 改进，保留总路程最短的一条。机器狗只执行首个动作，随后根据新位置和接口返回重新规划。这样可以把临时出现的复测与清除任务插入途中，同时不会因路线优化遗漏尚未访问的保证点。

若两次测向后的定位圆半径不超过 200 m，先在圆心尝试一次清除。该阈值大于 20 m，意味着尝试不一定成功；采用它是因为一次失败只增加 3 s，而成功可以省去后续网格搜索。若第二测点返回无信号，则不能再利用该点构造新的误差带，但第一次示向仍然有效。程序把首次误差扇形旋转到局部坐标系，将 0–1500 m 的前向距离划分为不超过 30 m 的小段，每段由近及远访问 1–2 个光学覆盖点。沿扇形长轴搜索可避免在其世界坐标外接矩形中反复横穿空白区域。

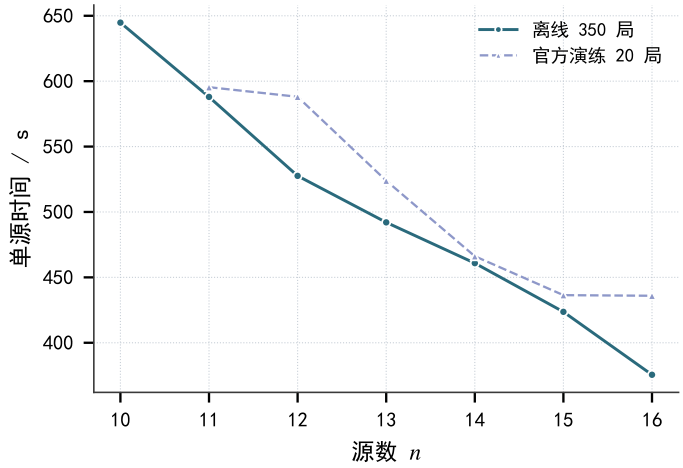
4.4.4 离线实验与接口演练

按源数分层的离线结果见表 12。

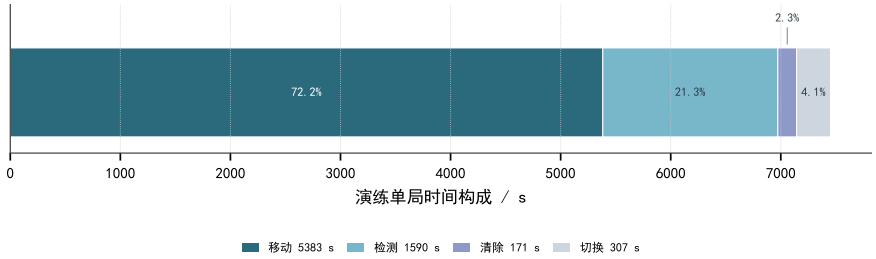
350 局均完成清除。按全部源汇总，平均耗时为 488.11 s/源；按局等权计算为 501.67 s/源，中位数 495.40 s/源，P95 为 657.11 s/源，最大值 767.20 s/源。图 13还给出一局官方演练中各类动作的时间占比。

表 12 问题四原批次离线混合场景结果（350 局）

源数	10	11	12	13	14	15	16
平均时间/(s/源)	644.75	587.96	527.60	492.05	460.87	423.65	375.53



(a) 离线分层验证



(b) 官方演练动作时间占比

图 13 问题四的分层耗时与动作构成

演练测试按题面口径统计于表 13：20 局共记录 275 个源，全部在 `exit` 前被清除，清除比例为 100%，按被清除源数折算的平均定位清除时间为 490.67 s/源。练习数据只用于检查接口和动作顺序，不作为正式成绩。

表 13 问题四演练测试统计（原始日志：代码/`logs`）

方案	局数	源数	清除数（比例）	单源时间/(s/源)	最差局/(s/源)
混合源滚动规划（本文方案）	20	275	275（100%）	490.67	627.36

三次正式测试均正常运行至`/exit`，分别清除 11、12 和 12 个干扰源，单源时间为 590.08、503.03 和 493.76 s；按 35 个已清除源汇总为 527.21 s/源。按相同源数组合对离线分层结果加权，基准为 547.04 s/源，正式结果低 3.6%，数量级和随源数增加而下降的趋势均一致。由于正式测试只有 3 次，该差值只作一致性检查，不据此宣称算法进一步提升。三次程序墙钟时间均不超过 3.83 s，说明滚动重规划的计算开销远小于题面给出的 20 min 上限。逐次记录见表 24。

表 14 问题四独立复跑的分层统计（350 局）

(a) 单源时间分布

源数	局数	清除率/%	均值/s	标准差/s	中位数/s	P95/s	最大值/s
10	47	100	652.92	50.63	643.52	755.40	811.42
11	52	100	582.95	47.68	571.78	668.33	737.22
12	33	100	530.58	42.69	521.33	624.30	656.32
13	59	100	504.46	50.59	490.45	606.48	626.15
14	58	100	458.47	37.33	449.21	528.90	567.68
15	56	100	431.23	43.60	421.41	503.42	539.07
16	45	100	363.17	65.62	353.99	471.21	493.55

(b) 每局动作统计

源数	平均测向次数	平均清除尝试	启用异常处理的局数	平均异常频道数
10	324.21	16.98	4	0.11
11	306.02	20.90	2	0.04
12	290.79	20.21	3	0.12
13	271.03	25.12	16	0.29
14	253.05	24.43	7	0.12
15	237.95	28.02	7	0.13
16	184.36	26.60	9	0.22

4.4.5 双环参数检验与源数影响

内环半径影响中心附近检测点的角度分布，外环半径决定边界源点能否落入邻点凸包。外环过小时，边界附近的局部凸包向内收缩；过大时，检测点会远离目标圆盘并拉长路线。先在 $(r_{\mathrm{in}}, r_{\mathrm{out}}, \varphi)$ 网格上筛选，再以径向 5 m、角度 0.25° 加密检查。参数 $(950, 1875, 15°)$ 的最小角裕度为 2.623°，而同一加密口径下最好的 24 点候选仍为负值 $(-3.734°)$，所以后者虽路程略短，仍不满足发现条件。

为检验原批次 350 局结果是否受一组随机种子影响，使用同一程序和参数，以种子 3000–3349 独立复跑 350 局。该批次不删去高耗时局，也不按源数筛选；各组样本量为 33–59 局。表 14同时给出耗时离散程度和动作次数。

独立复跑的 350 局仍全部清除，按源汇总为 487.95 s/源，按局等权均值为 501.02 s/源，中位数 493.69 s/源，P95 为 669.44 s/源；按式(21)计算的局均值 95% 置信区间为 [490.48,511.55] s/源。原批次对应的按源汇总结果为 488.11 s/源，两批相差 0.16 s/源（约 0.03%）。源数增加时，每源时间下降，主要因为保证点扫描的固定成本被更多干扰源分摊；表中的标准差和 P95 也说明，低源数场景不仅均值较高，尾部耗时也更明显。

4.4.6 混合源状态机与失败局隔离

对定向源，单次示向只得到一条扇形带，不能直接按全向源的圆心规则清除。状态仍按式(18)更新，动作顺序为：`direction`→ 保存扇形 → 选择保证点或顺路点 → 第二次示向 → 凸包/MEC 判定 →`clear`。若第二测点返回 `no_signal`，当前候选入口作废，频道状态转为“兜底”，并改用局部旋转光学网格；`no_target_in_range` 则补充近距离测向；半平面交退化时保留原始测量并重新选点。三种异常都不删除其他频道或未访问

保证点，日志据此记录失败出现在哪一步。

状态机与失败局隔离见图 14。

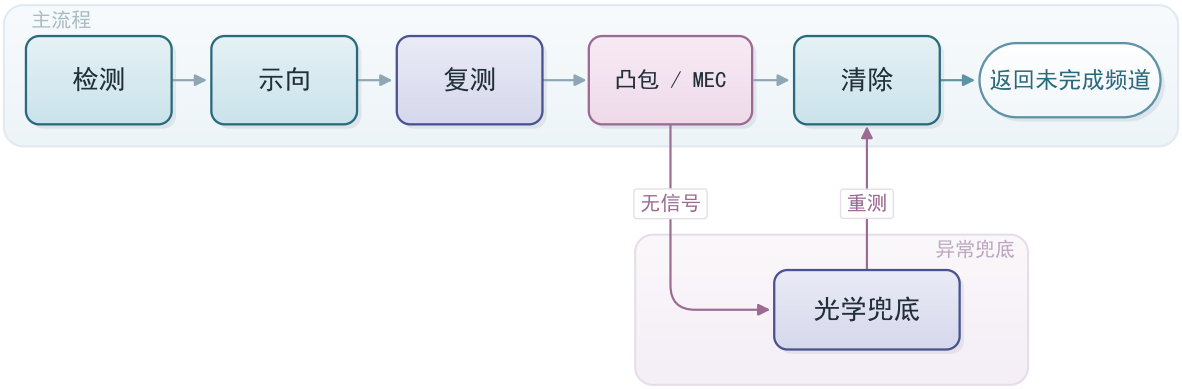


图 14 问题四混合源状态机

需要注意定向源的“背向无信号”特性：按模拟器协议，检测点即使在 5 m 以内，只要落在辐射半平面之外，`measure` 仍返回 `no_signal`；而 `clear` 是否成功只与距离有关（不超过 20 m 即成功），与辐射朝向无关。对定向源不必依赖 `near` 触发清除：一旦路线把机器狗带到预测源位附近，直接在该点及其邻域发 `clear`，失败仅计 3 s，成功后按全向源同样的方式继续。这一条使问题四在“第二次测向无信号”时仍能保证清除，而不必重新绕行。

10 源场景的固定成本较高。若逐点检查 10 个空频道，按每次测量 6 s 计，25 个保证点共需约 1500 s；双环的最短开放路线约 18.02 km，移动约 3605 s。两项合计已为 510.5 s/源，尚未计入定位和清除。在现有动作规则下，无法承诺所有 10 源场景都低于 400 s/源；若要继续降时，应优先减少保证点或利用频道先验，而不是压缩已经较短的局部定位过程。

5　模型评价

模型检验与实验口径

检验顺序与求解过程一致。先用几何单元测试检查误差带、半平面交、直径和最小包围圆，测试输入包括空集、无界、点、线段和近似平行边界；这一步回答定位结果是否算对。再在固定种子的离线场景中运行完整策略，比较改动前后的成功率、虚拟时间和移动距离；这一步回答路线规则是否有收益。最后接入官方接口，检查请求编号、频道状态以及 `near/direction/no_signal` 等分支能否按预期转移。离线实验、接口演练和正式测试的用途不同，统计时不合并样本。

每局日志保存时间戳、当前位置、频道、动作类型、返回状态、累计虚拟时间和请求编号。总时间取 `exit` 响应中的虚拟时刻，单源耗时以成功清除数为分母。若一局未清除全部源，该局仍计入成功率和失败原因统计，不从样本中删除。距离、时间分别使

用 m 和 s，输入角度在进入三角函数前统一转为弧度。

统计量与不确定性

对 N_{run} 局观测量 $z_1,\dots,z_{N_{\mathrm{run}}}$，正文报告均值 $\bar{z}$、中位数、最大值和经验 95% 分位数。均值的近似 95% 置信区间写为

$$\bar{z} \pm t_{0.975,N_{\mathrm{run}}-1}\frac{s_z}{\sqrt{N_{\mathrm{run}}}}, \qquad s_z^2=\frac{1}{N_{\mathrm{run}}-1}\sum_{k=1}^{N_{\mathrm{run}}}(z_k-\bar{z})^2. \tag{21}$$

均值描述总体水平，中位数反映典型场景，P95 和最大值用于检查少数困难场景。配对实验的 500 局使用相同随机种子，逐局计算 $\Delta_k = z_k^{\mathrm{base}} - z_k^{\mathrm{new}}$，使每一对样本具有相同的源数、位置和频道，从而减少场景难度差异对比较的干扰。60 局逐级消融用于追踪改进来自哪条规则，角阈值和圆半径上限各用 120 局扫描检验参数稳定性，演练日志只用于检查接口一致性。正式测试仅有三局，样本过少，故只逐次登记并与原始日志核对，不据此估计置信区间。

计算量

设当前有 n 条半平面、n_P 个多边形顶点、q 个待处理任务，每个任务至多有 b 个入口。半平面交的复杂度为 $O(n\log n)$，旋转卡壳为 $O(n_P)$，MEC 随机增量算法的期望复杂度为 $O(n_P)$。贪心初始路线和一次完整 2-opt 邻域扫描为 $O(q^2b^2)$，固定任务次序后选择入口的动态规划为 $O(qb^2)$。差分进化、SHGO 和 Or-opt 属于数值或启发式搜索，其耗时还取决于采样数、种群规模和迭代上限，不能由上述多项式写成全局最优复杂度。题目中频道数至多 20，程序对迭代次数设置上限；每次反馈只重算有关频道的可行域和剩余路线，不遍历已经执行的轨迹。

模型的优点

模型把发现保证放在路线优化之前。有界测角误差直接进入半平面约束，七点覆盖和双环凸包分别处理全向源与定向源的最坏发现条件；粒子评分和局部搜索只改变可选任务的入口与顺序，不删除保证点，也不收缩真实可行域。因此，即使概率近似或启发式路线没有取得最优值，也不会直接破坏前面的几何条件。

几何模块保留空集、无界、退化和近似平行等状态，程序能够区分“信息还不足”和“数值计算异常”。路线模块按当前位置和频道反馈滚动更新；某个频道出现无信号时，只把该频道转入兜底状态，其他频道的测量和未访问保证点仍然保留。网络重试继续使用原请求编号，避免同一动作被服务器重复执行或被本地状态重复计数。

评价指标、基线与判定顺序

成功率、时间和定位精度不合并成一个加权分数，因为漏检不能由节省的时间抵消。离线模拟器已知第 k 局的真实源集合；只有每个源都收到一次成功的 `clear` 返回，且 `exit` 时剩余源数为 0，才记 $Y_k=1$，否则记 $Y_k=0$。成功率为 $N^{-1}\sum_{k=1}^{N}Y_k$。中途出现 `EMPTY`、`UNBOUNDED` 或 `no_signal`，若最终恢复并清除全部源，该局仍记为成功，但异常次数保留在日志中。先按成功率排除不可行方案，再比较定位精度、总时间、移动距离和高分位耗时。各项指标及其复核依据列于表 15。完整性按是否全部清除作布

表 15 评价指标、基线与检查方法

维度	判定方式	基线 / 目标	证据与复核方式
完整性	覆盖检查与成功率	100% 且无漏检	七点覆盖与双环凸包判据；问题三 500 局、问题四两批各 350 局全部清除
定位精度	最坏 MEC 半径与直径	问题二 $R^* = 67.03$ m	差分进化 5 次、SHGO 与活跃圆弧交叉复核
效率	单源虚拟时间与移动距离	相对基线的配对降幅	500 局配对 (-45.87%)、60 局结构消融、参数扫描
稳定性	中位数、P95、最坏值	不只报告均值	两批各 350 局混合场景、20 局演练及问题三、四正式日志
可复现性	测试通过数与种子固定	43 项测试（7312 次断言）	附录 A 伪代码、支撑材料源码与测试入口
合规性	演练与正式结果分离	演练不计入正式成绩	按题面表 1 登记正式结果，保留原始 JSONL 日志

尔判定，只有通过该项后才比较效率。一次运行即使耗时较短，只要存在漏检，也不计入可行方案的平均时间。

参数影响

三类参数的作用不同。问题二的安全裕度直接收紧必接收域：从 0 m 增至 5 m 后，最坏包围圆半径由 67.032926 m 增至 67.610529 m，代价是 0.86% 的半径增加，换得接收边界的实施余量。问题三的顺路半径阈值决定多少线段候选能够代替专用工程点；从 90 m 放宽至 110 m 时绕路明显减少，继续放宽后收益趋缓而定位区域继续变大。问题四的点数先受凸包条件限制，24 点方案虽能缩短路线，但最小角裕度为负，不能用效率优势弥补漏检风险。

模型的不足

问题二的测点位置由多种数值算法相互核对，但尚未得到区间算术意义下的全局最优证明；问题三、四的路线也只是当前候选集上的改进解。离线场景由自建模拟器生成，接口演练样本数量有限，都不能代表正式隐藏实例的分布。时间统计是题面成本模型下的虚拟时间，不等同于真实平台的墙钟时间；模型没有计入转弯与加减速、信噪比相关测量时延、平台定位误差、系统偏置和多径传播。若能获得更多练习数据，可估计源数、定向比例和有效半径的先验，再调整任务优先级；七点覆盖、双环凸包条件和光学网格仍用于保证最坏情况下能够继续搜索。

推广与改进方向

若误差模型以后加入距离区间，可把环带边界离散成凸多边形，再沿用求交、直径和最小包围圆计算；代价是多边形近似会引入额外保守量，需要随边数检查。若接收半径或覆盖角改变，七点和双环坐标不能直接照搬，但仍可按“局部距离约束 + 方向侧命中”的判据重新搜索保证点。对于动态源或多机协同，本文的静态状态转移已经不够，还需增加时间索引和机器间任务分配，这超出本文模型的适用范围。

6 结论

四个问题可归结为定位范围计算、第二测点选择、全向源搜索和混合源搜索。问题一用半平面交求定位范围，并分别计算直径和最小包围圆；问题二在必接收条件下最小化最坏包围圆半径，得到 $Q^* = (804.895, \pm 600.052)$ m，实际运行取带 5 m 裕度的工程点；问题三采用七点覆盖，并用顺路复测和滚动 GTSP 减少往返；问题四以 25 点双环满足定向源的凸包条件，无信号时转入局部光学搜索。

四问的模型、算法与关键结果汇总于表 16。

表 16 四问的主要模型、算法与关键结果

问	主要模型与算法	关键结果
一	误差带半平面交、旋转卡壳求直径、最小包围圆	定位区域直径与清除半径可在同一凸集上算出；直径为 D 的圆能覆盖定位区域当且仅当式(6)或其圆心可任取的等价条件成立
二	必接收候选域上的极小极大优化，差分进化+ SHGO +活跃圆弧复核	$Q^* = (804.895, \pm 600.052)$ m，最坏半径 $R^* = 67.032926$ m；工程点半径 67.610529 m，移动距离少 5.01 m
三	七点确定性覆盖、顺路第二测向、滚动 GTSP、粒子后验测点	500 局配对单源时间由 441.51 s 降至 238.97 s (-45.87%)，移动距离 -52.78%，两种方案均清除 6415 个源；三次正式测试共清除 39 个源，按源汇总为 230.43 s/源
四	25 点双环凸包布局、滚动联合规划、局部光学搜索	加密检查最小角裕度 2.623°；两批各 350 局均全部清除；三次正式测试共清除 35 个源，按源汇总为 527.21 s/源

在 500 局配对实验中，问题三两种方案均清除全部源（6415 个源），本文方案的单源时间由 441.51 s 降至 238.97 s，平均移动距离降低 52.78%，检测次数基本相同。这说明降时主要来自覆盖巡视之外的路线缩短，而不是减少必要测量；60 局消融实验中，顺路复测、滚动联合路线、归航细化和粒子后验测点加入后耗时依次下降，也与这一解释相符。问题四原批次 350 局和独立复跑 350 局均完成全部清除，按所有源汇总分别为 488.11 和 487.95 s/源；两批结果接近，说明汇总均值不是由单批种子的偶然取值造成的。三次正式测试的单源时间落在 493.76–590.08 s 之间，按源汇总为 527.21 s/源，与 11–12 源离线分层结果处在同一范围内。

这些数值只说明算法在题设二维静态场景和自建模拟器中的表现。确定性保证仅覆盖 1° 测向误差、最小接收半径和未知半平面朝向；对真实平台误差、传播环境和移动源的适用性尚需另行试验。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试、批量模拟测试脚本编写和 LaTeX 排版检查，详细使用情况见支撑材料。为提高测试效率，参赛

队员给出接口规则和统计口径后，使用 Codex 编写批量模拟测试脚本，重复生成测试场景并汇总耗时数据；参赛队员逐项核对脚本和原始日志，再根据耗时构成自行修改程序、调整参数并完成优化。具体使用情况见表 17。

审核机制上，凡有 AI 参与的内容都逐条人工核对。由 AI 辅助生成的测试脚本须经队员检查并通过全部单元测试后才进入实验流程；论文中的每个数值都由程序输出或原始演练日志复算验证；离线实验与官方演练的统计口径、样本数与失败样本记录均按原始日志核对，不作删改。AI 生成的代码与文字不构成任何结论依据，文字润色不改变任何数据与判断。

我们对论文内容的真实性、实验数据的可靠性与全部结论负完全责任。

表 17 人工智能工具的使用情况

工具（类别）	使用环节	具体用途
Codex	模拟测试、论文审查	按队员确定的接口和指标编写批量模拟测试脚本，汇总大量测试的耗时数据；并核对量纲、边界情形及少量 LaTeX 编译问题
DeepSeek、Claude Code	代码审查	与 Codex 交叉复核程序逻辑、参数设置与数值结果
代码补全类工具	程序开发	函数框架补全、少量参数调整等重复性工作
通用对话类工具	文字与公式	少量语句的表达润色，以及 Markdown / LaTeX 数学公式的格式书写

参考文献

[1] 邓新蒲. 单站无源定位可观测性评述. 中国工程科学, 9(11):54–62, 2007.

[2] 吴龙文, 王宝莹, 魏俊杰, 何胜阳, and 赵雅琴. 基于 aoa 的双机无源定位模型及其解算方法. 系统工程与电子技术, 42(5):978–986, 2020.

[3] 冯奇, 曲长文, and 李廷军. 基于约束加权最小二乘的无源定位闭式解算方法. 系统工程与电子技术, 39(2):263–268, 2017.

[4] 周培德. 计算几何：算法设计与分析. 清华大学出版社, 北京, 2 edition, 2005.

[5] Emo Welzl. Smallest enclosing disks (balls and ellipsoids). In Hermann Maurer, editor, *New Results and New Trends in Computer Science*, volume 555 of *Lecture Notes in Computer Science*, pages 359–370. Springer, 1991.

[6] Rainer Storn and Kenneth Price. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.

[7] Stefan C. Endres, Carl Sandrock, and Walter W. Focke. A simplicial homology algorithm for lipschitz optimisation. *Journal of Global Optimization*, 72:181–217, 2018.

[8] Charles E. Noon and James C. Bean. An efficient transformation of the generalized traveling salesman problem. *INFOR: Information Systems and Operational Research*, 31(1):39–44, 1993.

[9] 陈洋, 赵新刚, and 韩建达. 移动机器人 3 维路径规划方法综述. 机器人, 32(4):568–576, 2010.

[10] Giordano Fusco and Himanshu Gupta. Selection and orientation of directional sensors for coverage maximization. In *2009 6th Annual IEEE Communications Society Conference on Sensor, Mesh and Ad Hoc Communications and Networks*, Rome, Italy, 2009.

附　录

A　核心算法伪代码

误差带定位

输入测站 S_i、方位 θ_i 和误差 δ；将每个扇形转为式(3)的两个定向半平面；按方向角排序并合并同向平行约束；双端队列扫描并闭合清理；判定空集/无界/退化；对有界顶点执行旋转卡壳和最小包围圆；输出可行域、直径、圆心及状态码。

滚动清除

初始化频道状态、保证点集合和当前位置；在保证点上扫描尚未知频道；若返回 `near` 则光学定位并清除；若返回 `direction` 则保存误差带，并按式 (16)用粒子后验在剩余路线上选出测点；把待处理频道构造成带左右入口的 GTSP，执行贪心、2-opt、Or-opt 与左右分支动态规划；每执行一个动作即更新状态并重规划；定位圆半径不超过 20 m 时直接清除，否则追加测量；对不超过两次测向的频道同样执行测点后验寻优；信号丢失或几何退化时切换旋转光学兜底。

滚动规划的伪代码（可直接对应实现）

输入：保证点集合 V、频道集合 C、初始位置 X_0。
while $C \neq \varnothing$ **or** 存在未访问保证点：
(1) 对未访问保证点和已生成的服务入口计算相对当前路线的插入路程，按增量从小到大生成候选次序；
(2) 对每个已返回 `direction` 的频道，生成顺路候选并检查接收裕度、交会角和 MEC 上限；
(3) 将保证点和服务入口合并为 GTSP 任务，贪心生成 $K = 5$ 个初解，交替执行 2-opt 与 Or-opt(1)；
(4) 对固定频道顺序用式(17)精确选择左右入口，取代价最小的路线；
(5) 只执行路线的第一个移动/测量/清除动作，记录请求编号并更新 X, C, V；
(6) 若清除失败或几何状态为 `EMPTY/UNBOUNDED/DEGENERATE`，仅对当前频道调用光学兜底；
end while；输出清除列表、累计时间和逐动作日志。

问题四保证点坐标表

为便于复核，表 18列出全部 25 个保证点的极坐标。实际程序使用双精度笛卡尔坐标，表中保留三位小数仅用于阅读。

日志字段与异常处理

表 18 25 点双环布局的极坐标

编号	半径/m	角度/°	编号	半径/m	角度/°
P0	0	–	I1–I6	950	0,30,60,90,120,150
I7–I12	950	180,210,240,270,300,330	O1–O6	1875	15,45,75,105,135,165
O7–O12	1875	195,225,255,285,315,345	–	–	–

每条日志包含 `request_id`、`action`、`channel`、`x/y`、`result`、`virtual_time` 和 `state_before/after`。网络重试沿用原 `request_id`，重复响应不再次更新本地状态。`NO_SIGNAL`、`NO_TARGET_IN_RANGE` 和 `GEOMETRY_DEGENERATE` 分别触发继续扫描保证点、补充近距离测向和旋转光学网格，不直接记作整局失败。

测试用例与复现实验说明

表 19列出复核论文主要结论所需的最小测试集。程序、参数配置和原始输出随支撑材料提交，具体运行命令不在论文中展开。更换运行环境时需保持依赖版本、随机种子和数值容差一致，并按表中通过标准核对结果。这些测试用于检查几何计算、路径规

表 19 主要测试用例及通过标准

测试内容	输入或场景	规模	通过标准
几何与退化测试	空集、无界、点、线段、近似平行	43 项测试（7312 次断言）	状态码正确，直径与 MEC 误差不超过设定容差
问题二算法比较	同一候选域下的 7 种测点策略	7 种策略	接收约束成立，报告最坏直径和 MEC 半径
问题三配对实验	同一批源位置和频道组成分别运行两种方案	500 局配对、60 局消融	不删除失败局，按式定义成功并报告时间、里程
问题四混合实验	全向/定向比例及源数分层场景	350 局	先检查凸包命中，再统计成功率和单源时间
接口演练	平台返回的近距离、示向和无信号分支	20 局、275 个源	请求编号幂等，动作状态与原始日志一致

划和状态转移，不替代正式隐藏实例。正式结果由平台原始日志统计；若接口字段或动作时延与离线模拟器不同，应先修改接口适配部分，再重新计算有关表格。

B　补充推导与边界案例

接收候选域的极值检查

在问题二中，源点集合 $\mathcal{K}$ 由半径区间与小角度扇形构成。对固定 $Q=(a,b)$，平方距离

$$f(\rho,\phi)=\rho^2+a^2+b^2-2\rho(a\cos\phi+b\sin\phi) \tag{22}$$

关于 ρ 是凸函数，最大值必在 $\rho = 5$ 或 1500 m 处；关于 ϕ 的一阶导数只有一个内部驻点，只需检查角度端点与落在区间内的驻点。程序先计算这些位置，再在端点附近作连续搜索，把二维最坏情形的检查缩减为少量边界场景。

光学兜底网格的覆盖半径

当定位多边形无法继续缩小时，在有限扇形内采用步长 $s \le r_c\sqrt{2}$ 的蛇形网格。任意源点到最近网格点的水平、竖直偏差均不超过 $s/2$，故欧氏距离不超过 $s/\sqrt{2} \le r_c$。网格先与误差扇形求交，再裁剪到目标圆盘，避免搜索整个外接矩形。该分支只处理异常输入，不计入正常场景的平均耗时。

边界案例示例

三类边界案例与预期动作见表 20。

表 20 三类边界案例及预期动作

案例	几何特征	预期处理
两条测向几乎平行	交会角低于 3°，直径可超过目标圆域尺度	标记低可观测，优先移动到工程第二测点
第二测点恰在接收边界	数值误差可能使 $\|Q - G\|$ 略超 1000 m	使用 5 m 裕度点并保留原最优点作理论对照
定向源落在双环外缘	邻近检测点方向可能只覆盖一个半平面	依靠外环凸包和相邻内环点共同命中

C　补充检验

保证点的时间代价

设保证点集合为 V，保证点巡视阶段共执行 N_{probe} 次频道测量；从 v 移动到 w 的时间为 $\|v - w\|/v_{\mathrm{dog}}$。忽略频道切换、定位和清除耗时后，扫描阶段仍满足

$$T_{\mathrm{scan}}^{\mathrm{lb}} = N_{\mathrm{probe}} t_{\mathrm{meas}} + \frac{\mathrm{len}(\mathrm{MST}(V \cup \{X_0\}))}{v_{\mathrm{dog}}}. \tag{23}$$

式(23)两项的量纲均为时间，是未计频道切换与后续服务动作时的下界。问题四中 25 点双环的最短开放路线约 18.02 km；所有频道均为空时，移动与逐点测量已占去大部分时间。保证点集合先按几何条件确定，访问顺序再按剩余频道数和已有方位调整。

各环节能否压缩见表 21。

四问主要结果的复核材料汇总于表 22。

关键参数变化

由式(7)，小角度下的定位直径 D 近似正比于误差上界 δ 和测站距离 d_s。问题三的 $h(r)$ 取两个距离项的较大值，r 偏离 $r_{\min}$ 后至少有一项增大。问题四增大外环半径有

表 21 在线动作的时间构成与可优化环节

环节	是否可跳过	处理方式
频道切换/测量	不可跳过（对未知频道）	在保证点批量扫描；顺路复测只复用移动，不额外产生路径长度
保证点移动	不可完全跳过	由覆盖命题确定点集，由 GTSP 局部搜索减少交叉和折返
定位与清除	仅在发现源后发生	MEC 半径达到阈值即清除，否则只对当前频道追加测量
网络重试	偶发	复用同一请求编号，不重复计入动作状态
光学兜底	仅异常分支	采用扇形内蛇形网格，失败局不影响其他频道

表 22 主要结果及复核材料

结果	主要决定因素	复核材料
问题一定位直径	交会角、测站距离、误差上界	半平面交顶点与灵敏度表
问题二第二测点	必接收域边界与最坏 MEC	差分进化、SHGO、活跃圆弧检查
问题三时间下降	顺路复测与路线局部搜索	500 局配对、60 局消融、移动距离日志
问题四不漏检与耗时	双环凸包角裕度、固定扫描成本	25 点坐标表、加密射线检查、两批 350 局日志

利于包住边界源点，却会拉长开放路径。因而测角精度、测站距离和保证点数量不能分别单独优化。

参数检验取 $\delta = 0.5^\circ, 1^\circ, 1.5^\circ$，$L = 950, 1000, 1050$ m，双环角偏移为 $0^\circ, 15^\circ, 30^\circ$。每组均重新计算可行域、覆盖条件和路径长度；不满足硬约束的组合记为不可行，不再比较耗时。

主要结果的复核位置

Q^* 由问题二最坏 MEC 计算得到；1122.955832 m 是七点覆盖可行区间的下端点；2.623° 为双环布置在加密采样下的最小角裕度（由 `audit_problem4_ring_margin.py` 复核）；488.11 s/源来自原批 350 局离线混合场景，487.95 s/源来自种子 3000–3349 的独立复跑；490.67 s/源来自 20 局练习日志。问题三、四正式测试按源汇总分别为 230.43 和 527.21 s/源，均由正式统计数据库与本地动作日志交叉核对。相应程序和数据文件列于表 22。

D　问题三正式测试记录

正式测试按题面表 1 的列名登记，不使用离线或官方演练数据替代。其中“平均定位清除时间”按题面定义计算，即该次测试的定位清除总时间除以被清除干扰源个数；“程序运行时间”取程序调用 `/enter` 成功到调用 `/exit` 的墙钟时间。

三次测试的统计量由模拟器正式统计数据库读取，并与本地动作日志中的`/exit` 虚拟时刻核对，结果见表 23。

表 23 问题 3 正式测试结果（按题面表 1 格式）

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
27V6-WQWS-V9KU-Z8GE	16	205.11	1.97
XUYN-U4ET-Q6VX-XYUH	12	230.02	2.08
67YN-BK63-C7SH-NYJX	11	267.68	1.72

三次测试共清除 39 个干扰源，按源汇总的平均定位清除时间为 230.43 s/源。三次程序运行时间均不超过 2.08 s；日志中记录的测向次数依次为 89、103 和 108 次，清除失败次数依次为 3、1 和 1 次，失败后均继续执行而未提前退出。

E　问题四正式测试记录

问题四沿用题面表 1 的统计口径。“平均定位清除时间”取该次测试的虚拟总时间除以成功清除数；“程序运行时间”由模拟器正式统计数据库记录，并用本地`/enter--/exit` 日志核对。三次结果见表 24。

表 24 问题 4 正式测试结果（按题面表 1 格式）

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
V3TG-ZCG7-6AXQ-NZ2M	11	590.08	3.56
CEXN-K2DA-FE7P-SZMN	12	503.03	3.31
5HKV-ZCJV-F58F-MNY9	12	493.76	3.83

三次测试共清除 35 个干扰源，虚拟总时间依次为 6490.92、6036.38 和 5925.13 s，按源汇总的平均定位清除时间为 527.21 s/源。测向次数依次为 310、259 和 260 次，清除失败次数依次为 3、18 和 15 次；失败后算法均继续更新频道状态并完成后续任务。三次程序运行时间为 3.56、3.31 和 3.83 s，均远低于 20 min 上限。

F　支撑材料文件清单

支撑材料压缩包（RAR/ZIP，不超过 20 MB）内收录下列全部文件；表中标 $\star$ 的文件是建模与求解直接使用的核心程序，其完整源码在本节给出（两个规划器仅保留最终算法与消融基线所用的类，行为已与完整版逐位比对一致）；其余为实验、绘图与统计辅助脚本，随支撑材料提供。

表 25 支撑材料文件清单

文件	作用
analyze_official_problem3_logs.py	演练日志统计（清除比例与单源时间）
analyze_problem1_sensitivity.py	问题一敏感性分析
analyze_problem2_optimization.py	问题二结果整理
audit_problem2_optimum.py	问题二最优解审计（多算法交叉验证）
audit_problem4_ring_margin.py	按径向 5 m、角度 0.25° 复核双环凸包角裕度
⋆ b_model/__init__.py	模块导出
⋆ b_model/coverage.py	七点覆盖构型与可行环半径区间
⋆ b_model/geometry.py	半平面交、旋转卡壳直径、最小包围圆
b_model/offline_simulator.py	离线模拟器（按题面规则计时）
⋆ b_model/problem2_optimization.py	问题二最坏清除圆目标、差分进化/SHGO 复核
⋆ b_model/problem3_planner.py	问题三规划器族（含消融变体）
⋆ b_model/problem4_planner.py	问题四双环布局与滚动联合规划
b_model/simulator_client.py	官方接口客户端（幂等与重试）
⋆ b_model/station_selection.py	问题二第二测点的候选生成与必接收约束
benchmark_problem2_algorithms.py	问题二算法对比实验
benchmark_problem3_algorithms.py	问题三版本对比实验
build_problem4_route_visualization.py	问题四路线可视化构建
demo_coverage.py	覆盖构型演示图
demo_problem1.py	问题一演示与图形输出
explore_problem4_mesh.py	双环参数网格探索
explore_problem4_ring_mesh.py	双环半径组合探索
generate_paper_figures.py	论文配图生成脚本
optimize_problem2_circle_radius.py	问题二安全裕度扫描
optimize_problem4_cover_route.py	双环访问顺序优化
plot_official_problem4_route.py	问题四演练路线可视化
⋆ run_official_problem3_practice.py	问题三官方演练入口（含四个版本）
⋆ run_official_problem4_practice.py	问题四官方演练入口
run_offline_problem3.py	问题三离线运行入口
run_offline_problem4.py	问题四离线运行入口
search_problem4_24point.py	24 点双环反例搜索（否定性结果）
⋆ solve_problem1.py	问题一求解入口
summarize_problem4_logs.py	问题四演练日志统计
tests/test_geometry.py	模型/实验源码
tests/test_halfplane_intersection.py	模型/实验源码
tests/test_problem1_solver.py	模型/实验源码
tests/test_problem2_optimization.py	模型/实验源码

续表 25：支撑材料文件清单

文件	作用
tests/test_problem3_planner.py	模型/实验源码
tests/test_problem4_planner.py	模型/实验源码
tests/test_simulator_client.py	模型/实验源码
运行问题 3 官方演练.ps1	问题三演练启动脚本（PowerShell）
运行问题 4 官方演练.ps1	问题四演练启动脚本（PowerShell）

G 源程序代码

以下源码与支撑材料压缩包完全一致，包含建模与求解直接使用的核心程序：几何与覆盖模块、问题二优化、问题三/四规划器与三个入口脚本。离线模拟器、官方接口客户端、单元测试与实验脚本篇幅较长，只在文件清单中列出，源码随支撑材料提供。运行方式为 python < 入口脚本 >，命令行参数见各文件头部注释。为便于等宽字体排版，附录中把注释里的少量数学符号写成 ASCII 形式（例如 π 写作 pi、→ 写作 ->），并省略空行以控制篇幅；支撑材料中保留原格式与全部文件。

目录 b_model

__init__.py

```python
"""B题无线电干扰源定位与清除的核心算法包。"""
from .geometry import (
    Circle,
    HalfPlane,
    LocalizationResult,
    Point,
    build_localization_polygon,
    circle_on_diameter_pair,
    convex_polygon_diameter,
    diameter_pair_circle_covers_polygon,
    halfplane_intersection_deque,
    minimum_enclosing_circle,
    polygon_area,
)
from .station_selection import (
    StationCandidate,
    choose_practical_second_station,
    choose_second_station,
    choose_second_station_by_direct_diameter,
    choose_second_station_by_worst_circle,
    generate_second_station_candidates,
)
from .coverage import (
    center_hexagon_covering_radius,
    center_hexagon_points,
    feasible_hexagon_ring_interval,
    triangular_lattice_points,
)
__all__ = [
    "Circle",
    "HalfPlane",
    "LocalizationResult",
    "Point",
    "build_localization_polygon",
    "circle_on_diameter_pair",
    "convex_polygon_diameter",
    "diameter_pair_circle_covers_polygon",
    "halfplane_intersection_deque",
    "minimum_enclosing_circle",
    "polygon_area",
    "StationCandidate",
    "choose_practical_second_station",
    "choose_second_station",
    "choose_second_station_by_direct_diameter",
    "choose_second_station_by_worst_circle",
    "generate_second_station_candidates",
    "center_hexagon_covering_radius",
    "center_hexagon_points",
    "feasible_hexagon_ring_interval",
    "triangular_lattice_points",
]
```

geometry.py

```python
"""问题1使用的二维鲁棒定位几何算法。
每次示向度测量 theta 带有 [-error_deg, error_deg] 的有界误差，
因此干扰源必定位于以检测点为顶点、theta 为中线的角形区域中。
角形区域可表示为两个线性半平面；多个测量的可行域就是所有
半平面的交集，因此是凸多边形（空集和无界情形单独处理）。
"""
from __future__ import annotations
from collections import deque
from dataclasses import dataclass
from itertools import combinations
from math import atan2, cos, hypot, isfinite, pi, radians, sin, sqrt
from random import Random
from typing import Iterable, Sequence
EPS = 1e-9
@dataclass(frozen=True)
class Point:
    x: float
    y: float
    def distance_to(self, other: "Point") -> float:
        return hypot(self.x - other.x, self.y - other.y)
@dataclass(frozen=True)
class HalfPlane:
    """线性半平面 a*x + b*y <= c。"""
    a: float
    b: float
    c: float
    label: str = ""
    def value(self, p: Point) -> float:
        return self.a * p.x + self.b * p.y - self.c
    def contains(self, p: Point, tol: float = EPS) -> bool:
        scale = max(1.0, abs(self.c), abs(self.a * p.x), abs(self.b * p.y))
        return self.value(p) <= tol * scale
@dataclass(frozen=True)
class Circle:
    center: Point
    radius: float
    def contains(self, p: Point, tol: float = 1e-8) -> bool:
        return self.center.distance_to(p) <= self.radius + tol * max(1.0, self.radius)
@dataclass(frozen=True)
class LocalizationResult:
    polygon: tuple[Point, ...]
    bounded: bool
    diameter: float | None
    diameter_pair: tuple[Point, Point] | None
    minimum_circle: Circle | None
    diameter_circle_covers: bool | None
    selected_diameter_pair_circle_covers: bool | None = None
@dataclass(frozen=True)
class _DirectedLine:
    """边界有向直线；可行半平面始终位于直线左侧。"""
    point: Point
    direction: Point
    halfplane: HalfPlane
    angle: float
def _cross(ax: float, ay: float, bx: float, by: float) -> float:
    return ax * by - ay * bx
def _unit(angle_deg: float) -> Point:
    angle = radians(angle_deg)
    return Point(cos(angle), sin(angle))
def _normalize_halfplane(hp: HalfPlane) -> HalfPlane:
```

```python
    norm = hypot(hp.a, hp.b)
    if norm <= EPS:
        raise ValueError("半平面法向量不能为零")
    return HalfPlane(hp.a / norm, hp.b / norm, hp.c / norm, hp.label)
def _to_directed_line(hp: HalfPlane) -> _DirectedLine:
    normalized = _normalize_halfplane(hp)
    point = Point(normalized.a * normalized.c, normalized.b * normalized.c)
    direction = Point(-normalized.b, normalized.a)
    angle = atan2(direction.y, direction.x) % (2.0 * pi)
    return _DirectedLine(point, direction, normalized, angle)
def _line_intersection(first: _DirectedLine, second: _DirectedLine) -> Point | None:
    denominator = _cross(
        first.direction.x,
        first.direction.y,
        second.direction.x,
        second.direction.y,
    )
    if abs(denominator) <= 1e-12:
        return None
    qpx = second.point.x - first.point.x
    qpy = second.point.y - first.point.y
    t = _cross(qpx, qpy, second.direction.x, second.direction.y) / denominator
    return Point(
        first.point.x + t * first.direction.x,
        first.point.y + t * first.direction.y,
    )
def _same_direction(first: _DirectedLine, second: _DirectedLine) -> bool:
    cross_value = _cross(
        first.direction.x,
        first.direction.y,
        second.direction.x,
        second.direction.y,
    )
    dot_value = (
        first.direction.x * second.direction.x
        + first.direction.y * second.direction.y
    )
    return abs(cross_value) <= 1e-12 and dot_value > 0.0
def _stricter_parallel_line(
    first: _DirectedLine, second: _DirectedLine
) -> _DirectedLine:
    if first.halfplane.contains(second.point):
        return second
    return first
def find_halfplane_feasible_point(
    halfplanes: Sequence[HalfPlane], tol: float = 1e-9
) -> Point | None:
    """二维增量线性可行性算法，返回任意可行点或None。"""
    normalized = [_normalize_halfplane(hp) for hp in halfplanes]
    current = Point(0.0, 0.0)
    for i, hp in enumerate(normalized):
        if hp.contains(current, tol):
            continue
        boundary_point = Point(hp.a * hp.c, hp.b * hp.c)
        direction = Point(-hp.b, hp.a)
        lower = float("-inf")
        upper = float("inf")
        for previous in normalized[:i]:
            coefficient = previous.a * direction.x + previous.b * direction.y
            rhs = previous.c - (
                previous.a * boundary_point.x + previous.b * boundary_point.y
            )
            if abs(coefficient) <= 1e-12:
                if rhs < -tol:
                    return None
                continue
            bound = rhs / coefficient
            if coefficient > 0.0:
                upper = min(upper, bound)
            else:
                lower = max(lower, bound)
            if lower > upper + tol:
                return None
        parameter = min(max(0.0, lower), upper)
        current = Point(
            boundary_point.x + parameter * direction.x,
            boundary_point.y + parameter * direction.y,
        )
    return current
def has_nonzero_recession_direction(
    halfplanes: Sequence[HalfPlane], tol: float = 1e-10
) -> bool:
    """判断半平面交是否存在可无限延伸的非零方向。"""
    normalized = [_normalize_halfplane(hp) for hp in halfplanes]
    candidates = [Point(1.0, 0.0), Point(0.0, 1.0)]
    for hp in normalized:
        candidates.append(Point(hp.b, -hp.a))
        candidates.append(Point(-hp.b, hp.a))
    return any(
        all(hp.a * d.x + hp.b * d.y <= tol for hp in normalized)
        for d in candidates
    )
def _convex_hull(points: Sequence[Point]) -> list[Point]:
    unique = sorted({(round(p.x, 12), round(p.y, 12)) for p in points})
    pts = [Point(x, y) for x, y in unique]
    if len(pts) <= 1:
        return pts
    def turn(o: Point, a: Point, b: Point) -> float:
        return _cross(a.x - o.x, a.y - o.y, b.x - o.x, b.y - o.y)
    lower: list[Point] = []
    for p in pts:
        while len(lower) >= 2 and turn(lower[-2], lower[-1], p) <= EPS:
            lower.pop()
        lower.append(p)
    upper: list[Point] = []
    for p in reversed(pts):
        while len(upper) >= 2 and turn(upper[-2], upper[-1], p) <= EPS:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]
def _bounded_degenerate_vertices(halfplanes: Sequence[HalfPlane]) -> list[Point]:
    """仅在退化/数值异常时枚举边界交点作为诊断性备用。"""
    lines = [_to_directed_line(hp) for hp in halfplanes]
    candidates: list[Point] = []
    for first, second in combinations(lines, 2):
        point = _line_intersection(first, second)
        if point is not None and all(hp.contains(point, 1e-7) for hp in halfplanes):
            candidates.append(point)
    feasible = find_halfplane_feasible_point(halfplanes)
    if feasible is not None:
        candidates.append(feasible)
    return _convex_hull(candidates)
def _signed_polygon_area(points: Sequence[Point]) -> float:
    if len(points) < 3:
        return 0.0
    return 0.5 * sum(
        points[i].x * points[(i + 1) % len(points)].y
        - points[(i + 1) % len(points)].x * points[i].y
        for i in range(len(points))
    )
def polygon_area(points: Sequence[Point]) -> float:
    return abs(_signed_polygon_area(points))
def halfplane_intersection_deque(
    halfplanes: Sequence[HalfPlane],
) -> tuple[list[Point], str]:
    """标准双端队列半平面交，主流程复杂度为O(n log n)。"""
    if not halfplanes:
        return [], "unbounded"
    feasible = find_halfplane_feasible_point(halfplanes)
    if feasible is None:
        return [], "empty"
    if has_nonzero_recession_direction(halfplanes):
        return [], "unbounded"
    lines = sorted((_to_directed_line(hp) for hp in halfplanes), key=lambda x: x.angle)
    merged: list[_DirectedLine] = []
    for line in lines:
        if merged and _same_direction(merged[-1], line):
            merged[-1] = _stricter_parallel_line(merged[-1], line)
        else:
            merged.append(line)
    if len(merged) > 1 and _same_direction(merged[0], merged[-1]):
        merged[0] = _stricter_parallel_line(merged[0], merged[-1])
        merged.pop()
    active: deque[_DirectedLine] = deque()
    for line in merged:
        while len(active) >= 2:
            intersection = _line_intersection(active[-2], active[-1])
            if intersection is not None and line.halfplane.contains(intersection, 1e-9):
                break
            active.pop()
        while len(active) >= 2:
            intersection = _line_intersection(active[0], active[1])
            if intersection is not None and line.halfplane.contains(intersection, 1e-9):
                break
            active.popleft()
        active.append(line)
    while len(active) >= 3:
        intersection = _line_intersection(active[-2], active[-1])
        if intersection is not None and active[0].halfplane.contains(intersection, 1e-9):
            break
        active.pop()
    while len(active) >= 3:
        intersection = _line_intersection(active[0], active[1])
        if intersection is not None and active[-1].halfplane.contains(intersection, 1e-9):
            break
        active.popleft()
    active_lines = list(active)
    vertices: list[Point] = []
    if len(active_lines) >= 3:
        for index, line in enumerate(active_lines):
            point = _line_intersection(line, active_lines[(index + 1) % len(active_lines)])
            if point is None:
                vertices = []
                break
```

```python
            vertices.append(point)
        vertices = _remove_near_duplicate_vertices(vertices)
    if len(vertices) < 3 or not all(
        hp.contains(vertex, 1e-7)
        for hp in halfplanes
        for vertex in vertices
    ):
        vertices = _bounded_degenerate_vertices(halfplanes)
    if len(vertices) >= 3 and _signed_polygon_area(vertices) < 0.0:
        vertices.reverse()
    return vertices, "bounded"
def bearing_halfplanes(
    station: Point,
    bearing_deg: float,
    error_deg: float = 1.0,
    measurement_index: int | None = None,
) -> tuple[HalfPlane, HalfPlane]:
    """把一次带误差的示向度转成两个半平面。
    从低角边界逆时针旋转到高角边界得到允许的角形区域。
    题目误差仅为1度，角宽小于180度，因此该表示是唯一的。
    """
    if not (0.0 <= error_deg < 90.0):
        raise ValueError("error_deg 必须位于 [0, 90) 度")
    low = _unit(bearing_deg - error_deg)
    high = _unit(bearing_deg + error_deg)
    prefix = f"m{measurement_index}:" if measurement_index is not None else ""
    # cross(low, p-station) >= 0
    hp_low = HalfPlane(
        a=low.y,
        b=-low.x,
        c=low.y * station.x - low.x * station.y,
        label=prefix + "lower",
    )
    # cross(high, p-station) <= 0
    hp_high = HalfPlane(
        a=-high.y,
        b=high.x,
        c=-high.y * station.x + high.x * station.y,
        label=prefix + "upper",
    )
    return hp_low, hp_high
def _segment_boundary_intersection(p: Point, q: Point, hp: HalfPlane) -> Point:
    """计算线段 pq 与半平面边界的交点。"""
    vp = hp.value(p)
    vq = hp.value(q)
    denominator = vp - vq
    if abs(denominator) <= EPS:
        return p
    t = vp / denominator
    return Point(p.x + t * (q.x - p.x), p.y + t * (q.y - p.y))
def clip_polygon_with_halfplane(
    polygon: Sequence[Point], hp: HalfPlane
) -> list[Point]:
    """Sutherland-Hodgman凸多边形裁剪。"""
    if not polygon:
        return []
    output: list[Point] = []
    previous = polygon[-1]
    previous_inside = hp.contains(previous)
    for current in polygon:
        current_inside = hp.contains(current)
        if current_inside:
            if not previous_inside:
                output.append(_segment_boundary_intersection(previous, current, hp))
            output.append(current)
        elif previous_inside:
            output.append(_segment_boundary_intersection(previous, current, hp))
        previous = current
        previous_inside = current_inside
    return _remove_near_duplicate_vertices(output)
def _remove_near_duplicate_vertices(points: Sequence[Point]) -> list[Point]:
    if not points:
        return []
    cleaned = [points[0]]
    for p in points[1:]:
        if p.distance_to(cleaned[-1]) > 1e-7:
            cleaned.append(p)
    if len(cleaned) > 1 and cleaned[0].distance_to(cleaned[-1]) <= 1e-7:
        cleaned.pop()
    return cleaned
def halfplane_intersection_by_clipping(
    halfplanes: Sequence[HalfPlane], bounding_radius: float = 100_000.0
) -> tuple[list[Point], bool]:
    """计算半平面交，并识别结果是否触及人工边界。
    竞赛目标区域尺度为1800米，默认边界取10万米。若裁剪结果仍触及
    该边界，则原半平面交被视为无界。正式论文可将此过程表述为标准
    半平面交算法；这里采用裁剪实现，便于调试和保证数值稳定。
    """
    if not (isfinite(bounding_radius) and bounding_radius > 0):
        raise ValueError("bounding_radius 必须是正的有限数")
    r = bounding_radius
    polygon: list[Point] = [Point(-r, -r), Point(r, -r), Point(r, r), Point(-r, r)]
    for hp in halfplanes:
        polygon = clip_polygon_with_halfplane(polygon, hp)
        if not polygon:
            return [], True
    margin = max(1e-5, r * 1e-8)
    touches_boundary = any(
        abs(abs(p.x) - r) <= margin or abs(abs(p.y) - r) <= margin for p in polygon
    )
    return polygon, not touches_boundary
def convex_polygon_diameter(
    polygon: Sequence[Point],
) -> tuple[float, tuple[Point, Point] | None]:
    """用旋转卡壳计算按逆时针排列的凸多边形直径。"""
    n = len(polygon)
    if n == 0:
        return 0.0, None
    if n == 1:
        return 0.0, (polygon[0], polygon[0])
    if n == 2:
        return polygon[0].distance_to(polygon[1]), (polygon[0], polygon[1])
    def doubled_area(i: int, j: int, k: int) -> float:
        a, b, c = polygon[i], polygon[j], polygon[k]
        return abs(_cross(b.x - a.x, b.y - a.y, c.x - a.x, c.y - a.y))
    best_distance = 0.0
    best_pair: tuple[Point, Point] | None = None
    j = 1
    for i in range(n):
        ni = (i + 1) % n
        while doubled_area(i, ni, (j + 1) % n) > doubled_area(i, ni, j) + EPS:
            j = (j + 1) % n
        for u, v in ((i, j), (ni, j)):
            distance = polygon[u].distance_to(polygon[v])
            if distance > best_distance:
                best_distance = distance
                best_pair = (polygon[u], polygon[v])
    return best_distance, best_pair
def _circle_from_two_points(a: Point, b: Point) -> Circle:
    center = Point((a.x + b.x) / 2.0, (a.y + b.y) / 2.0)
    return Circle(center, center.distance_to(a))
def circle_on_diameter_pair(pair: tuple[Point, Point]) -> Circle:
    """返回以给定点对连线为直径的圆。"""
    return _circle_from_two_points(pair[0], pair[1])
def diameter_pair_circle_covers_polygon(
    polygon: Sequence[Point], pair: tuple[Point, Point]
) -> bool:
    circle = circle_on_diameter_pair(pair)
    return all(circle.contains(point) for point in polygon)
def _circle_from_three_points(a: Point, b: Point, c: Point) -> Circle | None:
    d = 2.0 * (
        a.x * (b.y - c.y) + b.x * (c.y - a.y) + c.x * (a.y - b.y)
    )
    if abs(d) <= EPS:
        return None
    a2 = a.x * a.x + a.y * a.y
    b2 = b.x * b.x + b.y * b.y
    c2 = c.x * c.x + c.y * c.y
    ux = (a2 * (b.y - c.y) + b2 * (c.y - a.y) + c2 * (a.y - b.y)) / d
    uy = (a2 * (c.x - b.x) + b2 * (a.x - c.x) + c2 * (b.x - a.x)) / d
    center = Point(ux, uy)
    return Circle(center, center.distance_to(a))
def _smallest_circle_on_boundary_two(
    points: Sequence[Point], p: Point, q: Point
) -> Circle:
    diameter_circle = _circle_from_two_points(p, q)
    if all(diameter_circle.contains(r) for r in points):
        return diameter_circle
    candidates: list[Circle] = []
    for r in points:
        circle = _circle_from_three_points(p, q, r)
        if circle is not None and all(circle.contains(x) for x in points):
            candidates.append(circle)
    if not candidates:
        # 数值退化时，三点共线，最远点对的直径圆必然可行。
        farthest = max(combinations(points, 2), key=lambda pair: pair[0].distance_to(pair[1]))
        return _circle_from_two_points(*farthest)
    return min(candidates, key=lambda circle: circle.radius)
def minimum_enclosing_circle(points: Iterable[Point], seed: int = 2026) -> Circle:
    """随机增量法计算点集的最小包围圆，期望线性时间。"""
    shuffled = list(points)
    if not shuffled:
        return Circle(Point(0.0, 0.0), 0.0)
    Random(seed).shuffle(shuffled)
    circle: Circle | None = None
    for i, p in enumerate(shuffled):
        if circle is not None and circle.contains(p):
            continue
        circle = Circle(p, 0.0)
        for j, q in enumerate(shuffled[:i]):
            if circle.contains(q):
                continue
            circle = _circle_from_two_points(p, q)
            prefix = shuffled[:j]
            if any(not circle.contains(r) for r in prefix):
                circle = _smallest_circle_on_boundary_two(shuffled[: j + 1], p, q)
    assert circle is not None
```

```
    return circle
def build_localization_polygon(
    measurements: Sequence[tuple[Point, float]],
    error_deg: float = 1.0,
    bounding_radius: float = 100_000.0,
    method: str = "deque",
) -> LocalizationResult:
    """由若干(检测点, 示向度)构建定位多边形并计算问题1指标。"""
    halfplanes: list[HalfPlane] = []
    for index, (station, bearing) in enumerate(measurements,
       start=1):
        halfplanes.extend(
            bearing_halfplanes(station, bearing, error_deg,
      measurement_index=index)
        )
    if method == "deque":
        polygon, status = halfplane_intersection_deque(
      halfplanes)
        bounded = status != "unbounded"
    elif method == "clipping":
        polygon, bounded =
      halfplane_intersection_by_clipping(
            halfplanes, bounding_radius
        )
        status = "bounded" if bounded else "unbounded"
        if not polygon:
            status = "empty"
    else:
        raise ValueError("method必须是'deque'或'clipping'")
    if not polygon:
        return LocalizationResult(
            tuple(), status != "unbounded", None, None, None
      , None
        )
    if not bounded:
        return LocalizationResult(tuple(polygon), False,
      None, None, None, None)
    diameter, pair = convex_polygon_diameter(polygon)
    minimum_circle = minimum_enclosing_circle(polygon)
    covers = minimum_circle.radius <= diameter / 2.0 + 1e-7
      * max(1.0, diameter)
    selected_pair_covers = (
        diameter_pair_circle_covers_polygon(polygon, pair)
        if pair is not None
        else None
    )
    return LocalizationResult(
        tuple(polygon),
        True,
        diameter,
        pair,
        minimum_circle,
        covers,
        selected_pair_covers,
    )
```

coverage.py

```
"""问题3和问题4使用的确定性区域覆盖构造。"""
from __future__ import annotations
from math import ceil, cos, hypot, pi, sin, sqrt
from typing import Sequence
from .geometry import Point
def center_hexagon_points(
    center: Point = Point(0.0, 0.0), ring_radius_m: float =
      1200.0
) -> list[Point]:
    """返回中心点和正六边形的6个顶点。"""
    points = [center]
    for k in range(6):
        angle = k * pi / 3.0
        points.append(
            Point(
                center.x + ring_radius_m * cos(angle),
                center.y + ring_radius_m * sin(angle),
            )
        )
    return points
def center_hexagon_covering_radius(
    target_radius_m: float = 1800.0, ring_radius_m: float =
      1200.0
) -> float:
    """中心加正六边形对整个目标圆域的最坏最近点距离。
    由六折对称性，只需研究极角偏离最近六边形顶点不超过30度的扇
      形。
    最坏点只可能出现在中心点与环点Voronoi边界，或目标圆周的扇形
      中线。
    """
    c30 = cos(pi / 6.0)
    transition_distance = ring_radius_m / (2.0 * c30)
    boundary_distance = sqrt(
        target_radius_m * target_radius_m
        + ring_radius_m * ring_radius_m
        - 2.0 * target_radius_m * ring_radius_m * c30
    )
    return max(transition_distance, boundary_distance)
def feasible_hexagon_ring_interval(
    target_radius_m: float = 1800.0, receive_radius_m: float
      = 1000.0
) -> tuple[float, float]:
    """给出能够由7点完全覆盖目标圆域的六边形环半径区间。"""
    c30 = cos(pi / 6.0)
    discriminant = receive_radius_m**2 - target_radius_m**2
      * (1.0 - c30**2)
    if discriminant < 0:
        raise ValueError("给定接收半径无法用单层正六边形覆盖目
      标圆域")
    lower = target_radius_m * c30 - sqrt(discriminant)
    upper_from_boundary = target_radius_m * c30 + sqrt(
      discriminant)
    upper_from_transition = 2.0 * c30 * receive_radius_m
    return max(0.0, lower), min(upper_from_boundary,
      upper_from_transition)
def triangular_lattice_points(
    target_radius_m: float = 1800.0,
    spacing_m: float = 900.0,
    center: Point = Point(0.0, 0.0),
) -> list[Point]:
    """生成覆盖目标圆域及一层边界缓冲的三角格点。
    保留到圆心距离不超过 target_radius + spacing 的格点，可确
      保目标圆域中
    任一点所在的完整格网三角形三个顶点都被保留。spacing<=1000
      时，任一
    顶点到三角形内任一点的距离不超过最小接收半径1000米。
    """
    if spacing_m <= 0:
        raise ValueError("spacing_m 必须为正数")
    limit = target_radius_m + spacing_m
    row_height = spacing_m * sqrt(3.0) / 2.0
    max_row = ceil(limit / row_height) + 2
    max_col = ceil(limit / spacing_m) + 2
    points: list[Point] = []
    for row in range(-max_row, max_row + 1):
        y = row * row_height
        shift = 0.5 * spacing_m if row % 2 else 0.0
        for col in range(-max_col, max_col + 1):
            x = col * spacing_m + shift
            if hypot(x, y) <= limit + 1e-8:
                points.append(Point(center.x + x, center.y +
      y))
    # 蛇形排序便于机器狗逐行移动，避免每行末尾空跑回同一侧。
    def row_key(p: Point) -> tuple[int, float]:
        row = round((p.y - center.y) / row_height)
        local_x = p.x - center.x
        return row, local_x if row % 2 == 0 else -local_x
    return sorted(points, key=row_key)
def sampled_max_nearest_distance(
    scan_points: Sequence[Point],
    target_radius_m: float = 1800.0,
    radial_samples: int = 181,
    angular_samples: int = 720,
) -> tuple[float, Point]:
    """密集采样验证覆盖半径，返回最坏距离和对应采样点。"""
    if not scan_points:
        raise ValueError("scan_points 不能为空")
    worst_distance = -1.0
    worst_point = Point(0.0, 0.0)
    for i in range(radial_samples):
        radius = target_radius_m * i / max(1, radial_samples
      - 1)
        count = 1 if i == 0 else angular_samples
        for j in range(count):
            angle = 2.0 * pi * j / count
            point = Point(radius * cos(angle), radius * sin(
      angle))
            nearest = min(point.distance_to(scan) for scan
      in scan_points)
            if nearest > worst_distance:
                worst_distance = nearest
                worst_point = point
    return worst_distance, worst_point
def route_length(points: Sequence[Point], start: Point =
      Point(0.0, 0.0)) -> float:
    total = 0.0
    current = start
    for point in points:
        total += current.distance_to(point)
        current = point
    return total
```

station_selection.py

```
"""问题2: 第二个检测点的鲁棒选择。
第一次检测后，源点只知道位于示向度误差扇形内，距离上界取1500
  米。
为了不依赖未知的实际接收半径，第二测点必须使扇形内任意可能源点
到第二测点的距离不超过题目给出的最小接收半径1000米。
在满足接收保证的候选点中，再最大化最差交会角。
"""
from __future__ import annotations
from dataclasses import dataclass, replace
from functools import lru_cache
from math import acos, atan2, cos, degrees, hypot, pi,
      radians, sin
from typing import Iterable
from .geometry import EPS, Point
@dataclass(frozen=True)
class StationCandidate:
    point: Point
    forward_m: float
    lateral_m: float
    travel_distance_m: float
    worst_source_distance_m: float
    guaranteed_reception_margin_m: float
    approximate_min_crossing_angle_deg: float
    score: float
```

```python
    sampled_worst_localization_diameter_m: float | None = None
    sampled_worst_circle_radius_m: float | None = None
DIRECT_DIAMETER_FORWARD_M = 804.894658
DIRECT_DIAMETER_LATERAL_M = 600.052153
DIRECT_DIAMETER_VALIDATED_WORST_M = 134.065851525
PRACTICAL_FORWARD_M = 798.078289
PRACTICAL_LATERAL_M = 600.787616
PRACTICAL_RECEPTION_MARGIN_M = 5.0
PRACTICAL_WORST_CIRCLE_RADIUS_M = 67.61052882
def local_to_world(
    first_station: Point, bearing_deg: float, forward_m: float, lateral_m: float
) -> Point:
    """把沿首次示向线的局部坐标转回题目全局坐标。"""
    theta = radians(bearing_deg)
    ex = (cos(theta), sin(theta))
    ey = (-sin(theta), cos(theta))
    return Point(
        first_station.x + forward_m * ex[0] + lateral_m * ey[0],
        first_station.y + forward_m * ex[1] + lateral_m * ey[1],
    )
def _angles_equivalent_inside(value: float, low: float, high: float) -> list[float]:
    """返回与value相差2pi且落入[low, high]的表示。"""
    candidates: list[float] = []
    k_min = int((low - value) / (2 * pi)) - 1
    k_max = int((high - value) / (2 * pi)) + 1
    for k in range(k_min, k_max + 1):
        shifted = value + 2 * pi * k
        if low - EPS <= shifted <= high + EPS:
            candidates.append(shifted)
    return candidates
def worst_distance_to_initial_sector(
    forward_m: float,
    lateral_m: float,
    min_source_range_m: float = 5.0,
    max_source_range_m: float = 1500.0,
    bearing_error_deg: float = 1.0,
) -> float:
    """精确计算候选点到首次不确定扇形中最远点的距离。
    对固定角度，距离平方关于源点径向距离是凸函数，最大值位于径向
    区间端点；对固定径向距离，最大角差出现在角区间端点，或候选点
    反方向落入角区间时取该反方向。因此只需检查有限个极值候选。
    """
    delta = radians(bearing_error_deg)
    candidate_angle = atan2(lateral_m, forward_m)
    angle_candidates = [-delta, delta]
    angle_candidates.extend(
        _angles_equivalent_inside(candidate_angle + pi, -delta, delta)
    )
    worst = 0.0
    for source_range in (min_source_range_m, max_source_range_m):
        for phi in angle_candidates:
            source_x = source_range * cos(phi)
            source_y = source_range * sin(phi)
            worst = max(worst, hypot(source_x - forward_m, source_y - lateral_m))
    return worst
def _line_crossing_angle_deg(
    source_x: float, source_y: float, second_x: float, second_y: float
) -> float:
    """两条测向直线在源点处的锐交会角，范围0到90度。"""
    v1x, v1y = -source_x, -source_y
    v2x, v2y = second_x - source_x, second_y - source_y
    n1 = hypot(v1x, v1y)
    n2 = hypot(v2x, v2y)
    if n1 <= EPS or n2 <= EPS:
        return 90.0
    cosine = abs((v1x * v2x + v1y * v2y) / (n1 * n2))
    cosine = min(1.0, max(0.0, cosine))
    return degrees(acos(cosine))
def approximate_min_crossing_angle_deg(
    forward_m: float,
    lateral_m: float,
    min_source_range_m: float = 5.0,
    max_source_range_m: float = 1500.0,
    bearing_error_deg: float = 1.0,
    radial_samples: int = 81,
    angular_samples: int = 9,
) -> float:
    """在首次不确定扇形上离散计算最差交会角。"""
    minimum = 90.0
    for i in range(radial_samples):
        fraction = i / max(1, radial_samples - 1)
        source_range = min_source_range_m + fraction * (
            max_source_range_m - min_source_range_m
        )
        for j in range(angular_samples):
            angular_fraction = j / max(1, angular_samples - 1)
            phi_deg = -bearing_error_deg + 2 * bearing_error_deg * angular_fraction
            phi = radians(phi_deg)
            source_x = source_range * cos(phi)
            source_y = source_range * sin(phi)
            minimum = min(
                minimum,
                _line_crossing_angle_deg(
                    source_x, source_y, forward_m, lateral_m
                ),
            )
    return minimum
def generate_second_station_candidates(
    first_station: Point,
    bearing_deg: float,
    *,
    forward_values_m: Iterable[float] = range(550, 951, 10),
    lateral_values_m: Iterable[float] = range(250, 701, 10),
    min_source_range_m: float = 5.0,
    max_source_range_m: float = 1500.0,
    guaranteed_receive_radius_m: float = 1000.0,
    bearing_error_deg: float = 1.0,
    minimum_crossing_angle_deg: float = 25.0,
    movement_penalty_deg_per_m: float = 0.002,
) -> list[StationCandidate]:
    """生成左右对称的第二测点候选区，并按综合得分降序排列。"""
    candidates: list[StationCandidate] = []
    for forward_m in forward_values_m:
        for lateral_abs_m in lateral_values_m:
            for lateral_m in (-float(lateral_abs_m), float(lateral_abs_m)):
                worst_distance = worst_distance_to_initial_sector(
                    float(forward_m),
                    lateral_m,
                    min_source_range_m,
                    max_source_range_m,
                    bearing_error_deg,
                )
                if worst_distance > guaranteed_receive_radius_m + 1e-8:
                    continue
                min_angle = approximate_min_crossing_angle_deg(
                    float(forward_m),
                    lateral_m,
                    min_source_range_m,
                    max_source_range_m,
                    bearing_error_deg,
                )
                if min_angle + 1e-10 < minimum_crossing_angle_deg:
                    continue
                travel_distance = hypot(float(forward_m), lateral_m)
                margin = guaranteed_receive_radius_m - worst_distance
                score = min_angle - movement_penalty_deg_per_m * travel_distance
                candidates.append(
                    StationCandidate(
                        point=local_to_world(
                            first_station, bearing_deg, float(forward_m), lateral_m
                        ),
                        forward_m=float(forward_m),
                        lateral_m=lateral_m,
                        travel_distance_m=travel_distance,
                        worst_source_distance_m=worst_distance,
                        guaranteed_reception_margin_m=margin,
                        approximate_min_crossing_angle_deg=min_angle,
                        score=score,
                    )
                )
    candidates.sort(
        key=lambda item: (
            -item.score,
            -item.guaranteed_reception_margin_m,
            item.travel_distance_m,
            abs(item.lateral_m),
        )
    )
    return candidates
def choose_second_station(
    first_station: Point, bearing_deg: float, **kwargs: object
) -> StationCandidate:
    if not kwargs:
        local_candidate = _default_local_candidate()
        return replace(
            local_candidate,
            point=local_to_world(
                first_station,
                bearing_deg,
                local_candidate.forward_m,
                local_candidate.lateral_m,
            ),
        )
    candidates = generate_second_station_candidates(
        first_station, bearing_deg, **kwargs
    )
    if not candidates:
        raise ValueError("当前参数下没有同时满足接收与交会角要求的第二测点")
    return candidates[0]
def choose_second_station_by_direct_diameter(
    first_station: Point,
    bearing_deg: float,
    *,
    side: int = 1,
) -> StationCandidate:
    """返回标准参数下直接最小化最坏定位直径得到的数值推荐点。
    side=1与side=-1分别选择首次示向线左、右两侧的对称点。该结
```

```
    果适用
    于1度误差、源点距离(5,1500]米、保证接收半径1000米的题设参
      数。
    """
    if side not in (-1, 1):
        raise ValueError("side必须是-1或1")
    forward_m = DIRECT_DIAMETER_FORWARD_M
    lateral_m = side * DIRECT_DIAMETER_LATERAL_M
    worst_distance = worst_distance_to_initial_sector(
      forward_m, lateral_m)
    min_angle = approximate_min_crossing_angle_deg(
        forward_m,
        lateral_m,
        radial_samples=301,
        angular_samples=21,
    )
    return StationCandidate(
        point=local_to_world(first_station, bearing_deg,
      forward_m, lateral_m),
        forward_m=forward_m,
        lateral_m=lateral_m,
        travel_distance_m=hypot(forward_m, lateral_m),
        worst_source_distance_m=worst_distance,
        guaranteed_reception_margin_m=1000.0 -
      worst_distance,
        approximate_min_crossing_angle_deg=min_angle,
        score=-DIRECT_DIAMETER_VALIDATED_WORST_M,
        sampled_worst_localization_diameter_m=
      DIRECT_DIAMETER_VALIDATED_WORST_M,
        sampled_worst_circle_radius_m=
      DIRECT_DIAMETER_VALIDATED_WORST_M / 2.0,
    )
def choose_second_station_by_worst_circle(
    first_station: Point,
    bearing_deg: float,
    *,
    side: int = 1,
) -> StationCandidate:
    """问题二纯定位目标：最小化最坏最小包围圆半径。"""
    return choose_second_station_by_direct_diameter(
        first_station,
        bearing_deg,
        side=side,
    )
def choose_practical_second_station(
    first_station: Point,
    bearing_deg: float,
    *,
    side: int = 1,
) -> StationCandidate:
    """返回保留5米接收余量的工程推荐点，供问题3/4继续规划。"""
    if side not in (-1, 1):
        raise ValueError("side必须是-1或1")
    forward_m = PRACTICAL_FORWARD_M
    lateral_m = side * PRACTICAL_LATERAL_M
    worst_distance = worst_distance_to_initial_sector(
      forward_m, lateral_m)
    min_angle = approximate_min_crossing_angle_deg(
        forward_m,
        lateral_m,
        radial_samples=301,
        angular_samples=21,
    )
    return StationCandidate(
        point=local_to_world(first_station, bearing_deg,
      forward_m, lateral_m),
        forward_m=forward_m,
        lateral_m=lateral_m,
        travel_distance_m=hypot(forward_m, lateral_m),
        worst_source_distance_m=worst_distance,
        guaranteed_reception_margin_m=1000.0 -
      worst_distance,
        approximate_min_crossing_angle_deg=min_angle,
        score=-PRACTICAL_WORST_CIRCLE_RADIUS_M,
        sampled_worst_localization_diameter_m=2.0
        * PRACTICAL_WORST_CIRCLE_RADIUS_M,
        sampled_worst_circle_radius_m=
      PRACTICAL_WORST_CIRCLE_RADIUS_M,
    )
@lru_cache(maxsize=1)
def _default_local_candidate() -> StationCandidate:
    """默认候选只计算一次；平移旋转不会改变距离与交会角指标。
      """
    candidates = generate_second_station_candidates(Point
      (0.0, 0.0), 0.0)
    if not candidates:
        raise RuntimeError("默认参数未生成第二测点候选")
    return candidates[0]
```

problem2_optimization.py

```
"""问题2的多目标第二测点优化与统一评价。"""
from __future__ import annotations
from dataclasses import asdict, dataclass
from math import acos, atan2, cos, degrees, hypot, log, pi,
      radians, sin, sqrt
from statistics import fmean
from typing import Callable, Iterable, Sequence
from scipy.optimize import differential_evolution,
      minimize_scalar
from .geometry import LocalizationResult, Point,
      build_localization_polygon
from .station_selection import
      worst_distance_to_initial_sector

@dataclass(frozen=True)
class UncertaintyScenario:
    source_range_m: float
    source_angle_deg: float
    second_error_deg: float
@dataclass(frozen=True)
class StrategyResult:
    strategy: str
    forward_m: float
    lateral_m: float
    travel_m: float
    worst_receive_distance_m: float
    minimum_crossing_angle_deg: float
    worst_diameter_m: float
    mean_diameter_m: float
    p95_diameter_m: float
    worst_e_optimal_scale: float
    minimum_d_optimal_information: float
    worst_scenario: tuple[float, float, float]
    def to_dict(self) -> dict[str, object]:
        return asdict(self)
@dataclass(frozen=True)
class ActiveArcCircleSolution:
    receive_limit_m: float
    reception_margin_m: float
    arc_parameter_deg: float
    forward_m: float
    lateral_m: float
    travel_m: float
    worst_circle_radius_m: float
    worst_diameter_m: float
    worst_scenario: tuple[float, float, float]
def linspace(low: float, high: float, count: int) -> list[
      float]:
    if count <= 1:
        return [(low + high) / 2.0]
    return [low + (high - low) * i / (count - 1) for i in
      range(count)]
def scenario_grid(
    radial_samples: int,
    angular_samples: int,
    error_samples: int,
) -> list[UncertaintyScenario]:
    return [
        UncertaintyScenario(rho, phi, error)
        for rho in linspace(5.001, 1500.0, radial_samples)
        for phi in linspace(-1.0, 1.0, angular_samples)
        for error in linspace(-1.0, 1.0, error_samples)
    ]
def source_point(scenario: UncertaintyScenario) -> Point:
    angle = radians(scenario.source_angle_deg)
    return Point(
        scenario.source_range_m * cos(angle),
        scenario.source_range_m * sin(angle),
    )
def localization_result_for_scenario(
    forward_m: float,
    lateral_m: float,
    scenario: UncertaintyScenario,
) -> LocalizationResult:
    source = source_point(scenario)
    second_bearing = degrees(
        atan2(source.y - lateral_m, source.x - forward_m)
    ) + scenario.second_error_deg
    return build_localization_polygon(
        [
            (Point(0.0, 0.0), 0.0),
            (Point(forward_m, lateral_m), second_bearing),
        ]
    )
def localization_diameter_for_scenario(
    forward_m: float,
    lateral_m: float,
    scenario: UncertaintyScenario,
) -> float:
    result = localization_result_for_scenario(forward_m,
      lateral_m, scenario)
    if not result.bounded or result.diameter is None:
        return float("inf")
    return result.diameter
def localization_circle_radius_for_scenario(
    forward_m: float,
    lateral_m: float,
    scenario: UncertaintyScenario,
) -> float:
    """返回该场景定位多边形的最小包围圆半径。"""
    result = localization_result_for_scenario(forward_m,
      lateral_m, scenario)
    if not result.bounded or result.minimum_circle is None:
        return float("inf")
    return result.minimum_circle.radius
def crossing_angle_deg(forward_m: float, lateral_m: float,
      source: Point) -> float:
    v1x, v1y = -source.x, -source.y
    v2x, v2y = forward_m - source.x, lateral_m - source.y
    denominator = hypot(v1x, v1y) * hypot(v2x, v2y)
    if denominator <= 1e-12:
        return 90.0
    cosine = abs((v1x * v2x + v1y * v2y) / denominator)
    cosine = min(1.0, max(0.0, cosine))
    return degrees(acos(cosine))
def information_metrics(
    forward_m: float,
    lateral_m: float,
    source: Point,
) -> tuple[float, float]:
    """返回(E最优误差尺度, D最优信息量)，忽略公共角噪声常数。
      """
```

```
    rows: list[tuple[float, float]] = []
    for sx, sy in ((0.0, 0.0), (forward_m, lateral_m)):
        dx, dy = source.x - sx, source.y - sy
        distance_sq = dx * dx + dy * dy
        if distance_sq <= 1e-12:
            return 0.0, float("inf")
        rows.append((-dy / distance_sq, dx / distance_sq))
    ixx = sum(row[0] * row[0] for row in rows)
    ixy = sum(row[0] * row[1] for row in rows)
    iyy = sum(row[1] * row[1] for row in rows)
    determinant = max(0.0, ixx * iyy - ixy * ixy)
    trace = ixx + iyy
    discriminant = max(0.0, trace * trace - 4.0 * determinant)
    lambda_min = max(0.0, (trace - sqrt(discriminant)) / 2.0)
    e_scale = float("inf") if lambda_min <= 1e-24 else 1.0 / sqrt(lambda_min)
    return e_scale, determinant
def _source_scenarios_from_grid(
    scenarios: Sequence[UncertaintyScenario],
) -> list[UncertaintyScenario]:
    unique = {
        (scenario.source_range_m, scenario.source_angle_deg): scenario
        for scenario in scenarios
    }
    return list(unique.values())
def evaluate_candidate(
    strategy: str,
    forward_m: float,
    lateral_m: float,
    scenarios: Sequence[UncertaintyScenario],
) -> StrategyResult:
    diameters: list[float] = []
    worst_diameter = -1.0
    worst_scenario = scenarios[0]
    for scenario in scenarios:
        diameter = localization_diameter_for_scenario(
            forward_m, lateral_m, scenario
        )
        diameters.append(diameter)
        if diameter > worst_diameter:
            worst_diameter = diameter
            worst_scenario = scenario
    ordered = sorted(diameters)
    p95_index = min(len(ordered) - 1, int(0.95 * (len(ordered) - 1)))
    minimum_angle = 90.0
    worst_e_scale = 0.0
    minimum_d_info = float("inf")
    for scenario in _source_scenarios_from_grid(scenarios):
        source = source_point(scenario)
        minimum_angle = min(
            minimum_angle,
            crossing_angle_deg(forward_m, lateral_m, source),
        )
        e_scale, d_info = information_metrics(forward_m, lateral_m, source)
        worst_e_scale = max(worst_e_scale, e_scale)
        minimum_d_info = min(minimum_d_info, d_info)
    return StrategyResult(
        strategy=strategy,
        forward_m=forward_m,
        lateral_m=lateral_m,
        travel_m=hypot(forward_m, lateral_m),
        worst_receive_distance_m=worst_distance_to_initial_sector(
            forward_m, lateral_m
        ),
        minimum_crossing_angle_deg=minimum_angle,
        worst_diameter_m=worst_diameter,
        mean_diameter_m=fmean(diameters),
        p95_diameter_m=ordered[p95_index],
        worst_e_optimal_scale=worst_e_scale,
        minimum_d_optimal_information=minimum_d_info,
        worst_scenario=(
            worst_scenario.source_range_m,
            worst_scenario.source_angle_deg,
            worst_scenario.second_error_deg,
        ),
    )
class Problem2MultiObjectiveOptimizer:
    """在同一必接收候选域上连续优化多种准则。"""
    def __init__(self, receive_limit_m: float = 1000.0) -> None:
        if not 0.0 < receive_limit_m <= 1000.0:
            raise ValueError("receive_limit_m必须位于(0,1000]")
        self.receive_limit_m = receive_limit_m
        self.optimization_scenarios = scenario_grid(7, 3, 3)
        self.mean_scenarios = scenario_grid(21, 5, 5)
        self.source_scenarios = _source_scenarios_from_grid(
            self.optimization_scenarios
        )
        self.bounds = [(450.0, 1050.0), (100.0, 750.0)]
    def _constraint_penalty(self, forward_m: float, lateral_m: float) -> float:
        excess = (
            worst_distance_to_initial_sector(forward_m, lateral_m)
            - self.receive_limit_m
        )
        return 0.0 if excess <= 0.0 else 1_000_000.0 + 10_000.0 * excess
    def _diameters(self, forward_m: float, lateral_m: float) -> list[float]:
        return [
            localization_diameter_for_scenario(forward_m, lateral_m, scenario)
            for scenario in self.optimization_scenarios
        ]
    def _objective(self, strategy: str) -> Callable[[Sequence[float]], float]:
        def objective(values: Sequence[float]) -> float:
            forward_m, lateral_m = float(values[0]), float(values[1])
            penalty = self._constraint_penalty(forward_m, lateral_m)
            if penalty:
                return penalty
            if strategy in {
                "minimax_diameter",
                "minimax_circle_radius",
                "mean_diameter",
                "epsilon_time",
            }:
                if strategy == "minimax_circle_radius":
                    return max(
                        localization_circle_radius_for_scenario(
                            forward_m, lateral_m, scenario
                        )
                        for scenario in self.optimization_scenarios
                    )
                diameter_scenarios = (
                    self.mean_scenarios
                    if strategy == "mean_diameter"
                    else self.optimization_scenarios
                )
                diameters = [
                    localization_diameter_for_scenario(
                        forward_m, lateral_m, scenario
                    )
                    for scenario in diameter_scenarios
                ]
                worst = max(diameters)
                if strategy == "minimax_diameter":
                    return worst
                if strategy == "mean_diameter":
                    return fmean(diameters)
                excess = worst - 135.0
                if excess > 0.0:
                    return 100_000.0 + 10_000.0 * excess
                return hypot(forward_m, lateral_m)
            angles: list[float] = []
            e_scales: list[float] = []
            d_infos: list[float] = []
            for scenario in self.source_scenarios:
                source = source_point(scenario)
                angles.append(crossing_angle_deg(forward_m, lateral_m, source))
                e_scale, d_info = information_metrics(
                    forward_m, lateral_m, source
                )
                e_scales.append(e_scale)
                d_infos.append(d_info)
            if strategy == "maximin_angle":
                return -min(angles)
            if strategy == "e_optimal":
                return max(e_scales)
            if strategy == "d_optimal":
                return -log(max(min(d_infos), 1e-300))
            raise ValueError(f"未知策略: {strategy}")
        return objective
    def optimize(
        self,
        strategy: str,
        *,
        seeds: Iterable[int] = (2026, 2027, 2028),
    ) -> tuple[float, float, list[float]]:
        objective = self._objective(strategy)
        is_robust_geometry = strategy in {
            "minimax_diameter",
            "minimax_circle_radius",
        }
        popsize = 16 if is_robust_geometry else 8
        maxiter = 80 if is_robust_geometry else 30
        runs = []
        for seed in seeds:
            result = differential_evolution(
                objective,
                bounds=self.bounds,
                seed=seed,
                popsize=popsize,
                maxiter=maxiter,
                tol=1e-9 if is_robust_geometry else 1e-6,
                polish=True,
                workers=1,
            )
            runs.append(result)
        best = min(runs, key=lambda result: float(result.fun))
        return (
            float(best.x[0]),
            float(best.x[1]),
            [float(result.fun) for result in runs],
        )
    def optimize_epsilon_time(
        self,
        diameter_limit_m: float,
        *,
        seeds: Iterable[int] = (5201, 5202),
    ) -> tuple[float, float, list[float]]:
```

```
        """在最坏直径不超过给定阈值时最小化第二站移动距离。"""
        def objective(values: Sequence[float]) -> float:
            forward_m, lateral_m = float(values[0]), float(values[1])
            penalty = self._constraint_penalty(forward_m, lateral_m)
            if penalty:
                return penalty
            worst = max(self._diameters(forward_m, lateral_m))
            excess = worst - diameter_limit_m
            if excess > 0.0:
                return 100_000.0 + 10_000.0 * excess
            return hypot(forward_m, lateral_m)
        runs = []
        for seed in seeds:
            result = differential_evolution(
                objective,
                bounds=self.bounds,
                seed=seed,
                popsize=8,
                maxiter=34,
                tol=1e-7,
                polish=True,
                workers=1,
            )
            runs.append(result)
        best = min(runs, key=lambda result: float(result.fun))
        return (
            float(best.x[0]),
            float(best.x[1]),
            [float(result.fun) for result in runs],
        )
def optimize_active_arc_circle_radius(
    receive_limit_m: float = 1000.0,
    *,
    angular_grid_size: int = 4001,
) -> ActiveArcCircleSolution:
    """利用活跃接收约束，把最坏包围圆半径优化降为一维圆弧搜索。
    该方法用于题设标准参数和其保守接收余量变体；结果仍通过全部角点
    场景取最大值，不预先假定是哪一个角点控制目标。
    """
    if not 0.0 < receive_limit_m <= 1000.0:
        raise ValueError("receive_limit_m必须位于(0,1000]")
    delta = radians(1.0)
    active_center = Point(5.0 * cos(-delta), 5.0 * sin(-delta))
    scenarios = [
        UncertaintyScenario(rho, phi, error)
        for rho in (5.001, 1500.0)
        for phi in (-1.0, 1.0)
        for error in (-1.0, 1.0)
    ]
    def point_on_arc(angle: float) -> Point:
        return Point(
            active_center.x + receive_limit_m * cos(angle),
            active_center.y + receive_limit_m * sin(angle),
        )
    feasible_angles: list[float] = []
    for index in range(18001):
        angle = (pi / 2.0) * index / 18000.0
        point = point_on_arc(angle)
        if (
            point.y >= 0.0
            and worst_distance_to_initial_sector(point.x, point.y)
            <= receive_limit_m + 1e-8
        ):
            feasible_angles.append(angle)
    if not feasible_angles:
        raise ValueError("给定接收限制下活跃圆弧没有可行点")
    lower, upper = min(feasible_angles), max(feasible_angles)
    def radius_objective(angle: float) -> float:
        point = point_on_arc(angle)
        return max(
            localization_circle_radius_for_scenario(point.x, point.y, scenario)
            for scenario in scenarios
        )
    grid_angles = [
        lower + (upper - lower) * index / (angular_grid_size - 1)
        for index in range(angular_grid_size)
    ]
    grid_values = [radius_objective(angle) for angle in grid_angles]
    best_index = min(range(len(grid_values)), key=grid_values.__getitem__)
    left = grid_angles[max(0, best_index - 2)]
    right = grid_angles[min(len(grid_angles) - 1, best_index + 2)]
    result = minimize_scalar(
        radius_objective,
        bounds=(left, right),
        method="bounded",
        options={"xatol": 1e-14},
    )
    point = point_on_arc(float(result.x))
    scenario_values = [
        (
            localization_circle_radius_for_scenario(point.x, point.y, scenario),
            scenario,
        )
        for scenario in scenarios
    ]
    worst_radius, worst_scenario = max(scenario_values, key=lambda item: item[0])
    worst_diameter = localization_diameter_for_scenario(
        point.x, point.y, worst_scenario
    )
    return ActiveArcCircleSolution(
        receive_limit_m=receive_limit_m,
        reception_margin_m=1000.0 - receive_limit_m,
        arc_parameter_deg=degrees(float(result.x)),
        forward_m=point.x,
        lateral_m=point.y,
        travel_m=hypot(point.x, point.y),
        worst_circle_radius_m=worst_radius,
        worst_diameter_m=worst_diameter,
        worst_scenario=(
            worst_scenario.source_range_m,
            worst_scenario.source_angle_deg,
            worst_scenario.second_error_deg,
        ),
    )
```

problem3_planner.py

```
"""问题3全向干扰源规划器：保留首版，并提供巡视定位融合的优化版。"""
from __future__ import annotations
# 论文附录版：仅保留最终算法与消融基线所用的类，
# 完整工程（含其余变体与实验脚本）见支撑材料 code/ 目录。
from dataclasses import dataclass, field
from math import acos, atan2, ceil, cos, degrees, pi, radians, sin, sqrt
from random import Random
from typing import Protocol
from .coverage import (
    center_hexagon_points,
    feasible_hexagon_ring_interval,
)
from .geometry import (
    HalfPlane,
    Point,
    bearing_halfplanes,
    build_localization_polygon,
    clip_polygon_with_halfplane,
    minimum_enclosing_circle,
)
from .problem2_optimization import (
    UncertaintyScenario,
    crossing_angle_deg,
    localization_circle_radius_for_scenario,
    source_point,
)
from .station_selection import (
    PRACTICAL_FORWARD_M,
    PRACTICAL_LATERAL_M,
    choose_second_station,
    local_to_world,
    worst_distance_to_initial_sector,
)
class RobotInterface(Protocol):
    def enter(self) -> dict[str, object]: ...
    def measure(self, x: float, y: float, channel: int) -> dict[str, object]: ...
    def clear(self, x: float, y: float, channel: int) -> dict[str, object]: ...
    def exit(self) -> dict[str, object]: ...
@dataclass(frozen=True)
class BearingObservation:
    station: Point
    bearing_deg: float
@dataclass
class ChannelRecord:
    channel: int
    observations: list[BearingObservation] = field(default_factory=list)
    no_signal_stations: list[Point] = field(default_factory=list)
    cleared: bool = False
@dataclass(frozen=True)
class PlanningReport:
    detected_channels: tuple[int, ...]
    cleared_channels: tuple[int, ...]
    unknown_channels_after_cover: tuple[int, ...]
    exit_response: dict[str, object]
class LegacyProblem3Planner:
    def __init__(
        self,
        *,
        bearing_error_deg: float = 1.0,
        cover_ring_radius_m: float = 1200.0,
        max_refinement_measurements: int = 5,
    ) -> None:
        self.bearing_error_deg = bearing_error_deg
        self.cover_points = center_hexagon_points(ring_radius_m=cover_ring_radius_m)
        self.max_refinement_measurements = max_refinement_measurements
        self.records = {channel: ChannelRecord(channel) for channel in range(1, 21)}
    def run(self, robot: RobotInterface) -> PlanningReport:
        robot.enter()
        unknown = set(range(1, 21))
        detected: set[int] = set()
```

```python
        current_channel = 1
        for point in self.cover_points:
            ordered_channels = sorted(unknown, key=lambda channel: channel != current_channel)
            for channel in ordered_channels:
                response = robot.measure(point.x, point.y, channel)
                current_channel = channel
                result = response["measure_result"]
                if result == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] != "success":
                        raise RuntimeError("near后在原点清除失败，模拟器状态不一致")
                    self.records[channel].cleared = True
                    detected.add(channel)
                    unknown.remove(channel)
                elif result == "direction":
                    self.records[channel].observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                    detected.add(channel)
                    unknown.remove(channel)
            if len(detected) == 16:
                break
        current_position = point
        pending = [
            self.records[channel]
            for channel in detected
            if not self.records[channel].cleared
        ]
        while pending:
            record, second_point = min(
                (
                    (record, candidate)
                    for record in pending
                    for candidate in self._symmetric_second_points(record)
                ),
                key=lambda item: current_position.distance_to(item[1]),
            )
            current_position = self._localize_and_clear(
                robot, record, second_point
            )
            pending.remove(record)
        exit_response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return PlanningReport(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            unknown_channels_after_cover=tuple(sorted(unknown)),
            exit_response=exit_response,
        )
    @staticmethod
    def _symmetric_second_points(record: ChannelRecord) -> tuple[Point, Point]:
        first = record.observations[0]
        base = choose_second_station(first.station, first.bearing_deg)
        lateral = abs(base.lateral_m)
        return (
            local_to_world(first.station, first.bearing_deg, base.forward_m, -lateral),
            local_to_world(first.station, first.bearing_deg, base.forward_m, lateral),
        )
    def _localize_and_clear(
        self, robot: RobotInterface, record: ChannelRecord, second_point: Point
    ) -> Point:
        response = robot.measure(second_point.x, second_point.y, record.channel)
        if response["measure_result"] == "near":
            self._clear_or_raise(robot, second_point, record)
            return second_point
        if response["measure_result"] != "direction":
            raise RuntimeError(
                f"频道{record.channel}的鲁棒第二测点未接收到信号，需检查几何假设"
            )
        record.observations.append(
            BearingObservation(second_point, float(response["svd_deg"]))
        )
        for _ in range(self.max_refinement_measurements):
            result = build_localization_polygon(
                [(item.station, item.bearing_deg) for item in record.observations],
                error_deg=self.bearing_error_deg,
            )
            if not result.polygon or not result.bounded or result.minimum_circle is None:
                raise RuntimeError(f"频道{record.channel}未形成有效有界定位区域")
            probe = result.minimum_circle.center
            if result.minimum_circle.radius <= 20.0 - 1e-6:
                self._clear_or_raise(robot, probe, record)
                return probe
            response = robot.measure(probe.x, probe.y, record.channel)
            if response["measure_result"] == "near":
                self._clear_or_raise(robot, probe, record)
                return probe
            if response["measure_result"] != "direction":
                raise RuntimeError(
                    f"频道{record.channel}在定位区域中心未接收到信号"
                )
            record.observations.append(
                BearingObservation(probe, float(response["svd_deg"]))
            )
        result = build_localization_polygon(
            [(item.station, item.bearing_deg) for item in record.observations],
            error_deg=self.bearing_error_deg,
        )
        return self._clear_polygon_by_grid(robot, result.polygon, record)
    @staticmethod
    def _clear_or_raise(
        robot: RobotInterface, point: Point, record: ChannelRecord
    ) -> None:
        response = robot.clear(point.x, point.y, record.channel)
        if response["clear_result"] != "success":
            raise RuntimeError(f"频道{record.channel}在保证半径内清除失败")
        record.cleared = True
    @staticmethod
    def _clear_polygon_by_grid(
        robot: RobotInterface, polygon: tuple[Point, ...], record: ChannelRecord
    ) -> Point:
        if not polygon:
            raise RuntimeError("定位区域为空，无法执行兜底覆盖")
        step = 20.0 * sqrt(2.0)
        min_x = min(p.x for p in polygon)
        max_x = max(p.x for p in polygon)
        min_y = min(p.y for p in polygon)
        max_y = max(p.y for p in polygon)
        nx = max(1, ceil((max_x - min_x) / step))
        ny = max(1, ceil((max_y - min_y) / step))
        for row in range(ny + 1):
            y = min_y + (max_y - min_y) * row / ny
            columns = range(nx + 1) if row % 2 == 0 else range(nx, -1, -1)
            for col in columns:
                x = min_x + (max_x - min_x) * col / nx
                response = robot.clear(x, y, record.channel)
                if response["clear_result"] == "success":
                    record.cleared = True
                    return Point(x, y)
        raise RuntimeError(f"频道{record.channel}的兜底网格仍未清除目标")
@dataclass(frozen=True)
class ServiceVariant:
    """一个待处理频道的入口测点及预计清除位置。"""
    entry: Point
    predicted_exit: Point
@dataclass(frozen=True)
class OpportunisticMeasurement:
    """位于剩余巡视折线上的零额外路程测量。"""
    segment_index: int
    fraction: float
    point: Point
    worst_circle_radius_m: float
class Problem3Planner:
    """巡视定位融合、滚动路径优化并带降级分支的问题3规划器。"""
    def __init__(
        self,
        *,
        bearing_error_deg: float = 1.0,
        cover_ring_radius_m: float | None = None,
        reception_margin_m: float = 5.0,
        opportunistic_circle_limit_m: float = 110.0,
        opportunistic_min_crossing_angle_deg: float = 10.0,
        opportunistic_samples_per_segment: int = 41,
        enable_route_local_search: bool = True,
        route_search_mode: str = "local",
        predicted_source_range_m: float = 625.0,
        max_refinement_measurements: int = 5,
    ) -> None:
        if cover_ring_radius_m is None:
            cover_ring_radius_m = feasible_hexagon_ring_interval()[0]
        if opportunistic_samples_per_segment < 3:
            raise ValueError("每段顺路候选采样数至少为3")
        if route_search_mode not in {"local", "exact"}:
            raise ValueError("route_search_mode必须是local或exact")
        self.bearing_error_deg = bearing_error_deg
        self.cover_points = center_hexagon_points(
            ring_radius_m=cover_ring_radius_m
        )
        self.reception_margin_m = reception_margin_m
        self.opportunistic_circle_limit_m = opportunistic_circle_limit_m
        self.opportunistic_min_crossing_angle_deg = (
            opportunistic_min_crossing_angle_deg
        )
```

```python
        self.opportunistic_samples_per_segment = opportunistic_samples_per_segment
        self.enable_route_local_search = enable_route_local_search
        self.route_search_mode = route_search_mode
        self.predicted_source_range_m = predicted_source_range_m
        self.max_refinement_measurements = max_refinement_measurements
        self.records = {channel: ChannelRecord(channel) for channel in range(1, 21)}
        self._corner_scenarios = tuple(
            UncertaintyScenario(rho, phi, error)
            for rho in (5.001, 1500.0)
            for phi in (-bearing_error_deg, bearing_error_deg)
            for error in (-bearing_error_deg, bearing_error_deg)
        )
    def run(self, robot: RobotInterface) -> PlanningReport:
        robot.enter()
        unknown = set(range(1, 21))
        detected: set[int] = set()
        scheduled: dict[int, OpportunisticMeasurement] = {}
        current_channel = 1
        current_position = Point(0.0, 0.0)
        for index, point in enumerate(self.cover_points):
            ordered_channels = sorted(
                unknown, key=lambda channel: channel != current_channel
            )
            for channel in ordered_channels:
                response = robot.measure(point.x, point.y, channel)
                current_position = point
                current_channel = channel
                result = response["measure_result"]
                if result == "near":
                    if self._attempt_clear(robot, point, self.records[channel]):
                        detected.add(channel)
                        unknown.remove(channel)
                elif result == "direction":
                    record = self.records[channel]
                    record.observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                    detected.add(channel)
                    unknown.remove(channel)
                    opportunity = self._best_remaining_route_measurement(
                        record, start_segment=index
                    )
                    if opportunity is not None:
                        scheduled[channel] = opportunity
                else:
                    self.records[channel].no_signal_stations.append(point)
            if len(detected) == 16:
                break
            if index >= len(self.cover_points) - 1:
                continue
            events = sorted(
                (
                    (channel, opportunity)
                    for channel, opportunity in scheduled.items()
                    if opportunity.segment_index == index
                ),
                key=lambda item: item[1].fraction,
            )
            for channel, opportunity in events:
                record = self.records[channel]
                response = robot.measure(
                    opportunity.point.x,
                    opportunity.point.y,
                    channel,
                )
                current_position = opportunity.point
                current_channel = channel
                if response["measure_result"] == "near":
                    self._attempt_clear(robot, opportunity.point, record)
                elif response["measure_result"] == "direction":
                    record.observations.append(
                        BearingObservation(
                            opportunity.point,
                            float(response["svd_deg"]),
                        )
                    )
                else:
                    record.no_signal_stations.append(opportunity.point)
                del scheduled[channel]
        pending = [
            self.records[channel]
            for channel in detected
            if not self.records[channel].cleared
        ]
        while pending:
            record, entry = self._choose_next_service(current_position, pending)
            current_position = self._localize_and_clear(robot, record, entry)
            pending.remove(record)
        exit_response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return PlanningReport(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            unknown_channels_after_cover=tuple(sorted(unknown)),
            exit_response=exit_response,
        )
    @staticmethod
    def _world_to_local(first: BearingObservation, point: Point) -> tuple[float, float]:
        theta = radians(first.bearing_deg)
        dx = point.x - first.station.x
        dy = point.y - first.station.y
        return (
            dx * cos(theta) + dy * sin(theta),
            -dx * sin(theta) + dy * cos(theta),
        )
    def _best_remaining_route_measurement(
        self,
        record: ChannelRecord,
        *,
        start_segment: int,
    ) -> OpportunisticMeasurement | None:
        """在剩余巡视折线上寻找不增加路程的鲁棒第二测点。"""
        first = record.observations[0]
        best: OpportunisticMeasurement | None = None
        shortlist: list[tuple[float, int, float, Point, float, float]] = []
        denominator = self.opportunistic_samples_per_segment - 1
        receive_limit = 1000.0 - self.reception_margin_m
        source_corners = tuple(
            source_point(scenario)
            for scenario in self._corner_scenarios[::2]
        )
        for segment_index in range(start_segment, len(self.cover_points) - 1):
            start = self.cover_points[segment_index]
            end = self.cover_points[segment_index + 1]
            for sample in range(self.opportunistic_samples_per_segment):
                fraction = sample / denominator
                point = Point(
                    start.x + fraction * (end.x - start.x),
                    start.y + fraction * (end.y - start.y),
                )
                forward, lateral = self._world_to_local(first, point)
                if abs(forward) + abs(lateral) <= 1e-6:
                    continue
                if (
                    worst_distance_to_initial_sector(
                        forward,
                        lateral,
                        bearing_error_deg=self.bearing_error_deg,
                    )
                    > receive_limit + 1e-8
                ):
                    continue
                minimum_angle = min(
                    crossing_angle_deg(forward, lateral, source)
                    for source in source_corners
                )
                if minimum_angle < self.opportunistic_min_crossing_angle_deg:
                    continue
                shortlist.append(
                    (
                        -minimum_angle,
                        segment_index,
                        fraction,
                        point,
                        forward,
                        lateral,
                    )
                )
        # 交会角只负责廉价预筛；最终选择仍以问题二的最坏包围圆为准。
        shortlist.sort(key=lambda item: item[:3])
        for _, segment_index, fraction, point, forward, lateral in shortlist[:12]:
            worst_circle = max(
                localization_circle_radius_for_scenario(
                    forward, lateral, scenario
                )
                for scenario in self._corner_scenarios
            )
            if worst_circle > self.opportunistic_circle_limit_m:
                continue
            candidate = OpportunisticMeasurement(
                segment_index,
                fraction,
                point,
                worst_circle,
            )
            if best is None or (
                candidate.worst_circle_radius_m,
                candidate.segment_index,
                candidate.fraction,
            ) < (
```

```
                best.worst_circle_radius_m,
                best.segment_index,
                best.fraction,
            ):
                best = candidate
        return best
    def _service_variants(self, record: ChannelRecord) ->
      tuple[ServiceVariant, ...]:
        if len(record.observations) >= 2:
            result = build_localization_polygon(
                [(item.station, item.bearing_deg) for item
      in record.observations],
                error_deg=self.bearing_error_deg,
            )
            if result.bounded and result.minimum_circle is
      not None:
                center = result.minimum_circle.center
                return (ServiceVariant(center, center),)
        first = record.observations[0]
        predicted_exit = local_to_world(
            first.station,
            first.bearing_deg,
            self.predicted_source_range_m,
            0.0,
        )
        return tuple(
            ServiceVariant(
                local_to_world(
                    first.station,
                    first.bearing_deg,
                    PRACTICAL_FORWARD_M,
                    side * PRACTICAL_LATERAL_M,
                ),
                predicted_exit,
            )
            for side in (-1, 1)
        )
    def _best_variants_for_order(
        self,
        start: Point,
        order: tuple[ChannelRecord, ...],
        variant_map: dict[int, tuple[ServiceVariant, ...]] |
       None = None,
    ) -> tuple[float, tuple[ServiceVariant, ...]]:
        """给定频道次序后，用动态规划精确选择左右测点。"""
        if not order:
            return 0.0, ()
        if variant_map is None:
            variant_map = {
                record.channel: self._service_variants(
      record) for record in order
            }
        variants = [variant_map[record.channel] for record
      in order]
        costs = [
            start.distance_to(item.entry) + item.entry.
      distance_to(item.predicted_exit)
            for item in variants[0]
        ]
        parents: list[list[int]] = [[-1] * len(variants[0])]
        for index in range(1, len(order)):
            row_costs: list[float] = []
            row_parents: list[int] = []
            for item in variants[index]:
                transition = [
                    costs[previous_index]
                    + previous.predicted_exit.distance_to(
      item.entry)
                    + item.entry.distance_to(item.
      predicted_exit)
                    for previous_index, previous in
      enumerate(variants[index - 1])
                ]
                best_previous = min(range(len(transition)),
      key=transition.__getitem__)
                row_costs.append(transition[best_previous])
                row_parents.append(best_previous)
            costs = row_costs
            parents.append(row_parents)
        selected_index = min(range(len(costs)), key=costs.
      __getitem__)
        selected: list[ServiceVariant] = [variants[-1][
      selected_index]]
        for index in range(len(order) - 1, 0, -1):
            selected_index = parents[index][selected_index]
            selected.append(variants[index - 1][
      selected_index])
        selected.reverse()
        return min(costs), tuple(selected)
    def _optimized_service_route(
        self,
        start: Point,
        pending: list[ChannelRecord],
    ) -> tuple[tuple[ChannelRecord, ...], tuple[
      ServiceVariant, ...]]:
        """最近插入思想构造初解，再用2-opt优化频道次序。"""
        if self.route_search_mode == "exact":
            return self._exact_service_route(start, pending)
        remaining = set(record.channel for record in pending
      )
        by_channel = {record.channel: record for record in
      pending}
        variant_map = {
            record.channel: self._service_variants(record)
      for record in pending
        }
        order: list[ChannelRecord] = []
        current = start
        while remaining:
            channel, variant = min(
                (
                    (channel, variant)
                    for channel in remaining
                    for variant in variant_map[channel]
                ),
                key=lambda item: current.distance_to(item
      [1].entry)
                + item[1].entry.distance_to(item[1].
      predicted_exit),
            )
            order.append(by_channel[channel])
            remaining.remove(channel)
            current = variant.predicted_exit
        best_order = tuple(order)
        best_cost, best_variants = self.
      _best_variants_for_order(
            start, best_order, variant_map
        )
        if not self.enable_route_local_search:
            return best_order, best_variants
        improved = True
        while improved:
            improved = False
            candidate_order = best_order
            candidate_cost = best_cost
            candidate_variants = best_variants
            for left in range(len(best_order) - 1):
                for right in range(left + 1, len(best_order)
      ):
                    trial_order = (
                        best_order[:left]
                        + tuple(reversed(best_order[left :
      right + 1]))
                        + best_order[right + 1 :]
                    )
                    trial_cost, trial_variants = self.
      _best_variants_for_order(
                        start, trial_order, variant_map
                    )
                    if trial_cost < candidate_cost - 1e-7:
                        candidate_order = trial_order
                        candidate_cost = trial_cost
                        candidate_variants = trial_variants
            # Or-opt(1)：把一个频道移到任意新位置，可修复2-opt
      难以处理的单点错位。
            for source_index in range(len(best_order)):
                shortened = list(best_order)
                moved = shortened.pop(source_index)
                for target_index in range(len(best_order)):
                    trial_list = shortened.copy()
                    trial_list.insert(target_index, moved)
                    trial_order = tuple(trial_list)
                    if trial_order == best_order:
                        continue
                    trial_cost, trial_variants = self.
      _best_variants_for_order(
                        start, trial_order, variant_map
                    )
                    if trial_cost < candidate_cost - 1e-7:
                        candidate_order = trial_order
                        candidate_cost = trial_cost
                        candidate_variants = trial_variants
            if candidate_cost < best_cost - 1e-7:
                best_order = candidate_order
                best_cost = candidate_cost
                best_variants = candidate_variants
                improved = True
        return best_order, best_variants
    def _exact_service_route(
        self,
        start: Point,
        pending: list[ChannelRecord],
    ) -> tuple[tuple[ChannelRecord, ...], tuple[
      ServiceVariant, ...]]:
        """用Held-Karp动态规划精确求解当前服务代价模型下的开放
      GTSP。"""
        if not pending:
            return (), ()
        records = tuple(pending)
        variants = tuple(self._service_variants(record) for
      record in records)
        count = len(records)
        full_mask = (1 << count) - 1
        layers: list[dict[tuple[int, int], float]] = [
            {} for _ in range(full_mask + 1)
        ]
        for record_index, choices in enumerate(variants):
            mask = 1 << record_index
            for variant_index, variant in enumerate(choices)
      :
                layers[mask][record_index, variant_index] =
      (
                    start.distance_to(variant.entry)
                    + variant.entry.distance_to(variant.
      predicted_exit)
                )
        for mask in range(1, full_mask):
            if not layers[mask]:
                continue
            remaining_mask = full_mask ^ mask
            for (last_index, last_variant_index), cost in
      layers[mask].items():
                previous = variants[last_index][
      last_variant_index]
```

```
                remaining = remaining_mask
                while remaining:
                    bit = remaining & -remaining
                    next_index = bit.bit_length() - 1
                    next_mask = mask | bit
                    for next_variant_index, candidate in enumerate(
                        variants[next_index]
                    ):
                        candidate_cost = (
                            cost
                            + previous.predicted_exit.distance_to(candidate.entry)
                            + candidate.entry.distance_to(candidate.predicted_exit)
                        )
                        key = (next_index, next_variant_index)
                        if candidate_cost < layers[next_mask].get(key, float("inf")):
                            layers[next_mask][key] = candidate_cost
                    remaining -= bit
            last_state = min(layers[full_mask], key=layers[full_mask].__getitem__)
            mask = full_mask
            selected_records: list[ChannelRecord] = []
            selected_variants: list[ServiceVariant] = []
            while mask:
                record_index, variant_index = last_state
                selected_records.append(records[record_index])
                selected_variants.append(variants[record_index][variant_index])
                previous_mask = mask ^ (1 << record_index)
                if previous_mask == 0:
                    break
                current = variants[record_index][variant_index]
                service_cost = current.entry.distance_to(current.predicted_exit)
                target_cost = layers[mask][last_state] - service_cost
                last_state = min(
                    layers[previous_mask],
                    key=lambda previous_state: abs(
                        layers[previous_mask][previous_state]
                        + variants[previous_state[0]][previous_state[1]].predicted_exit.distance_to(
                            current.entry
                        )
                        - target_cost
                    ),
                )
                mask = previous_mask
            selected_records.reverse()
            selected_variants.reverse()
            return tuple(selected_records), tuple(selected_variants)
        def _choose_next_service(
            self,
            current: Point,
            pending: list[ChannelRecord],
        ) -> tuple[ChannelRecord, Point]:
            order, variants = self._optimized_service_route(current, pending)
            return order[0], variants[0].entry
        def _localize_and_clear(
            self,
            robot: RobotInterface,
            record: ChannelRecord,
            entry: Point,
        ) -> Point:
            if len(record.observations) < 2:
                response = robot.measure(entry.x, entry.y, record.channel)
                if response["measure_result"] == "near":
                    if self._attempt_clear(robot, entry, record):
                        return entry
                elif response["measure_result"] == "direction":
                    record.observations.append(
                        BearingObservation(entry, float(response["svd_deg"]))
                    )
                else:
                    return self._clear_initial_sector(robot, record)
            for _ in range(self.max_refinement_measurements):
                result = build_localization_polygon(
                    [(item.station, item.bearing_deg) for item in record.observations],
                    error_deg=self.bearing_error_deg,
                )
                if not result.polygon or not result.bounded or result.minimum_circle is None:
                    return self._clear_initial_sector(robot, record)
                probe = result.minimum_circle.center
                if result.minimum_circle.radius <= 20.0 - 1e-6:
                    if self._attempt_clear(robot, probe, record):
                        return probe
                return self._clear_polygon(robot, result.polygon, record)
                response = robot.measure(probe.x, probe.y, record.channel)
                if response["measure_result"] == "near":
                    if self._attempt_clear(robot, probe, record):
                        return probe
                elif response["measure_result"] == "direction":
                    record.observations.append(
                        BearingObservation(probe, float(response["svd_deg"]))
                    )
                else:
                    return self._clear_polygon(robot, result.polygon, record)
                result = build_localization_polygon(
                    [(item.station, item.bearing_deg) for item in record.observations],
                    error_deg=self.bearing_error_deg,
                )
                return self._clear_polygon(robot, result.polygon, record)
            return self._clear_initial_sector(robot, record)
        @staticmethod
        def _attempt_clear(
            robot: RobotInterface,
            point: Point,
            record: ChannelRecord,
        ) -> bool:
            response = robot.clear(point.x, point.y, record.channel)
            if response["clear_result"] == "success":
                record.cleared = True
                return True
            return False
        def _clear_initial_sector(
            self,
            robot: RobotInterface,
            record: ChannelRecord,
        ) -> Point:
            """第二次测量异常时，对首次有限误差扇形执行保证覆盖。"""
            first = record.observations[0]
            points = tuple(
                local_to_world(first.station, first.bearing_deg, rho * cos(phi), rho * sin(phi))
                for rho in (5.0, 1500.0)
                for phi in (
                    -radians(self.bearing_error_deg),
                    radians(self.bearing_error_deg),
                )
            )
            return self._clear_polygon(robot, points, record)
        @staticmethod
        def _clear_polygon(
            robot: RobotInterface,
            polygon: tuple[Point, ...],
            record: ChannelRecord,
        ) -> Point:
            """以间距不超过20sqrt2的蛇形网格覆盖包围盒，不抛出整局异常。"""
            if not polygon:
                return record.observations[-1].station
            step = 20.0 * sqrt(2.0)
            min_x = min(point.x for point in polygon)
            max_x = max(point.x for point in polygon)
            min_y = min(point.y for point in polygon)
            max_y = max(point.y for point in polygon)
            nx = max(1, ceil((max_x - min_x) / step))
            ny = max(1, ceil((max_y - min_y) / step))
            last = Point(min_x, min_y)
            for row in range(ny + 1):
                y = min_y + (max_y - min_y) * row / ny
                columns = range(nx + 1) if row % 2 == 0 else range(nx, -1, -1)
                for column in columns:
                    x = min_x + (max_x - min_x) * column / nx
                    last = Point(x, y)
                    if Problem3Planner._attempt_clear(robot, last, record):
                        return last
            return last
    class JointProblem3Planner(Problem3Planner):
        """将剩余覆盖点与已发现干扰源放入同一滚动路线的实验规划器。"""
        def __init__(
            self,
            *,
            max_services_between_cover_points: int | None = None,
            **kwargs: object,
        ) -> None:
            super().__init__(**kwargs)
            if (
                max_services_between_cover_points is not None
                and max_services_between_cover_points < 1
            ):
                raise ValueError("max_services_between_cover_points必须为正整数或None")
            self.max_services_between_cover_points = max_services_between_cover_points
        def run(self, robot: RobotInterface) -> PlanningReport:
            robot.enter()
            unknown = set(range(1, 21))
            detected: set[int] = set()
            remaining_cover = set(range(len(self.cover_points)))
            current_position = Point(0.0, 0.0)
            current_channel = 1
            services_since_cover = 0
            while remaining_cover or any(
                record.cleared is False and record.observations
                for record in self.records.values()
            ):
                if len(detected) == 16:
```

```
                remaining_cover.clear()
            if 0 in remaining_cover and current_position.distance_to(
                self.cover_points[0]
            ) <= 1e-8:
                current_channel = self._scan_cover_point(
                    robot,
                    self.cover_points[0],
                    unknown,
                    detected,
                    current_channel,
                )
                remaining_cover.remove(0)
                continue
            pending = [
                self.records[channel]
                for channel in sorted(detected)
                if not self.records[channel].cleared
            ]
            if not remaining_cover and not pending:
                break
            force_cover = (
                bool(remaining_cover)
                and self.max_services_between_cover_points is not None
                and services_since_cover >= self.max_services_between_cover_points
            )
            task, variant = self._choose_joint_task(
                current_position,
                remaining_cover,
                [] if force_cover else pending,
            )
            opportunity = self._first_segment_opportunity(
                current_position,
                variant.entry,
                pending,
            )
            if opportunity is not None:
                record, point = opportunity
                response = robot.measure(point.x, point.y, record.channel)
                current_position = point
                current_channel = record.channel
                if response["measure_result"] == "near":
                    self._attempt_clear(robot, point, record)
                elif response["measure_result"] == "direction":
                    record.observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                else:
                    self._record_opportunity_no_signal(record, point)
                continue
            task_kind, task_value = task
            if task_kind == "cover":
                cover_index = task_value
                point = self.cover_points[cover_index]
                current_channel = self._scan_cover_point(
                    robot,
                    point,
                    unknown,
                    detected,
                    current_channel,
                )
                current_position = point
                remaining_cover.remove(cover_index)
                services_since_cover = 0
            else:
                record = self.records[task_value]
                self._service_start_position = current_position
                current_position = self._localize_and_clear(
                    robot,
                    record,
                    variant.entry,
                )
                services_since_cover += 1
        exit_response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return PlanningReport(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            unknown_channels_after_cover=tuple(sorted(unknown)),
            exit_response=exit_response,
        )
    def _scan_cover_point(
        self,
        robot: RobotInterface,
        point: Point,
        unknown: set[int],
        detected: set[int],
        current_channel: int,
    ) -> int:
        ordered_channels = sorted(
            unknown,
            key=lambda channel: channel != current_channel,
        )
        for channel in ordered_channels:
            response = robot.measure(point.x, point.y, channel)
            current_channel = channel
            result = response["measure_result"]
            if result == "near":
                if self._attempt_clear(robot, point, self.records[channel]):
                    detected.add(channel)
                    unknown.remove(channel)
            elif result == "direction":
                self.records[channel].observations.append(
                    BearingObservation(point, float(response["svd_deg"]))
                )
                detected.add(channel)
                unknown.remove(channel)
            else:
                self.records[channel].no_signal_stations.append(point)
        return current_channel
    @staticmethod
    def _record_opportunity_no_signal(
        record: ChannelRecord,
        point: Point,
    ) -> None:
        record.no_signal_stations.append(point)
    def _segment_opportunity(
        self,
        record: ChannelRecord,
        start: Point,
        end: Point,
    ) -> Point | None:
        if len(record.observations) != 1 or start.distance_to(end) <= 1e-8:
            return None
        first = record.observations[0]
        receive_limit = 1000.0 - self.reception_margin_m
        source_corners = tuple(source_point(item) for item in self._corner_scenarios[::2])
        denominator = self.opportunistic_samples_per_segment - 1
        candidates: list[tuple[float, float, Point, float, float]] = []
        for sample in range(1, self.opportunistic_samples_per_segment + 1):
            fraction = sample / self.opportunistic_samples_per_segment
            point = Point(
                start.x + fraction * (end.x - start.x),
                start.y + fraction * (end.y - start.y),
            )
            forward, lateral = self._world_to_local(first, point)
            if (
                worst_distance_to_initial_sector(
                    forward,
                    lateral,
                    bearing_error_deg=self.bearing_error_deg,
                )
                > receive_limit + 1e-8
            ):
                continue
            minimum_angle = min(
                crossing_angle_deg(forward, lateral, source)
                for source in source_corners
            )
            if minimum_angle < self.opportunistic_min_crossing_angle_deg:
                continue
            candidates.append((-minimum_angle, fraction, point, forward, lateral))
        candidates.sort(key=lambda item: (item[0], item[1]))
        feasible: list[tuple[float, float, Point]] = []
        for _, fraction, point, forward, lateral in candidates[:12]:
            worst_circle = max(
                localization_circle_radius_for_scenario(
                    forward,
                    lateral,
                    scenario,
                )
                for scenario in self._corner_scenarios
            )
            if worst_circle <= self.opportunistic_circle_limit_m:
                feasible.append((fraction, worst_circle, point))
        if not feasible:
            return None
        return min(feasible, key=lambda item: (item[0], item[1]))[2]
    def _first_segment_opportunity(
        self,
        start: Point,
        end: Point,
        pending: list[ChannelRecord],
    ) -> tuple[ChannelRecord, Point] | None:
        choices: list[tuple[float, ChannelRecord, Point]] = []
        length = start.distance_to(end)
        if length <= 1e-8:
            return None
        for record in pending:
            point = self._segment_opportunity(record, start, end)
            if point is not None:
                choices.append((start.distance_to(point) / length, record, point))
        if not choices:
```

```
            return None
        _, record, point = min(
            choices,
            key=lambda item: (item[0], item[1].channel),
        )
        return record, point
    @staticmethod
    def _best_joint_variants_for_order(
        start: Point,
        order: tuple[tuple[str, int], ...],
        variant_map: dict[tuple[str, int], tuple[ServiceVariant, ...]],
    ) -> tuple[float, tuple[ServiceVariant, ...]]:
        if not order:
            return 0.0, ()
        variants = [variant_map[task] for task in order]
        costs = [
            start.distance_to(item.entry) + item.entry.distance_to(item.predicted_exit)
            for item in variants[0]
        ]
        parents: list[list[int]] = [[-1] * len(variants[0])]
        for index in range(1, len(order)):
            row_costs: list[float] = []
            row_parents: list[int] = []
            for item in variants[index]:
                transition = [
                    costs[previous_index]
                    + previous.predicted_exit.distance_to(item.entry)
                    + item.entry.distance_to(item.predicted_exit)
                    for previous_index, previous in enumerate(variants[index - 1])
                ]
                best_previous = min(range(len(transition)), key=transition.__getitem__)
                row_costs.append(transition[best_previous])
                row_parents.append(best_previous)
            costs = row_costs
            parents.append(row_parents)
        selected_index = min(range(len(costs)), key=costs.__getitem__)
        selected: list[ServiceVariant] = [variants[-1][selected_index]]
        for index in range(len(order) - 1, 0, -1):
            selected_index = parents[index][selected_index]
            selected.append(variants[index - 1][selected_index])
        selected.reverse()
        return min(costs), tuple(selected)
    def _choose_joint_task(
        self,
        start: Point,
        remaining_cover: set[int],
        pending: list[ChannelRecord],
    ) -> tuple[tuple[str, int], ServiceVariant]:
        variant_map: dict[tuple[str, int], tuple[ServiceVariant, ...]] = {}
        for cover_index in sorted(remaining_cover):
            point = self.cover_points[cover_index]
            variant_map["cover", cover_index] = (ServiceVariant(point, point),)
        for record in pending:
            variant_map["source", record.channel] = self._service_variants(record)
        remaining = set(variant_map)
        order: list[tuple[str, int]] = []
        current = start
        while remaining:
            task, variant = min(
                (
                    (task, variant)
                    for task in remaining
                    for variant in variant_map[task]
                ),
                key=lambda item: (
                    current.distance_to(item[1].entry)
                    + item[1].entry.distance_to(item[1].predicted_exit),
                    item[0],
                ),
            )
            order.append(task)
            remaining.remove(task)
            current = variant.predicted_exit
        best_order = tuple(order)
        best_cost, best_variants = self._best_joint_variants_for_order(
            start,
            best_order,
            variant_map,
        )
        improved = True
        while improved:
            improved = False
            candidate_order = best_order
            candidate_cost = best_cost
            candidate_variants = best_variants
            for left in range(len(best_order) - 1):
                for right in range(left + 1, len(best_order)):
                    trial_order = (
                        best_order[:left]
                        + tuple(reversed(best_order[left : right + 1]))
                        + best_order[right + 1 :]
                    )
                    trial_cost, trial_variants = self._best_joint_variants_for_order(
                        start,
                        trial_order,
                        variant_map,
                    )
                    if trial_cost < candidate_cost - 1e-7:
                        candidate_order = trial_order
                        candidate_cost = trial_cost
                        candidate_variants = trial_variants
            for source_index in range(len(best_order)):
                shortened = list(best_order)
                moved = shortened.pop(source_index)
                for target_index in range(len(best_order)):
                    trial_list = shortened.copy()
                    trial_list.insert(target_index, moved)
                    trial_order = tuple(trial_list)
                    if trial_order == best_order:
                        continue
                    trial_cost, trial_variants = self._best_joint_variants_for_order(
                        start,
                        trial_order,
                        variant_map,
                    )
                    if trial_cost < candidate_cost - 1e-7:
                        candidate_order = trial_order
                        candidate_cost = trial_cost
                        candidate_variants = trial_variants
            if candidate_cost < best_cost - 1e-7:
                best_order = candidate_order
                best_cost = candidate_cost
                best_variants = candidate_variants
                improved = True
        return best_order[0], best_variants[0]
class JointHomingProblem3Planner(JointProblem3Planner):
    """联合路线加最小包围圆归航的实验规划器。"""
    def __init__(
        self,
        *,
        homing_prediction_scale: float | None = 1.1,
        homing_probe_bias: float = 0.0,
        use_signal_history_prediction: bool = False,
        speculative_clear_radius_m: float = 0.0,
        speculative_clear_at_centroid: bool = False,
        aggressive_opportunity_distance_m: float | None = None,
        aggressive_opportunity_min_angle_deg: float = 10.0,
        max_aggressive_opportunity_failures: int = 1,
        adaptive_first_probe: bool = False,
        adaptive_probe_circle_limit_m: float = 120.0,
        cover_point_design: str = "seven_guaranteed",
        six_ring_radius_m: float = 1039.0,
        flexible_homing_variants: bool = False,
        flexible_probe_radius_m: float = 750.0,
        aggressive_first_probe_scale: float | None = None,
        custom_probe_max_observations: int = 1,
        **kwargs: object,
    ) -> None:
        kwargs.setdefault("max_refinement_measurements", 8)
        super().__init__(**kwargs)
        if cover_point_design not in {
            "seven_guaranteed",
            "six_ring",
            "center_five",
        }:
            raise ValueError("未知覆盖点设计")
        if homing_prediction_scale is not None and not (
            0.0 <= homing_prediction_scale <= 1.5
        ):
            raise ValueError("homing_prediction_scale必须位于0至1.5")
        if not -2.0 <= homing_probe_bias <= 1.0:
            raise ValueError("homing_probe_bias必须位于-2至1")
        if speculative_clear_radius_m < 0.0:
            raise ValueError("speculative_clear_radius_m不能为负数")
        if (
            aggressive_opportunity_distance_m is not None
            and aggressive_opportunity_distance_m <= 0.0
        ):
            raise ValueError("aggressive_opportunity_distance_m必须为正数或None")
        if max_aggressive_opportunity_failures < 1:
            raise ValueError("max_aggressive_opportunity_failures必须为正整数")
        self.homing_prediction_scale = homing_prediction_scale
        self.homing_probe_bias = homing_probe_bias
        self.use_signal_history_prediction = use_signal_history_prediction
        self.speculative_clear_radius_m = speculative_clear_radius_m
        self.speculative_clear_at_centroid = speculative_clear_at_centroid
        self.aggressive_opportunity_distance_m = aggressive_opportunity_distance_m
        self.aggressive_opportunity_min_angle_deg = (
            aggressive_opportunity_min_angle_deg
        )
        self.max_aggressive_opportunity_failures = (
            max_aggressive_opportunity_failures
        )
        self.adaptive_first_probe = adaptive_first_probe
        self.adaptive_probe_circle_limit_m = adaptive_probe_circle_limit_m
```

```
        self.cover_point_design = cover_point_design
        self.flexible_homing_variants = flexible_homing_variants
        self.flexible_probe_radius_m = flexible_probe_radius_m
        self.aggressive_first_probe_scale = aggressive_first_probe_scale
        self.custom_probe_max_observations = int(custom_probe_max_observations)
        if cover_point_design == "six_ring":
            self.cover_points = tuple(
                Point(
                    six_ring_radius_m * cos(index * pi / 3.0),
                    six_ring_radius_m * sin(index * pi / 3.0),
                )
                for index in range(6)
            )
        elif cover_point_design == "center_five":
            five_ring_radius_m = 1456.0
            self.cover_points = (Point(0.0, 0.0),) + tuple(
                Point(
                    five_ring_radius_m * cos(index * 2.0 * pi / 5.0),
                    five_ring_radius_m * sin(index * 2.0 * pi / 5.0),
                )
                for index in range(5)
            )
        self._failed_aggressive_opportunity: dict[int, int] = {}
        self._prediction_cache: dict[tuple[int, int, int], Point] = {}
    def _record_opportunity_no_signal(
        self,
        record: ChannelRecord,
        point: Point,
    ) -> None:
        super()._record_opportunity_no_signal(record, point)
        self._failed_aggressive_opportunity[record.channel] = (
            self._failed_aggressive_opportunity.get(record.channel, 0) + 1
        )
    def _initial_sector_polygon(self, record: ChannelRecord) -> tuple[Point, ...]:
        first = record.observations[0]
        theta = radians(first.bearing_deg)
        delta = radians(self.bearing_error_deg)
        reach = 1500.0 / cos(delta)
        polygon = [
            first.station,
            Point(
                first.station.x + reach * cos(theta - delta),
                first.station.y + reach * sin(theta - delta),
            ),
            Point(
                first.station.x + reach * cos(theta + delta),
                first.station.y + reach * sin(theta + delta),
            ),
        ]
        # 用目标圆的一组外切半平面构成外包多边形。真实目标圆严格位于
        # 这些半平面内，因此裁剪只会缩小虚假的远距离扇形，不会排除真源。
        for angle_deg in range(0, 360, 5):
            angle = radians(float(angle_deg))
            polygon = clip_polygon_with_halfplane(
                polygon,
                HalfPlane(cos(angle), sin(angle), 1800.0, "target-disk"),
            )
            if not polygon:
                return ()
        return tuple(polygon)
    def _bounded_feasible_polygon(
        self,
        record: ChannelRecord,
    ) -> tuple[Point, ...]:
        polygon = list(self._initial_sector_polygon(record))
        for observation_index, observation in enumerate(record.observations[1:], 2):
            for halfplane in bearing_halfplanes(
                observation.station,
                observation.bearing_deg,
                self.bearing_error_deg,
                observation_index,
            ):
                polygon = clip_polygon_with_halfplane(polygon, halfplane)
                if not polygon:
                    return ()
        return tuple(polygon)
    def _service_variants(self, record: ChannelRecord) -> tuple[ServiceVariant, ...]:
        polygon = self._bounded_feasible_polygon(record)
        if polygon:
            entry = self._homing_probe(polygon)
        else:
            first = record.observations[0]
            entry = local_to_world(
                first.station,
                first.bearing_deg,
                750.0,
                0.0,
            )
        first = record.observations[0]
        if polygon and self.homing_prediction_scale is not None:
            centroid = (
                self._signal_history_prediction(record)
                if self.use_signal_history_prediction
                else self._polygon_centroid(polygon)
            )
            predicted_exit = Point(
                first.station.x
                + self.homing_prediction_scale * (centroid.x - first.station.x),
                first.station.y
                + self.homing_prediction_scale * (centroid.y - first.station.y),
            )
        else:
            predicted_exit = local_to_world(
                first.station,
                first.bearing_deg,
                self.predicted_source_range_m,
                0.0,
            )
        variants = [ServiceVariant(entry, predicted_exit)]
        if self.flexible_homing_variants and len(record.observations) == 1 and polygon:
            predicted_source = self._polygon_centroid(polygon)
            first = record.observations[0]
            for angle_index in range(12):
                angle = angle_index * pi / 6.0
                candidate = Point(
                    predicted_source.x + self.flexible_probe_radius_m * cos(angle),
                    predicted_source.y + self.flexible_probe_radius_m * sin(angle),
                )
                crossing = self._crossing_angle_for_source(
                    first.station,
                    candidate,
                    predicted_source,
                )
                if crossing < self.aggressive_opportunity_min_angle_deg:
                    continue
                hypothetical_bearing = degrees(
                    atan2(
                        predicted_source.y - candidate.y,
                        predicted_source.x - candidate.x,
                    )
                ) % 360.0
                refined = list(polygon)
                for halfplane in bearing_halfplanes(
                    candidate,
                    hypothetical_bearing,
                    self.bearing_error_deg,
                    2,
                ):
                    refined = clip_polygon_with_halfplane(refined, halfplane)
                    if not refined:
                        break
                if not refined:
                    continue
                if (
                    minimum_enclosing_circle(refined).radius
                    > self.adaptive_probe_circle_limit_m
                ):
                    continue
                variants.append(ServiceVariant(candidate, predicted_exit))
        return tuple(variants)
    def _signal_history_prediction(self, record: ChannelRecord) -> Point:
        key = (
            record.channel,
            len(record.observations),
            len(record.no_signal_stations),
        )
        cached = self._prediction_cache.get(key)
        if cached is not None:
            return cached
        first = record.observations[0]
        angular_samples = 31
        radial_samples = 64
        weighted_x = 0.0
        weighted_y = 0.0
        total_weight = 0.0
        for angle_index in range(angular_samples):
            error = -self.bearing_error_deg + (
                2.0 * self.bearing_error_deg * (angle_index + 0.5)
                / angular_samples
            )
            angle = radians(first.bearing_deg + error)
            for radial_index in range(radial_samples):
                # 按rho平方等距，使每个样本代表近似相同的平面面积。
                rho = 1500.0 * sqrt((radial_index + 0.5) / radial_samples)
                point = Point(
                    first.station.x + rho * cos(angle),
                    first.station.y + rho * sin(angle),
                )
```

```
                if point.x * point.x + point.y * point.y > 1800.0**2:
                    continue
                consistent = True
                positive_distances: list[float] = []
                for observation in record.observations:
                    dx = point.x - observation.station.x
                    dy = point.y - observation.station.y
                    distance = sqrt(dx * dx + dy * dy)
                    if distance <= 5.0:
                        consistent = False
                        break
                    bearing = degrees(atan2(dy, dx)) % 360.0
                    difference = (
                        bearing - observation.bearing_deg + 180.0
                    ) % 360.0 - 180.0
                    if abs(difference) > self.bearing_error_deg + 1e-8:
                        consistent = False
                        break
                    positive_distances.append(distance)
                if not consistent:
                    continue
                lower_radius = max(1000.0, *positive_distances)
                upper_radius = 1500.0
                for station in record.no_signal_stations:
                    upper_radius = min(upper_radius, point.distance_to(station))
                radius_weight = max(0.0, upper_radius - lower_radius)
                if radius_weight <= 0.0:
                    continue
                weighted_x += radius_weight * point.x
                weighted_y += radius_weight * point.y
                total_weight += radius_weight
            if total_weight <= 1e-12:
                prediction = self._polygon_centroid(
                    self._bounded_feasible_polygon(record)
                )
            else:
                prediction = Point(weighted_x / total_weight, weighted_y / total_weight)
            self._prediction_cache[key] = prediction
            return prediction
        def _homing_probe(self, polygon: tuple[Point, ...]) -> Point:
            center = minimum_enclosing_circle(polygon).center
            if abs(self.homing_probe_bias) <= 1e-12:
                return center
            centroid = self._polygon_centroid(polygon)
            candidate = Point(
                center.x + self.homing_probe_bias * (centroid.x - center.x),
                center.y + self.homing_probe_bias * (centroid.y - center.y),
            )
            receive_limit = 1000.0 - self.reception_margin_m
            if max(candidate.distance_to(point) for point in polygon) <= receive_limit:
                return candidate
            low = 0.0
            high = 1.0
            for _ in range(40):
                middle = (low + high) / 2.0
                factor = middle * self.homing_probe_bias
                probe = Point(
                    center.x + factor * (centroid.x - center.x),
                    center.y + factor * (centroid.y - center.y),
                )
                if max(probe.distance_to(point) for point in polygon) <= receive_limit:
                    low = middle
                else:
                    high = middle
            factor = low * self.homing_probe_bias
            return Point(
                center.x + factor * (centroid.x - center.x),
                center.y + factor * (centroid.y - center.y),
            )
        def _custom_first_probe(
            self,
            record: ChannelRecord,
            polygon: tuple[Point, ...],
            start: Point,
        ) -> Point | None:
            return None
        def _adaptive_probe(
            self,
            record: ChannelRecord,
            polygon: tuple[Point, ...],
            start: Point,
        ) -> Point:
            """在保证接收的区域内联合优化第二测点和预计归航路程。
            """
            safe_center = minimum_enclosing_circle(polygon).center
            predicted_source = self._polygon_centroid(polygon)
            candidates: list[Point] = [safe_center]
            for target, samples in ((start, 61), (predicted_source, 31)):
                for index in range(samples + 1):
                    fraction = index / samples
                    candidates.append(
                        Point(
                            safe_center.x + fraction * (target.x - safe_center.x),
                            safe_center.y + fraction * (target.y - safe_center.y),
                        )
                    )
            receive_limit = 1000.0 - self.reception_margin_m
            first = record.observations[0]
            feasible: list[tuple[float, Point]] = []
            for point in candidates:
                if max(point.distance_to(source) for source in polygon) > receive_limit:
                    continue
                crossing_angle = self._crossing_angle_for_source(
                    first.station,
                    point,
                    predicted_source,
                )
                if crossing_angle < self.aggressive_opportunity_min_angle_deg:
                    continue
                bearing = degrees(
                    atan2(
                        predicted_source.y - point.y,
                        predicted_source.x - point.x,
                    )
                ) % 360.0
                refined = list(polygon)
                for halfplane in bearing_halfplanes(
                    point,
                    bearing,
                    self.bearing_error_deg,
                    len(record.observations) + 1,
                ):
                    refined = clip_polygon_with_halfplane(refined, halfplane)
                    if not refined:
                        break
                if not refined:
                    continue
                refined_circle = minimum_enclosing_circle(refined)
                if refined_circle.radius > self.adaptive_probe_circle_limit_m:
                    continue
                score = (
                    start.distance_to(point)
                    + point.distance_to(predicted_source)
                    + 0.25 * refined_circle.radius
                )
                feasible.append((score, point))
            if not feasible:
                return safe_center
            return min(feasible, key=lambda item: item[0])[1]
        @staticmethod
        def _polygon_centroid(polygon: tuple[Point, ...]) -> Point:
            if len(polygon) < 3:
                return minimum_enclosing_circle(polygon).center
            doubled_area = 0.0
            cx = 0.0
            cy = 0.0
            for index, point in enumerate(polygon):
                following = polygon[(index + 1) % len(polygon)]
                cross = point.x * following.y - following.x * point.y
                doubled_area += cross
                cx += (point.x + following.x) * cross
                cy += (point.y + following.y) * cross
            if abs(doubled_area) <= 1e-10:
                return minimum_enclosing_circle(polygon).center
            return Point(cx / (3.0 * doubled_area), cy / (3.0 * doubled_area))
        @staticmethod
        def _crossing_angle_for_source(
            first_station: Point,
            second_station: Point,
            source: Point,
        ) -> float:
            ax = source.x - first_station.x
            ay = source.y - first_station.y
            bx = source.x - second_station.x
            by = source.y - second_station.y
            na = sqrt(ax * ax + ay * ay)
            nb = sqrt(bx * bx + by * by)
            if na <= 1e-9 or nb <= 1e-9:
                return 90.0
            cosine = min(1.0, max(-1.0, abs((ax * bx + ay * by) / (na * nb))))
            return degrees(acos(cosine))
        def _segment_opportunity(
            self,
            record: ChannelRecord,
            start: Point,
            end: Point,
        ) -> Point | None:
            if len(record.observations) != 1 or start.distance_to(end) <= 1e-8:
                return None
            polygon = self._bounded_feasible_polygon(record)
            if not polygon:
                return None
            first = record.observations[0]
            candidates: list[tuple[float, float, Point]] = []
            for sample in range(1, self.opportunistic_samples_per_segment + 1):
                fraction = sample / self.opportunistic_samples_per_segment
                point = Point(
```

```
                start.x + fraction * (end.x - start.x),
                start.y + fraction * (end.y - start.y),
            )
            worst_distance = max(point.distance_to(source)
        for source in polygon)
            if worst_distance > 1000.0 - self.
        reception_margin_m + 1e-8:
                continue
            minimum_angle = min(
                self._crossing_angle_for_source(first.
        station, point, source)
                for source in polygon
            )
            if minimum_angle < self.
        opportunistic_min_crossing_angle_deg:
                continue
            candidates.append((fraction, -minimum_angle,
        point))
        if not candidates:
            safe_choice = None
        else:
            safe_choice = min(candidates, key=lambda item: (
        item[0], item[1]))[2]
        if safe_choice is not None:
            return safe_choice
        if (
            self.aggressive_opportunity_distance_m is None
            or self._failed_aggressive_opportunity.get(
        record.channel, 0)
            >= self.max_aggressive_opportunity_failures
        ):
            return None
        predicted_source = self._polygon_centroid(polygon)
        aggressive: list[tuple[float, float, Point]] = []
        for sample in range(1, self.
        opportunistic_samples_per_segment + 1):
            fraction = sample / self.
        opportunistic_samples_per_segment
            point = Point(
                start.x + fraction * (end.x - start.x),
                start.y + fraction * (end.y - start.y),
            )
            predicted_distance = point.distance_to(
        predicted_source)
            if predicted_distance > self.
        aggressive_opportunity_distance_m:
                continue
            angle = self._crossing_angle_for_source(
                first.station,
                point,
                predicted_source,
            )
            if angle < self.
        aggressive_opportunity_min_angle_deg:
                continue
            aggressive.append((fraction, -angle, point))
        if not aggressive:
            return None
        return min(aggressive, key=lambda item: (item[0],
        item[1]))[2]
    def _localize_and_clear(
        self,
        robot: RobotInterface,
        record: ChannelRecord,
        entry: Point,
    ) -> Point:
        aggressive_probe_failed = False
        for refinement_index in range(self.
        max_refinement_measurements):
            polygon = self._bounded_feasible_polygon(record)
            if not polygon:
                return self._clear_initial_sector(robot,
        record)
            circle = minimum_enclosing_circle(polygon)
            if circle.radius <= 20.0 - 1e-6:
                probe = circle.center
                if self._attempt_clear(robot, probe, record)
        :
                    return probe
                return self._clear_polygon(robot, polygon,
        record)
            if circle.radius <= self.
        speculative_clear_radius_m:
                probe = self._speculative_clear_probe(record
        , polygon, circle)
                if self._attempt_clear(robot, probe, record)
        :
                    return probe
            else:
                custom_probe = (
                    self._custom_first_probe(
                        record,
                        polygon,
                        self._service_start_position,
                    )
                    if (
                        not aggressive_probe_failed
                        and 1 <= len(record.observations)
                        <= max(1, self.
        custom_probe_max_observations)
                        and hasattr(self, "
        _service_start_position")
                    )
                    else None
                )
                if custom_probe is not None:
                    probe = custom_probe
                elif (
                    self.flexible_homing_variants
                    and refinement_index == 0
                    and len(record.observations) == 1
                ):
                    probe = entry
                elif (
                    self.aggressive_first_probe_scale is not
         None
                    and not aggressive_probe_failed
                    and len(record.observations) == 1
                ):
                    first = record.observations[0]
                    predicted = self._polygon_centroid(
        polygon)
                    probe = Point(
                        first.station.x
                        + self.aggressive_first_probe_scale
                        * (predicted.x - first.station.x),
                        first.station.y
                        + self.aggressive_first_probe_scale
                        * (predicted.y - first.station.y),
                    )
                elif (
                    self.adaptive_first_probe
                    and len(record.observations) == 1
                    and hasattr(self, "
        _service_start_position")
                ):
                    probe = self._adaptive_probe(
                        record,
                        polygon,
                        self._service_start_position,
                    )
                else:
                    probe = self._homing_probe(polygon)
            response = robot.measure(probe.x, probe.y,
        record.channel)
            if response["measure_result"] == "near":
                if self._attempt_clear(robot, probe, record)
        :
                    return probe
            elif response["measure_result"] == "direction":
                record.observations.append(
                    BearingObservation(probe, float(response
        ["svd_deg"]))
                )
            else:
                record.no_signal_stations.append(probe)
                receive_limit = 1000.0 - self.
        reception_margin_m
                if max(probe.distance_to(source) for source
        in polygon) > receive_limit:
                    aggressive_probe_failed = True
                    continue
                return self._clear_polygon(robot, polygon,
        record)
        polygon = self._bounded_feasible_polygon(record)
        return self._clear_polygon(robot, polygon, record)
    def _speculative_clear_probe(
        self,
        record: ChannelRecord,
        polygon: tuple[Point, ...],
        circle: object,
    ) -> Point:
        """Return the low-cost speculative clear point."""
        if self.speculative_clear_at_centroid:
            return self._polygon_centroid(polygon)
        return circle.center  # type: ignore[attr-defined]
@dataclass(frozen=True)
class SourceParticle:
    """与频道全部历史反馈一致的一种“位置—接收半径”假设。"""
    point: Point
    receive_radius_m: float
class ParticleRolloutProblem3Planner(
        JointHomingProblem3Planner):
    """用后验粒子评估下一动作的风险感知问题3候选规划器。"""
    def __init__(
        self,
        *,
        particle_count: int = 24,
        rollout_risk_weight: float = 0.15,
        rollout_candidate_limit: int = 5,
        rollout_depth: int = 10,
        rollout_min_relative_gain: float = 0.05,
        enable_particle_rollout: bool = True,
        particle_prediction_scale: float | None = None,
        **kwargs: object,
    ) -> None:
        if particle_count < 8:
            raise ValueError("particle_count至少为8")
        if rollout_risk_weight < 0.0:
            raise ValueError("rollout_risk_weight不能为负数"
        )
        if rollout_candidate_limit < 1 or rollout_depth < 1:
            raise ValueError("滚动候选数和深度必须为正整数")
        if not 0.0 <= rollout_min_relative_gain < 1.0:
            raise ValueError("粒子替换门槛必须位于0至1")
        kwargs.setdefault("homing_prediction_scale", 1.1)
        kwargs.setdefault("speculative_clear_radius_m",
        100.0)
        kwargs.setdefault("speculative_clear_at_centroid",
        True)
        kwargs.setdefault("aggressive_opportunity_distance_m
        ", 1000.0)
        kwargs.setdefault("
        aggressive_opportunity_min_angle_deg", 15.0)
        kwargs.setdefault("
```

```python
        max_aggressive_opportunity_failures", 1)
        kwargs.setdefault("cover_point_design", "
    seven_guaranteed")
        super().__init__(**kwargs)
        self.particle_count = particle_count
        self.rollout_risk_weight = rollout_risk_weight
        self.rollout_candidate_limit =
    rollout_candidate_limit
        self.rollout_depth = rollout_depth
        self.rollout_min_relative_gain =
    rollout_min_relative_gain
        self.enable_particle_rollout =
    enable_particle_rollout
        self.particle_prediction_scale =
    particle_prediction_scale
        self._particle_cache: dict[tuple[int, int, int],
    tuple[SourceParticle, ...]] = {}
    @staticmethod
    def _triangle_area(a: Point, b: Point, c: Point) ->
    float:
        return abs(
            (b.x - a.x) * (c.y - a.y)
            - (b.y - a.y) * (c.x - a.x)
        ) / 2.0
    def _draw_uniform_polygon_point(
        self,
        polygon: tuple[Point, ...],
        rng: Random,
    ) -> Point:
        if len(polygon) < 3:
            return minimum_enclosing_circle(polygon).center
        origin = polygon[0]
        triangles: list[tuple[Point, Point, float]] = []
        total_area = 0.0
        for index in range(1, len(polygon) - 1):
            area = self._triangle_area(origin, polygon[index
    ], polygon[index + 1])
            if area <= 1e-12:
                continue
            total_area += area
            triangles.append((polygon[index], polygon[index
    + 1], total_area))
        if total_area <= 1e-12:
            return minimum_enclosing_circle(polygon).center
        selector = rng.random() * total_area
        left, right, _ = next(
            triangle for triangle in triangles if selector
    <= triangle[2]
        )
        root = sqrt(rng.random())
        fraction = rng.random()
        return Point(
            (1.0 - root) * origin.x
            + root * (1.0 - fraction) * left.x
            + root * fraction * right.x,
            (1.0 - root) * origin.y
            + root * (1.0 - fraction) * left.y
            + root * fraction * right.y,
        )
    def _posterior_particles(
        self,
        record: ChannelRecord,
    ) -> tuple[SourceParticle, ...]:
        key = (
            record.channel,
            len(record.observations),
            len(record.no_signal_stations),
        )
        cached = self._particle_cache.get(key)
        if cached is not None:
            return cached
        polygon = self._bounded_feasible_polygon(record)
        if not polygon:
            first = record.observations[0]
            fallback = SourceParticle(
                local_to_world(first.station, first.
    bearing_deg, 750.0, 0.0),
                1250.0,
            )
            return (fallback,) * self.particle_count
        seed = (
            1_000_003 * record.channel
            + 10_007 * len(record.observations)
            + 101 * len(record.no_signal_stations)
        )
        rng = Random(seed)
        particles: list[SourceParticle] = []
        maximum_draws = self.particle_count * 400
        for _ in range(maximum_draws):
            point = self._draw_uniform_polygon_point(polygon
    , rng)
            if point.x * point.x + point.y * point.y >
    1800.0**2:
                continue
            positive_distances = [
                point.distance_to(observation.station)
                for observation in record.observations
            ]
            lower_radius = max(1000.0, *positive_distances)
            upper_radius = min(
                1500.0,
                *(
                    point.distance_to(station)
                    for station in record.no_signal_stations
                ),
            ) if record.no_signal_stations else 1500.0
            interval = upper_radius - lower_radius
            if interval <= 0.0:
                continue
            # 对联合均匀先验p(x,R)做拒绝采样。接受概率与该位置
    允许的
            # 接收半径区间长度成正比，从而不会把狭窄半径区间高
    估。
            if rng.random() * 500.0 > interval:
                continue
            receive_radius = lower_radius + rng.random() *
    interval
            particles.append(SourceParticle(point,
    receive_radius))
            if len(particles) >= self.particle_count:
                break
        if not particles:
            center = self._polygon_centroid(polygon)
            lower = max(
                1000.0,
                *(center.distance_to(item.station) for item
    in record.observations),
            )
            particles.append(SourceParticle(center, min
    (1500.0, lower)))
        while len(particles) < self.particle_count:
            particles.extend(particles[: self.particle_count
     - len(particles)])
        result = tuple(particles[: self.particle_count])
        self._particle_cache[key] = result
        return result
    def _rollout_entry(
        self,
        task: tuple[str, int],
        variant: ServiceVariant,
    ) -> Point:
        if task[0] == "cover":
            return variant.entry
        record = self.records[task[1]]
        polygon = self._bounded_feasible_polygon(record)
        if not polygon:
            return variant.entry
        circle = minimum_enclosing_circle(polygon)
        if circle.radius <= 20.0 - 1e-6:
            return circle.center
        if (
            circle.radius <= self.speculative_clear_radius_m
            and self.speculative_clear_at_centroid
        ):
            return self._polygon_centroid(polygon)
        return variant.entry
    def _service_variants(self, record: ChannelRecord) ->
     tuple[ServiceVariant, ...]:
        variants = super()._service_variants(record)
        if self.particle_prediction_scale is None:
            return variants
        particles = self._posterior_particles(record)
        mean_point = Point(
            sum(item.point.x for item in particles) / len(
    particles),
            sum(item.point.y for item in particles) / len(
    particles),
        )
        first = record.observations[0].station
        predicted = Point(
            first.x
            + self.particle_prediction_scale * (mean_point.x
     - first.x),
            first.y
            + self.particle_prediction_scale * (mean_point.y
     - first.y),
        )
        return tuple(ServiceVariant(item.entry, predicted)
     for item in variants)
    def _choose_joint_task(
        self,
        start: Point,
        remaining_cover: set[int],
        pending: list[ChannelRecord],
    ) -> tuple[tuple[str, int], ServiceVariant]:
        if not self.enable_particle_rollout:
            return super()._choose_joint_task(start,
    remaining_cover, pending)
        baseline_task, baseline_variant = super().
    _choose_joint_task(
            start,
            remaining_cover,
            pending,
        )
        variant_map: dict[tuple[str, int], tuple[
    ServiceVariant, ...]] = {}
        for cover_index in remaining_cover:
            point = self.cover_points[cover_index]
            variant_map["cover", cover_index] = (
    ServiceVariant(point, point),)
        for record in pending:
            variant_map["source", record.channel] = self.
    _service_variants(record)
        if len(variant_map) <= 1:
            task = next(iter(variant_map))
            return task, variant_map[task][0]
        particle_map = {
            record.channel: self._posterior_particles(record
    )
            for record in pending
        }
        indexed_variants = [
            (task, index, variant)
            for task, variants in variant_map.items()
            for index, variant in enumerate(variants)
```

```
        ]
        entry_map = {
            (task, index): self._rollout_entry(task, variant
    )
            for task, index, variant in indexed_variants
        }
        predicted_exit_map = {
            (task, index): variant.predicted_exit
            for task, index, variant in indexed_variants
        }
        def scenario_exit(
            task: tuple[str, int],
            variant: ServiceVariant,
            scenario_index: int,
        ) -> Point:
            if task[0] == "cover":
                return variant.predicted_exit
            return particle_map[task[1]][scenario_index].
    point
        ranked_first = sorted(
            indexed_variants,
            key=lambda item: start.distance_to(entry_map[
    item[0], item[1]])
            + entry_map[item[0], item[1]].distance_to(
                predicted_exit_map[item[0], item[1]]
            ),
        )[: self.rollout_candidate_limit]
        if not any(
            task == baseline_task and variant ==
    baseline_variant
            for task, _, variant in ranked_first
        ):
            baseline_index = variant_map[baseline_task].
    index(baseline_variant)
            ranked_first.append(
                (baseline_task, baseline_index,
    baseline_variant)
            )
        choices: list[tuple[float, tuple[str, int],
    ServiceVariant]] = []
        for first_task, first_index, first_variant in
    ranked_first:
            costs: list[float] = []
            for scenario_index in range(self.particle_count)
    :
                first_entry = entry_map[first_task,
    first_index]
                first_exit = scenario_exit(
                    first_task,
                    first_variant,
                    scenario_index,
                )
                cost = start.distance_to(first_entry) +
    first_entry.distance_to(first_exit)
                current = first_exit
                remaining = set(variant_map)
                remaining.remove(first_task)
                steps = 1
                while remaining and steps < self.
    rollout_depth:
                    task, variant_index, variant, entry,
    following = min(
                        (
                            (
                                task,
                                variant_index,
                                variant,
                                entry_map[task,
    variant_index],
                                scenario_exit(task, variant,
     scenario_index),
                            )
                            for task in remaining
                            for variant_index, variant in
    enumerate(variant_map[task])
                        ),
                        key=lambda item: current.distance_to
    (item[3])
                        + item[3].distance_to(item[4]),
                    )
                    cost += current.distance_to(entry) +
    entry.distance_to(following)
                    current = following
                    remaining.remove(task)
                    steps += 1
                costs.append(cost)
            expected = sum(costs) / len(costs)
            tail_count = max(1, len(costs) // 10)
            tail = sum(sorted(costs)[-tail_count:]) /
    tail_count
            score = expected + self.rollout_risk_weight * (
    tail - expected)
            choices.append((score, first_task, first_variant
    ))
        best_score, task, variant = min(choices, key=lambda
    item: item[0])
        baseline_score = next(
            score
            for score, choice_task, choice_variant in
    choices
            if choice_task == baseline_task and
    choice_variant == baseline_variant
        )
        if best_score > baseline_score * (1.0 - self.
    rollout_min_relative_gain):
            return baseline_task, baseline_variant
        return task, variant
class BayesianProbeProblem3Planner(
    ParticleRolloutProblem3Planner):
    """用粒子后验和假想方向带选择单观测源的主动第二测点。"""
    def __init__(
        self,
        *,
        probe_signal_probability: float = 0.85,
        probe_uncertainty_weight: float = 0.25,
        **kwargs: object,
    ) -> None:
        if not 0.0 < probe_signal_probability <= 1.0:
            raise ValueError("主动测点接收概率必须位于0至1")
        if probe_uncertainty_weight < 0.0:
            raise ValueError("主动测点不确定性权重不能为负数"
    )
        kwargs.setdefault("enable_particle_rollout", False)
        kwargs.setdefault("particle_prediction_scale", None)
        kwargs.setdefault("particle_count", 24)
        super().__init__(**kwargs)
        self.probe_signal_probability =
    probe_signal_probability
        self.probe_uncertainty_weight =
    probe_uncertainty_weight
    def _custom_first_probe(
        self,
        record: ChannelRecord,
        polygon: tuple[Point, ...],
        start: Point,
    ) -> Point | None:
        particles = self._posterior_particles(record)
        mean_source = Point(
            sum(item.point.x for item in particles) / len(
    particles),
            sum(item.point.y for item in particles) / len(
    particles),
        )
        safe_probe = self._homing_probe(polygon)
        candidates = [safe_probe, mean_source]
        for radius in (250.0, 450.0, 650.0, 850.0):
            for angle_index in range(16):
                angle = angle_index * pi / 8.0
                candidates.append(
                    Point(
                        mean_source.x + radius * cos(angle),
                        mean_source.y + radius * sin(angle),
                    )
                )
        for fraction in (0.25, 0.5, 0.75):
            candidates.append(
                Point(
                    start.x + fraction * (mean_source.x -
    start.x),
                    start.y + fraction * (mean_source.y -
    start.y),
                )
            )
        evaluated: list[tuple[float, Point]] = []
        for candidate in candidates:
            received = [
                particle
                for particle in particles
                if candidate.distance_to(particle.point)
                <= particle.receive_radius_m
            ]
            signal_probability = len(received) / len(
    particles)
            if signal_probability < self.
    probe_signal_probability:
                continue
            refined_radii: list[float] = []
            # 12个代表粒子足以评估交会几何，避免候选搜索占用过
    多现实时间。
            stride = max(1, len(received) // 12)
            for particle in received[::stride][:12]:
                bearing = degrees(
                    atan2(
                        particle.point.y - candidate.y,
                        particle.point.x - candidate.x,
                    )
                ) % 360.0
                refined = list(polygon)
                for halfplane in bearing_halfplanes(
                    candidate,
                    bearing,
                    self.bearing_error_deg,
                    2,
                ):
                    refined = clip_polygon_with_halfplane(
    refined, halfplane)
                    if not refined:
                        break
                if refined:
                    refined_radii.append(
    minimum_enclosing_circle(refined).radius)
            if not refined_radii:
                continue
            expected_remaining_travel = sum(
                candidate.distance_to(item.point) for item
    in received
            ) / len(received)
            failed_signal_penalty = (1.0 -
    signal_probability) * (
                candidate.distance_to(safe_probe)
                + safe_probe.distance_to(mean_source)
                + 25.0
            )
            score = (
                start.distance_to(candidate)
```

```
                + expected_remaining_travel
                + failed_signal_penalty
                + self.probe_uncertainty_weight
                * sum(refined_radii)
                / len(refined_radii)
            )
            evaluated.append((score, candidate))
        if not evaluated:
            return safe_probe
        return min(evaluated, key=lambda item: item[0])[1]
class ExpectedCostProblem3Planner(
    ParticleRolloutProblem3Planner):
    """以粒子期望转移距离求解滚动非对称旅行商的候选规划器。"""
    def __init__(
        self,
        *,
        edge_risk_weight: float = 0.0,
        **kwargs: object,
    ) -> None:
        if edge_risk_weight < 0.0:
            raise ValueError("边风险权重不能为负数")
        kwargs.setdefault("enable_particle_rollout", False)
        kwargs.setdefault("particle_prediction_scale", None)
        super().__init__(**kwargs)
        self.edge_risk_weight = edge_risk_weight
    def _choose_joint_task(
        self,
        start: Point,
        remaining_cover: set[int],
        pending: list[ChannelRecord],
    ) -> tuple[tuple[str, int], ServiceVariant]:
        variant_map: dict[tuple[str, int], ServiceVariant] = {}
        for cover_index in remaining_cover:
            point = self.cover_points[cover_index]
            variant_map["cover", cover_index] = ServiceVariant(point, point)
        for record in pending:
            # 该实验只使用每个任务的主入口；多入口版本单独评估，避免把
            # “入口选择”和“随机转移代价”两种变化混在一次消融实验里。
            variant_map["source", record.channel] = super()._service_variants(record)[0]
        if len(variant_map) == 1:
            task = next(iter(variant_map))
            return task, variant_map[task]
        entries = {
            task: self._rollout_entry(task, variant)
            for task, variant in variant_map.items()
        }
        particle_map = {
            record.channel: self._posterior_particles(record)
            for record in pending
        }
        def exit_points(task: tuple[str, int]) -> tuple[Point, ...]:
            if task[0] == "cover":
                return (variant_map[task].predicted_exit,)
            return tuple(item.point for item in particle_map[task[1]])
        exits = {task: exit_points(task) for task in variant_map}
        internal = {
            task: sum(entries[task].distance_to(point) for point in exits[task])
            / len(exits[task])
            for task in variant_map
        }
        def robust_mean(values: list[float]) -> float:
            expected = sum(values) / len(values)
            if self.edge_risk_weight <= 1e-12 or len(values) == 1:
                return expected
            tail_count = max(1, len(values) // 10)
            tail = sum(sorted(values)[-tail_count:]) / tail_count
            return expected + self.edge_risk_weight * (tail - expected)
        transition = {
            (left, right): robust_mean(
                [point.distance_to(entries[right]) for point in exits[left]]
            )
            for left in variant_map
            for right in variant_map
            if left != right
        }
        remaining = set(variant_map)
        first = min(
            remaining,
            key=lambda task: start.distance_to(entries[task]) + internal[task],
        )
        order = [first]
        remaining.remove(first)
        while remaining:
            following = min(
                remaining,
                key=lambda task: transition[order[-1], task] + internal[task],
            )
            order.append(following)
            remaining.remove(following)
        def route_cost(route: tuple[tuple[str, int], ...]) -> float:
            cost = start.distance_to(entries[route[0]])
            cost += sum(internal[task] for task in route)
            cost += sum(
                transition[left, right]
                for left, right in zip(route, route[1:])
            )
            return cost
        best_order = tuple(order)
        best_cost = route_cost(best_order)
        improved = True
        while improved:
            improved = False
            candidate_order = best_order
            candidate_cost = best_cost
            for left in range(len(best_order) - 1):
                for right in range(left + 1, len(best_order)):
                    trial = (
                        best_order[:left]
                        + tuple(reversed(best_order[left : right + 1]))
                        + best_order[right + 1 :]
                    )
                    cost = route_cost(trial)
                    if cost < candidate_cost - 1e-7:
                        candidate_order = trial
                        candidate_cost = cost
            for source_index in range(len(best_order)):
                shortened = list(best_order)
                moved = shortened.pop(source_index)
                for target_index in range(len(best_order)):
                    trial_list = shortened.copy()
                    trial_list.insert(target_index, moved)
                    trial = tuple(trial_list)
                    if trial == best_order:
                        continue
                    cost = route_cost(trial)
                    if cost < candidate_cost - 1e-7:
                        candidate_order = trial
                        candidate_cost = cost
            if candidate_cost < best_cost - 1e-7:
                best_order = candidate_order
                best_cost = candidate_cost
                improved = True
        task = best_order[0]
        return task, variant_map[task]
class RefinedProbeProblem3Planner(
    ExpectedCostProblem3Planner,
    BayesianProbeProblem3Planner,
):
    """Particle-posterior probe search with expected-cost rolling routing.
    The probabilistic probe only changes efficiency. A no-signal result falls
    back to the deterministic localization/grid-clear path, so completeness
    under the stated problem model does not depend on the particle prior.
    """
    probe_radii: tuple[float, ...] = (150.0, 300.0, 450.0, 600.0, 750.0, 900.0)
    probe_angles: int = 24
    probe_fractions: tuple[float, ...] = (
        0.1,
        0.2,
        0.3,
        0.4,
        0.5,
        0.6,
        0.7,
        0.8,
        0.9,
    )
    refine_top: int = 4
    refine_radius_steps: tuple[float, ...] = (-80.0, -40.0, 0.0, 40.0, 80.0)
    refine_angle_steps: int = 8
    def __init__(
        self,
        *,
        probe_uncertainty_weight: float = 0.4,
        custom_probe_max_observations: int = 2,
        **kwargs: object,
    ) -> None:
        kwargs.setdefault(
            "custom_probe_max_observations",
            custom_probe_max_observations
        )
        kwargs.setdefault("particle_count", 48)
        kwargs.setdefault("speculative_clear_radius_m", 100.0)
        kwargs.setdefault("speculative_clear_at_centroid", True)
        kwargs.setdefault("aggressive_opportunity_distance_m", 1000.0)
        kwargs.setdefault("aggressive_opportunity_min_angle_deg", 15.0)
        kwargs.setdefault("max_aggressive_opportunity_failures", 1)
        kwargs.setdefault("cover_point_design", "seven_guaranteed")
        super().__init__(
            probe_uncertainty_weight=probe_uncertainty_weight,
            **kwargs,
        )
    def _evaluate_probe_candidates(
        self,
```

```
        record: ChannelRecord,
        polygon: tuple[Point, ...],
        start: Point,
        candidates: list[Point],
    ) -> tuple[list[tuple[float, Point]], Point, Point]:
        particles = self._posterior_particles(record)
        mean_source = Point(
            sum(item.point.x for item in particles) / len(particles),
            sum(item.point.y for item in particles) / len(particles),
        )
        safe_probe = self._homing_probe(polygon)
        evaluated: list[tuple[float, Point]] = []
        for candidate in candidates:
            received = [
                particle
                for particle in particles
                if candidate.distance_to(particle.point) <= particle.receive_radius_m
            ]
            signal_probability = len(received) / len(particles)
            if signal_probability < self.probe_signal_probability:
                continue
            refined_radii: list[float] = []
            stride = max(1, len(received) // 12)
            for particle in received[::stride][:12]:
                bearing = degrees(
                    atan2(
                        particle.point.y - candidate.y,
                        particle.point.x - candidate.x,
                    )
                ) % 360.0
                refined = list(polygon)
                for halfplane in bearing_halfplanes(
                    candidate,
                    bearing,
                    self.bearing_error_deg,
                    2,
                ):
                    refined = clip_polygon_with_halfplane(refined, halfplane)
                    if not refined:
                        break
                if refined:
                    refined_radii.append(minimum_enclosing_circle(refined).radius)
            if not refined_radii:
                continue
            expected_remaining_travel = sum(
                candidate.distance_to(item.point) for item in received
            ) / len(received)
            failed_signal_penalty = (1.0 - signal_probability) * (
                candidate.distance_to(safe_probe)
                + safe_probe.distance_to(mean_source)
                + 25.0
            )
            score = (
                start.distance_to(candidate)
                + expected_remaining_travel
                + failed_signal_penalty
                + self.probe_uncertainty_weight
                * sum(refined_radii)
                / len(refined_radii)
            )
            evaluated.append((score, candidate))
        return evaluated, safe_probe, mean_source
    def _coarse_probe_candidates(
        self,
        start: Point,
        mean_source: Point,
        safe_probe: Point,
    ) -> list[Point]:
        candidates = [safe_probe, mean_source]
        for radius in self.probe_radii:
            for index in range(self.probe_angles):
                angle = 2.0 * pi * index / self.probe_angles
                candidates.append(
                    Point(
                        mean_source.x + radius * cos(angle),
                        mean_source.y + radius * sin(angle),
                    )
                )
        for fraction in self.probe_fractions:
            candidates.append(
                Point(
                    start.x + fraction * (mean_source.x - start.x),
                    start.y + fraction * (mean_source.y - start.y),
                )
            )
        return candidates
    def _custom_first_probe(
        self,
        record: ChannelRecord,
        polygon: tuple[Point, ...],
        start: Point,
    ) -> Point | None:
        particles = self._posterior_particles(record)
        mean_source = Point(
            sum(item.point.x for item in particles) / len(particles),
            sum(item.point.y for item in particles) / len(particles),
        )
        safe_probe = self._homing_probe(polygon)
        coarse = self._coarse_probe_candidates(start, mean_source, safe_probe)
        evaluated, safe_probe, _ = self._evaluate_probe_candidates(
            record,
            polygon,
            start,
            coarse,
        )
        if not evaluated:
            return safe_probe
        seeds = sorted(evaluated, key=lambda item: item[0])[: self.refine_top]
        fine: list[Point] = []
        for _, seed_point in seeds:
            for radius_step in self.refine_radius_steps:
                for index in range(self.refine_angle_steps):
                    angle = 2.0 * pi * index / self.refine_angle_steps
                    fine.append(
                        Point(
                            seed_point.x + radius_step * cos(angle),
                            seed_point.y + radius_step * sin(angle),
                        )
                    )
        refined, _, _ = self._evaluate_probe_candidates(
            record,
            polygon,
            start,
            fine,
        )
        pool = evaluated + refined
        return min(pool, key=lambda item: item[0])[1]
```

problem4_planner.py

```python
"""问题4混合全向/定向干扰源的确定性首版规划器。"""
from __future__ import annotations
# 论文附录版：仅保留最终算法与消融基线所用的类，
# 完整工程（含其余变体与实验脚本）见支撑材料 code/ 目录。
from dataclasses import dataclass, field
from math import acos, atan2, ceil, cos, degrees, pi, radians, sin, sqrt
from random import Random
from typing import Protocol
from .coverage import triangular_lattice_points
from .geometry import (
    HalfPlane,
    LocalizationResult,
    Point,
    bearing_halfplanes,
    build_localization_polygon,
    clip_polygon_with_halfplane,
    minimum_enclosing_circle,
)
from .station_selection import choose_second_station, local_to_world
class RobotInterface(Protocol):
    def enter(self) -> dict[str, object]: ...
    def measure(self, x: float, y: float, channel: int) -> dict[str, object]: ...
    def clear(self, x: float, y: float, channel: int) -> dict[str, object]: ...
    def exit(self) -> dict[str, object]: ...
@dataclass(frozen=True)
class BearingObservation:
    station: Point
    bearing_deg: float
@dataclass
class ChannelRecord:
    channel: int
    observations: list[BearingObservation] = field(default_factory=list)
    cleared: bool = False
@dataclass(frozen=True)
class Problem4Report:
    detected_channels: tuple[int, ...]
    cleared_channels: tuple[int, ...]
    fallback_channels: tuple[int, ...]
    exit_response: dict[str, object]
class Problem4Planner:
    def __init__(
        self,
        *,
        bearing_error_deg: float = 1.0,
        lattice_spacing_m: float = 900.0,
        max_refinement_measurements: int = 4,
        sector_search_mode: str = "outward",
    ) -> None:
        if sector_search_mode not in {"outward", "inward", "expected"}:
            raise ValueError("sector_search_mode必须是outward、inward或expected")
        self.bearing_error_deg = bearing_error_deg
        self.cover_points = triangular_lattice_points(1800.0, lattice_spacing_m)
        self.max_refinement_measurements = max_refinement_measurements
        self.sector_search_mode = sector_search_mode
```

```python
        self.records = {channel: ChannelRecord(channel) for channel in range(1, 21)}
        self.fallback_channels: list[int] = []
    def run(self, robot: RobotInterface) -> Problem4Report:
        robot.enter()
        unknown = set(range(1, 21))
        detected: set[int] = set()
        current_channel = 1
        current_position = Point(0.0, 0.0)
        for point in self.cover_points:
            current_position = point
            ordered_channels = sorted(unknown, key=lambda channel: channel != current_channel)
            for channel in ordered_channels:
                response = robot.measure(point.x, point.y, channel)
                current_channel = channel
                result = response["measure_result"]
                if result == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] != "success":
                        raise RuntimeError("near后清除失败")
                    self.records[channel].cleared = True
                    detected.add(channel)
                    unknown.remove(channel)
                elif result == "direction":
                    self.records[channel].observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                    detected.add(channel)
                    unknown.remove(channel)
            if len(detected) == 16:
                break
        # 仅用于离线瓶颈分析；正式接口没有这些属性时自动记为None。
        self.coverage_diagnostics = {
            "virtual_time_s": getattr(robot, "virtual_time_s", None),
            "travel_distance_m": getattr(robot, "travel_distance_m", None),
            "measure_count": getattr(robot, "measure_count", None),
            "clear_attempt_count": getattr(robot, "clear_attempt_count", None),
            "pending_count": sum(
                not self.records[channel].cleared for channel in detected
            ),
            "single_observation_count": sum(
                not self.records[channel].cleared
                and len(self.records[channel].observations) == 1
                for channel in detected
            ),
        }
        pending = [
            self.records[channel]
            for channel in detected
            if not self.records[channel].cleared
        ]
        while pending:
            record, selected = min(
                (
                    (record, point)
                    for record in pending
                    for point in self._symmetric_second_points(record)
                ),
                key=lambda item: current_position.distance_to(item[1]),
            )
            current_position = self._localize_or_cover(
                robot, record, selected, current_position
            )
            pending.remove(record)
        response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return Problem4Report(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            fallback_channels=tuple(self.fallback_channels),
            exit_response=response,
        )
    @staticmethod
    def _symmetric_second_points(record: ChannelRecord) -> tuple[Point, Point]:
        first = record.observations[0]
        base = choose_second_station(first.station, first.bearing_deg)
        lateral = abs(base.lateral_m)
        return (
            local_to_world(first.station, first.bearing_deg, base.forward_m, -lateral),
            local_to_world(first.station, first.bearing_deg, base.forward_m, lateral),
        )
    def _localize_or_cover(
        self,
        robot: RobotInterface,
        record: ChannelRecord,
        selected_second: Point,
        current_position: Point,
    ) -> Point:
        second_points = list(self._symmetric_second_points(record))
        if selected_second == second_points[1]:
            second_points.reverse()
        for point in second_points:
            response = robot.measure(point.x, point.y, record.channel)
            current_position = point
            if response["measure_result"] == "near":
                return self._clear_success(robot, point, record)
            if response["measure_result"] == "direction":
                record.observations.append(
                    BearingObservation(point, float(response["svd_deg"]))
                )
                return self._refine_after_two_bearings(
                    robot, record, current_position
                )
        self.fallback_channels.append(record.channel)
        return self._clear_initial_sector(robot, record, current_position)
    def _refine_after_two_bearings(
        self, robot: RobotInterface, record: ChannelRecord, current_position: Point
    ) -> Point:
        for _ in range(self.max_refinement_measurements):
            result = build_localization_polygon(
                [(obs.station, obs.bearing_deg) for obs in record.observations],
                error_deg=self.bearing_error_deg,
            )
            if not result.polygon or not result.bounded or result.minimum_circle is None:
                break
            probe = result.minimum_circle.center
            if result.minimum_circle.radius <= 20.0 - 1e-6:
                return self._clear_success(robot, probe, record)
            response = robot.measure(probe.x, probe.y, record.channel)
            current_position = probe
            if response["measure_result"] == "near":
                return self._clear_success(robot, probe, record)
            if response["measure_result"] != "direction":
                break
            record.observations.append(
                BearingObservation(probe, float(response["svd_deg"]))
            )
        self.fallback_channels.append(record.channel)
        return self._clear_initial_sector(robot, record, current_position)
    @staticmethod
    def _clear_success(
        robot: RobotInterface, point: Point, record: ChannelRecord
    ) -> Point:
        response = robot.clear(point.x, point.y, record.channel)
        if response["clear_result"] != "success":
            raise RuntimeError(f"频道{record.channel}的保证清除点失败")
        record.cleared = True
        return point
    def _clear_initial_sector(
        self, robot: RobotInterface, record: ChannelRecord, current_position: Point
    ) -> Point:
        """由近及远/由远及近覆盖真实误差扇形，而非扫描外接矩形。
        将0--1500米前向区间切成宽度不超过30米的小段。每段中心与
        段内任一点的前向差不超过15米，故可用
        sqrt(20^2-15^2)的横向覆盖预算布置1--2个清除点。这样每个
        扇形点到某个清除点严格小于20米，同时避免旧版逐整行折返。
        """
        first = record.observations[0]
        delta = radians(self.bearing_error_deg)
        max_range = 1500.0
        column_count = ceil(max_range / 30.0)
        column_width = max_range / column_count
        horizontal_error = column_width / 2.0
        lateral_budget = sqrt(20.0**2 - horizontal_error**2) - 0.05
        tangent = sin(delta) / max(1e-12, cos(delta))
        sector_tip_x = max_range * cos(delta)
        columns: list[list[tuple[float, float]]] = []
        for column_index in range(column_count):
            lower_x = column_index * column_width
            upper_x = (column_index + 1) * column_width
            center_x = (lower_x + upper_x) / 2.0
            candidates_x = (
                lower_x,
                upper_x,
                min(max(sector_tip_x, lower_x), upper_x),
            )
            maximum_half_width = max(
                min(
                    x * tangent,
                    sqrt(max(0.0, max_range**2 - x * x)),
```

```
                )
                for x in candidates_x
            )
            lateral_count = max(1, ceil(maximum_half_width / lateral_budget))
            if lateral_count == 1:
                lateral_values = [0.0]
            else:
                first_lateral = -maximum_half_width + lateral_budget
                last_lateral = maximum_half_width - lateral_budget
                lateral_values = [
                    first_lateral
                    + (last_lateral - first_lateral) * index / (lateral_count - 1)
                    for index in range(lateral_count)
                ]
            columns.append([(center_x, value) for value in lateral_values])
        def make_route(reverse: bool) -> list[Point]:
            route: list[Point] = []
            last_lateral = 0.0
            iterable = reversed(columns) if reverse else iter(columns)
            for column in iterable:
                ordered = sorted(column, key=lambda item: abs(item[1] - last_lateral))
                for forward, lateral in ordered:
                    point = local_to_world(
                        first.station, first.bearing_deg, forward, lateral
                    )
                    # 若某中心离目标圆超过20米，则它不可能是目标圆内
                    # 任意可行源点的最近保证中心，可以安全删去。
                    if sqrt(point.x * point.x + point.y * point.y) <= 1820.0 + 1e-8:
                        route.append(point)
                        last_lateral = lateral
            return route
        outward = make_route(False)
        inward = make_route(True)
        if self.sector_search_mode == "outward":
            world_points = outward
        elif self.sector_search_mode == "inward":
            world_points = inward
        else:
            predicted_range = float(getattr(self, "predicted_range_m", 850.0))
            predicted = local_to_world(
                first.station, first.bearing_deg, predicted_range, 0.0
            )
            def expected_prefix_cost(route: list[Point]) -> float:
                if not route:
                    return float("inf")
                target_index = min(
                    range(len(route)), key=lambda index: route[index].distance_to(predicted)
                )
                distance = current_position.distance_to(route[0])
                distance += sum(
                    route[index - 1].distance_to(route[index])
                    for index in range(1, target_index + 1)
                )
                # 清除失败耗时3秒，以5m/s折成15米等价距离。
                return distance + 15.0 * (target_index + 1)
            world_points = min(
                (outward, inward), key=expected_prefix_cost
            )
        for point in world_points:
            response = robot.clear(point.x, point.y, record.channel)
            if response["clear_result"] == "success":
                record.cleared = True
                return point
        raise RuntimeError(f"频道{record.channel}的首次扇形覆盖未清除目标")
class MeshProblem4Planner(Problem4Planner):
    """使用25点有限三角网格和开放路径2-opt顺序的保证覆盖版。
    """
    _COVER_ROUTE = (
        Point(148.500000, 85.736515),
        Point(-346.500000, 943.101665),
        Point(-841.500000, 1800.466814),
        Point(-1336.500000, 943.101665),
        Point(-1831.500000, 1800.466814),
        Point(-2326.500000, 943.101665),
        Point(-1831.500000, 85.736515),
        Point(-2326.500000, -771.628635),
        Point(-1831.500000, -1628.993785),
        Point(-1336.500000, -771.628635),
        Point(-841.500000, 85.736515),
        Point(-346.500000, -771.628635),
        Point(-841.500000, -1628.993785),
        Point(-346.500000, -2486.358934),
        Point(148.500000, -1628.993785),
        Point(643.500000, -771.628635),
        Point(1138.500000, 85.736515),
        Point(643.500000, 943.101665),
        Point(148.500000, 1800.466814),
        Point(1138.500000, 1800.466814),
        Point(1633.500000, 943.101665),
        Point(2128.500000, 85.736515),
        Point(1633.500000, -771.628635),
        Point(1138.500000, -1628.993785),
        Point(643.500000, -2486.358934),
    )
    def __init__(self, **kwargs: object) -> None:
        super().__init__(**kwargs)
        self.cover_points = list(self._COVER_ROUTE)
class MeshOpticalProblem4Planner(MeshProblem4Planner):
    """25点保证发现 + 已定位小区域的方向无关光学覆盖。"""
    def __init__(
        self,
        *,
        maximum_refinement_circle_m: float = 250.0,
        speculative_clear_radius_m: float = 0.0,
        refinement_probe_mode: str = "circle",
        polygon_search_mode: str = "legacy",
        **kwargs: object,
    ) -> None:
        super().__init__(**kwargs)
        self.maximum_refinement_circle_m = maximum_refinement_circle_m
        self.speculative_clear_radius_m = speculative_clear_radius_m
        if refinement_probe_mode not in {"circle", "centroid"}:
            raise ValueError("refinement_probe_mode必须是circle或centroid")
        if polygon_search_mode not in {"legacy", "nearest"}:
            raise ValueError("polygon_search_mode必须是legacy或nearest")
        self.refinement_probe_mode = refinement_probe_mode
        self.polygon_search_mode = polygon_search_mode
        self._bounded_result_cache: dict[
            tuple[tuple[float, float, float], ...], LocalizationResult
        ] = {}
    def _bounded_localization_result(
        self, record: ChannelRecord
    ) -> LocalizationResult:
        """示向半平面交再叠加目标圆和1500米接收上界。
        圆约束采用外切正多边形，因此所得区域是真实可行域的外包，
        只会略微保守，不会把真源错误裁掉。
        """
        key = tuple(
            (observation.station.x, observation.station.y, observation.bearing_deg)
            for observation in record.observations
        )
        cached = self._bounded_result_cache.get(key)
        if cached is not None:
            return cached
        sides = 72
        outer_radius = 1800.0 / cos(pi / sides)
        polygon = [
            Point(
                outer_radius * cos((index + 0.5) * 2.0 * pi / sides),
                outer_radius * sin((index + 0.5) * 2.0 * pi / sides),
            )
            for index in range(sides)
        ]
        for observation_index, observation in enumerate(record.observations, 1):
            for halfplane in bearing_halfplanes(
                observation.station,
                observation.bearing_deg,
                self.bearing_error_deg,
                observation_index,
            ):
                polygon = clip_polygon_with_halfplane(polygon, halfplane)
                if not polygon:
                    break
            if not polygon:
                break
            for normal_index in range(36):
                angle = normal_index * 2.0 * pi / 36.0
                normal_x, normal_y = cos(angle), sin(angle)
                polygon = clip_polygon_with_halfplane(
                    polygon,
                    HalfPlane(
                        normal_x,
                        normal_y,
                        normal_x * observation.station.x
                        + normal_y * observation.station.y
                        + 1500.0,
                        f"range-{observation_index}",
                    ),
                )
                if not polygon:
                    break
            if not polygon:
                break
        if polygon:
            circle = minimum_enclosing_circle(polygon)
            result = LocalizationResult(
                tuple(polygon), True, None, None, circle, None, None
            )
        else:
            result = LocalizationResult(tuple(), True, None, None, None, None, None)
        self._bounded_result_cache[key] = result
```

```
        return result
    @staticmethod
    def _polygon_centroid(polygon: tuple[Point, ...]) ->
      Point:
        if len(polygon) < 3:
            return minimum_enclosing_circle(polygon).center
        doubled_area = 0.0
        cx = 0.0
        cy = 0.0
        for index, point in enumerate(polygon):
            following = polygon[(index + 1) % len(polygon)]
            cross = point.x * following.y - following.x *
      point.y
            doubled_area += cross
            cx += (point.x + following.x) * cross
            cy += (point.y + following.y) * cross
        if abs(doubled_area) <= 1e-10:
            return minimum_enclosing_circle(polygon).center
        return Point(cx / (3.0 * doubled_area), cy / (3.0 *
      doubled_area))
    def _refinement_probe(self, polygon: tuple[Point, ...])
      -> Point:
        if self.refinement_probe_mode == "centroid":
            return self._polygon_centroid(polygon)
        return minimum_enclosing_circle(polygon).center
    def _refine_after_two_bearings(
        self, robot: RobotInterface, record: ChannelRecord,
      current_position: Point
    ) -> Point:
        for _ in range(self.max_refinement_measurements):
            result = self._bounded_localization_result(
      record)
            if not result.polygon or not result.bounded or
      result.minimum_circle is None:
                break
            if result.minimum_circle.radius > self.
      maximum_refinement_circle_m:
                self.fallback_channels.append(record.channel
      )
                return self._clear_initial_sector(robot,
      record, current_position)
            probe = self._refinement_probe(result.polygon)
            if result.minimum_circle.radius <= 20.0 - 1e-6:
                response = robot.clear(probe.x, probe.y,
      record.channel)
                if response["clear_result"] == "success":
                    record.cleared = True
                    return probe
                # 数值临界或模拟器误差时不中断整局，改扫当前多
      边形。
                return self._clear_polygon(robot, result.
      polygon, record, probe)
            if result.minimum_circle.radius <= self.
      speculative_clear_radius_m:
                response = robot.clear(probe.x, probe.y,
      record.channel)
                if response["clear_result"] == "success":
                    record.cleared = True
                    return probe
            response = robot.measure(probe.x, probe.y,
      record.channel)
            current_position = probe
            if response["measure_result"] == "near":
                clear_response = robot.clear(probe.x, probe.
      y, record.channel)
                if clear_response["clear_result"] == "
      success":
                    record.cleared = True
                    return probe
                return self._clear_polygon(robot, result.
      polygon, record, probe)
            if response["measure_result"] == "direction":
                record.observations.append(
                    BearingObservation(probe, float(response
      ["svd_deg"]))
                )
                continue
            # 定向源可能在此点无线电失信号，但两次测向已把源限
      制
            # 在小多边形中；直接使用与发射朝向无关的20米光学覆
      盖。
            return self._clear_polygon(robot, result.polygon
      , record, current_position)
        result = self._bounded_localization_result(record)
        if (
            result.polygon
            and result.bounded
            and result.minimum_circle is not None
            and result.minimum_circle.radius <= self.
      maximum_refinement_circle_m
        ):
            return self._clear_polygon(
                robot, result.polygon, record,
      current_position
            )
        self.fallback_channels.append(record.channel)
        return self._clear_initial_sector(robot, record,
      current_position)
    def _clear_polygon(
        self,
        robot: RobotInterface,
        polygon: tuple[Point, ...],
        record: ChannelRecord,
        current_position: Point,
    ) -> Point:
        """在首次示向局部坐标中覆盖多边形，避免斜细长区域放大。
      """
        step = 20.0 * sqrt(2.0)
        first = record.observations[0]
        theta = radians(first.bearing_deg)
        direction_x, direction_y = cos(theta), sin(theta)
        lateral_x, lateral_y = -sin(theta), cos(theta)
        local = [
            (
                (point.x - first.station.x) * direction_x
                + (point.y - first.station.y) * direction_y,
                (point.x - first.station.x) * lateral_x
                + (point.y - first.station.y) * lateral_y,
            )
            for point in polygon
        ]
        min_x = min(point[0] for point in local)
        max_x = max(point[0] for point in local)
        min_y = min(point[1] for point in local)
        max_y = max(point[1] for point in local)
        nx = max(1, ceil((max_x - min_x) / step))
        ny = max(1, ceil((max_y - min_y) / step))
        rows: list[list[Point]] = []
        for row in range(ny + 1):
            y = min_y + (max_y - min_y) * row / ny
            columns = range(nx + 1) if row % 2 == 0 else
      range(nx, -1, -1)
            rows.append(
                [
                    local_to_world(
                        first.station,
                        first.bearing_deg,
                        min_x + (max_x - min_x) * column /
      nx,
                        y,
                    )
                    for column in columns
                ]
            )
        forward = [point for row in rows for point in row]
        if self.polygon_search_mode == "nearest":
            remaining = list(forward)
            points: list[Point] = []
            cursor = current_position
            while remaining:
                selected = min(
                    remaining,
                    key=lambda point: (cursor.distance_to(
      point), point.x, point.y),
                )
                points.append(selected)
                remaining.remove(selected)
                cursor = selected
        else:
            reverse = list(reversed(forward))
            points = min(
                (forward, reverse),
                key=lambda route: current_position.
      distance_to(route[0]),
            )
        final = current_position
        for point in points:
            final = point
            response = robot.clear(point.x, point.y, record.
      channel)
            if response["clear_result"] == "success":
                record.cleared = True
                return point
        # 理论上不会到达；数值或接口异常时交给上层完整扇形保
      底。
        return final
class AdaptiveProblem4Planner(MeshOpticalProblem4Planner):
    """问题4独立自适应版：保证发现时顺路收集第二条示向线。"""
    def __init__(
        self,
        *,
        opportunity_attempts: int = 2,
        opportunity_distance_m: float = 1100.0,
        opportunity_min_angle_deg: float = 12.0,
        predicted_range_m: float = 850.0,
        inline_service_detour_m: float | None = None,
        inline_service_circle_limit_m: float = 140.0,
        single_observation_optical: bool = False,
        global_service_route: bool = False,
        **kwargs: object,
    ) -> None:
        super().__init__(**kwargs)
        self.opportunity_attempts = opportunity_attempts
        self.opportunity_distance_m = opportunity_distance_m
        self.opportunity_min_angle_deg =
      opportunity_min_angle_deg
        self.predicted_range_m = predicted_range_m
        self.inline_service_detour_m =
      inline_service_detour_m
        self.inline_service_circle_limit_m =
      inline_service_circle_limit_m
        self.single_observation_optical =
      single_observation_optical
        self.global_service_route = global_service_route
        self._opportunity_counts: dict[int, int] = {}
    @staticmethod
    def _crossing_angle(first: Point, second: Point, source:
      Point) -> float:
        ax = source.x - first.x
        ay = source.y - first.y
        bx = source.x - second.x
        by = source.y - second.y
        na = sqrt(ax * ax + ay * ay)
        nb = sqrt(bx * bx + by * by)
```

```python
        if na <= 1e-9 or nb <= 1e-9:
            return 90.0
        cosine = min(1.0, max(0.0, abs((ax * bx + ay * by) / (na * nb))))
        return degrees(acos(cosine))
    def _predicted_source(self, record: ChannelRecord) -> Point:
        first = record.observations[0]
        return local_to_world(
            first.station,
            first.bearing_deg,
            self.predicted_range_m,
            0.0,
        )
    def _service_target(self, record: ChannelRecord) -> Point:
        if len(record.observations) >= 2:
            result = self._bounded_localization_result(record)
            if result.bounded and result.minimum_circle is not None:
                return self._refinement_probe(result.polygon)
        return self._predicted_source(record)
    def _global_pending_order(
        self, current: Point, pending: list[ChannelRecord]
    ) -> list[ChannelRecord]:
        remaining = list(pending)
        route: list[ChannelRecord] = []
        cursor = current
        while remaining:
            record = min(
                remaining,
                key=lambda item: (
                    cursor.distance_to(self._service_target(item)),
                    item.channel,
                ),
            )
            route.append(record)
            remaining.remove(record)
            cursor = self._service_target(record)
        def length(order: list[ChannelRecord]) -> float:
            total = 0.0
            point = current
            for item in order:
                target = self._service_target(item)
                total += point.distance_to(target)
                point = target
            return total
        improved = True
        while improved:
            improved = False
            best = length(route)
            for left in range(len(route) - 1):
                for right in range(left + 1, len(route)):
                    candidate = (
                        route[:left]
                        + list(reversed(route[left : right + 1]))
                        + route[right + 1 :]
                    )
                    value = length(candidate)
                    if value + 1e-8 < best:
                        route = candidate
                        best = value
                        improved = True
        return route
    def _adapt_remaining_cover_route(
        self,
        point_index: int,
        current_position: Point,
        detected: set[int],
    ) -> None:
        del point_index, current_position, detected
    def run(self, robot: RobotInterface) -> Problem4Report:
        robot.enter()
        unknown = set(range(1, 21))
        detected: set[int] = set()
        current_channel = 1
        current_position = Point(0.0, 0.0)
        for point_index, point in enumerate(self.cover_points):
            scheduled_services = getattr(self, "_scheduled_services", {})
            scheduled_record = scheduled_services.pop(
                (round(point.x, 9), round(point.y, 9)), None
            )
            if scheduled_record is not None:
                if not scheduled_record.cleared:
                    if len(scheduled_record.observations) >= 2:
                        current_position = self._refine_after_two_bearings(
                            robot, scheduled_record, current_position
                        )
                    else:
                        entries = self._symmetric_second_points(scheduled_record)
                        entry = min(
                            entries,
                            key=lambda item: current_position.distance_to(item),
                        )
                        current_position = self._localize_or_cover(
                            robot, scheduled_record, entry, current_position
                        )
                continue
            ordered_channels = sorted(unknown, key=lambda channel: channel != current_channel)
            for channel in ordered_channels:
                response = robot.measure(point.x, point.y, channel)
                current_position = point
                current_channel = channel
                result = response["measure_result"]
                if result == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] == "success":
                        self.records[channel].cleared = True
                        detected.add(channel)
                        unknown.remove(channel)
                elif result == "direction":
                    self.records[channel].observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                    detected.add(channel)
                    unknown.remove(channel)
            # 只在无额外移动的当前覆盖点尝试第二次测向。
            candidates: list[tuple[float, int, ChannelRecord]] = []
            for channel in sorted(detected):
                record = self.records[channel]
                if record.cleared or len(record.observations) != 1:
                    continue
                if self._opportunity_counts.get(channel, 0) >= self.opportunity_attempts:
                    continue
                predicted = self._predicted_source(record)
                distance = point.distance_to(predicted)
                if distance > self.opportunity_distance_m:
                    continue
                angle = self._crossing_angle(
                    record.observations[0].station, point, predicted
                )
                if angle < self.opportunity_min_angle_deg:
                    continue
                candidates.append((-angle, channel, record))
            for _, channel, record in sorted(candidates):
                self._opportunity_counts[channel] = self._opportunity_counts.get(channel, 0) + 1
                response = robot.measure(point.x, point.y, channel)
                current_channel = channel
                if response["measure_result"] == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] == "success":
                        record.cleared = True
                elif response["measure_result"] == "direction":
                    record.observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
            if (
                self.inline_service_detour_m is not None
                and point_index + 1 < len(self.cover_points)
            ):
                following = self.cover_points[point_index + 1]
                while True:
                    service_candidates: list[
                        tuple[float, int, ChannelRecord]
                    ] = []
                    for channel in sorted(detected):
                        record = self.records[channel]
                        if record.cleared or len(record.observations) < 2:
                            continue
                        result = self._bounded_localization_result(record)
                        if (
                            not result.bounded
                            or result.minimum_circle is None
                            or result.minimum_circle.radius
                            > self.inline_service_circle_limit_m
                        ):
                            continue
                        center = result.minimum_circle.center
                        detour = (
                            current_position.distance_to(center)
                            + center.distance_to(following)
                            - current_position.distance_to(following)
                        )
                        if detour <= self.inline_service_detour_m:
                            service_candidates.append((detour, channel, record))
                    if not service_candidates:
                        break
                    _, _, record = min(service_candidates)
```

```
                    current_position = self._refine_after_two_bearings(
                        robot, record, current_position
                    )
            # 题设源数上限为16；已找到16个时不可能再有未知源。
            if len(detected) == 16:
                break
            self._adapt_remaining_cover_route(
                point_index, current_position, detected
            )
        self.coverage_diagnostics = {
            "virtual_time_s": getattr(robot, "virtual_time_s", None),
            "travel_distance_m": getattr(robot, "travel_distance_m", None),
            "measure_count": getattr(robot, "measure_count", None),
            "clear_attempt_count": getattr(robot, "clear_attempt_count", None),
            "pending_count": sum(
                not self.records[channel].cleared for channel in detected
            ),
            "single_observation_count": sum(
                not self.records[channel].cleared
                and len(self.records[channel].observations) == 1
                for channel in detected
            ),
        }
        pending = [
            self.records[channel]
            for channel in detected
            if not self.records[channel].cleared
        ]
        planned_order = (
            self._global_pending_order(current_position, pending)
            if self.global_service_route
            else None
        )
        while pending:
            # 已有两次示向的频道入口取当前小定位区域中心；
            # 只有一次示向的频道仍使用左右交会点保底。
            choices: list[tuple[float, int, int, ChannelRecord, Point]] = []
            for record in pending:
                if len(record.observations) >= 2:
                    result = self._bounded_localization_result(record)
                    entries = (
                        (result.minimum_circle.center,)
                        if result.bounded and result.minimum_circle is not None
                        else self._symmetric_second_points(record)
                    )
                else:
                    entries = self._symmetric_second_points(record)
                for entry_index, entry in enumerate(entries):
                    choices.append(
                        (
                            current_position.distance_to(entry),
                            record.channel,
                            entry_index,
                            record,
                            entry,
                        )
                    )
            if planned_order is None:
                _, _, _, record, entry = min(choices)
            else:
                record = planned_order.pop(0)
                record_choices = [choice for choice in choices if choice[3] is record]
                _, _, _, _, entry = min(record_choices)
            if len(record.observations) >= 2:
                current_position = self._refine_after_two_bearings(
                    robot, record, current_position
                )
            elif self.single_observation_optical:
                self.fallback_channels.append(record.channel)
                current_position = self._clear_initial_sector(
                    robot, record, current_position
                )
            else:
                current_position = self._localize_or_cover(
                    robot, record, entry, current_position
                )
            pending.remove(record)
        response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return Problem4Report(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            fallback_channels=tuple(self.fallback_channels),
            exit_response=response,
        )


class OptimizedProblem4Planner(AdaptiveProblem4Planner):
    """问题4当前综合最优版：双环保证网格与安全自适应清除。"""
    def __init__(self, **kwargs: object) -> None:
        self.dynamic_outer_route_weight = float(
            kwargs.pop("dynamic_outer_route_weight", 0.5)
        )
        self.joint_outer_services = bool(kwargs.pop("joint_outer_services", True))
        kwargs.setdefault("maximum_refinement_circle_m", 500.0)
        kwargs.setdefault("speculative_clear_radius_m", 180.0)
        kwargs.setdefault("opportunity_attempts", 8)
        kwargs.setdefault("opportunity_distance_m", 1600.0)
        kwargs.setdefault("opportunity_min_angle_deg", 12.0)
        kwargs.setdefault("predicted_range_m", 900.0)
        kwargs.setdefault("inline_service_detour_m", 500.0)
        kwargs.setdefault("single_observation_optical", False)
        kwargs.setdefault("global_service_route", True)
        super().__init__(**kwargs)
        inner = [
            Point(950.0 * cos(index * pi / 6.0), 950.0 * sin(index * pi / 6.0))
            for index in range(12)
        ]
        outer = [
            Point(
                1875.0 * cos(pi / 12.0 + index * pi / 6.0),
                1875.0 * sin(pi / 12.0 + index * pi / 6.0),
            )
            for index in range(12)
        ]
        # 中心、内环顺时针一周，再从最近的外环点继续一周。
        self.cover_points = [Point(0.0, 0.0), *inner, *outer[10:], *outer[:10]]
        self._outer_ring_points = tuple(outer)
        self._scheduled_services: dict[tuple[float, float], ChannelRecord] = {}
    @staticmethod
    def _point_route_length(start: Point, points: list[Point]) -> float:
        total = 0.0
        current = start
        for point in points:
            total += current.distance_to(point)
            current = point
        return total
    def _adapt_remaining_cover_route(
        self,
        point_index: int,
        current_position: Point,
        detected: set[int],
    ) -> None:
        if point_index != 12:
            return
        pending = [
            self.records[channel]
            for channel in detected
            if not self.records[channel].cleared
        ]
        pending_targets = [self._service_target(record) for record in pending]
        if self.joint_outer_services:
            eligible: list[ChannelRecord] = []
            for record in pending:
                if len(record.observations) < 2:
                    continue
                result = self._bounded_localization_result(record)
                if (
                    result.bounded
                    and result.minimum_circle is not None
                    and result.minimum_circle.radius <= self.maximum_refinement_circle_m
                ):
                    eligible.append(record)
            nodes: list[tuple[str, int, Point, ChannelRecord | None]] = [
                ("cover", index, point, None)
                for index, point in enumerate(self._outer_ring_points)
            ]
            nodes.extend(
                ("service", record.channel, self._service_target(record), record)
                for record in eligible
            )
            remaining = list(nodes)
            order: list[tuple[str, int, Point, ChannelRecord | None]] = []
            cursor = current_position
            while remaining:
                node = min(
                    remaining,
                    key=lambda item: (cursor.distance_to(item[2]), item[0], item[1]),
                )
                order.append(node)
                remaining.remove(node)
                cursor = node[2]
            def order_length(candidate: list[tuple[str, int, Point, ChannelRecord | None]]) -> float:
                return self._point_route_length(
                    current_position, [node[2] for node in candidate]
                )
```

```
            improved = True
            while improved:
                improved = False
                best_length = order_length(order)
                for left in range(len(order) - 1):
                    for right in range(left + 1, len(order)):
                        candidate = (
                            order[:left]
                            + list(reversed(order[left : right + 1]))
                            + order[right + 1 :]
                        )
                        value = order_length(candidate)
                        if value + 1e-8 < best_length:
                            order = candidate
                            best_length = value
                            improved = True
            self._scheduled_services = {
                (round(node[2].x, 9), round(node[2].y, 9)): node[3]
                for node in order
                if node[0] == "service" and node[3] is not None
            }
            self.cover_points[13:] = [node[2] for node in order]
            return
        best: tuple[float, int, int, list[Point]] | None = None
        outer = self._outer_ring_points
        for direction in (-1, 1):
            for start_index in range(len(outer)):
                route = [
                    outer[(start_index + direction * step) % len(outer)]
                    for step in range(len(outer))
                ]
                cover_cost = self._point_route_length(current_position, route)
                if pending_targets:
                    # 各源之间的后续路线基本不受外环终点影响，候选间
                    # 主要差异是从终点接入首个残余源的距离。
                    service_cost = min(
                        route[-1].distance_to(target)
                        for target in pending_targets
                    )
                else:
                    service_cost = 0.0
                score = cover_cost + self.dynamic_outer_route_weight * service_cost
                candidate = (score, direction, start_index, route)
                if best is None or candidate[:3] < best[:3]:
                    best = candidate
        assert best is not None
        self.cover_points[13:] = best[3]
class RollingOptimizedProblem4Planner(OptimizedProblem4Planner):
    """问题4滚动联合规划：反复合并剩余保证点和已定位源。"""
    def __init__(
        self,
        *,
        single_service_insertion_m: float | None = None,
        route_multistart: int = 5,
        cover_priority_m: float = 0.0,
        rollout_information_weight: float = 0.0,
        rollout_particles: int = 128,
        exact_route_limit: int = 12,
        **kwargs: object,
    ) -> None:
        kwargs.setdefault("refinement_probe_mode", "centroid")
        kwargs.setdefault("maximum_refinement_circle_m", 650.0)
        kwargs.setdefault("speculative_clear_radius_m", 200.0)
        kwargs.setdefault("joint_outer_services", False)
        kwargs.setdefault("dynamic_outer_route_weight", 0.25)
        super().__init__(**kwargs)
        self.single_service_insertion_m = single_service_insertion_m
        self.route_multistart = max(1, route_multistart)
        self.cover_priority_m = cover_priority_m
        self.rollout_information_weight = rollout_information_weight
        self.exact_route_limit = max(0, exact_route_limit)
        rng = Random(20260911)
        particles: list[tuple[Point, float, float | None]] = []
        for index in range(rollout_particles):
            radius = 1800.0 * sqrt(rng.random())
            angle = rng.random() * 2.0 * pi
            particles.append(
                (
                    Point(radius * cos(angle), radius * sin(angle)),
                    rng.uniform(1000.0, 1500.0),
                    rng.random() * 360.0 if index % 2 else None,
                )
            )
        self._discovery_particles = tuple(particles)
        self._visited_cover_points: list[Point] = []
    @staticmethod
    def _route_length_from(
        start: Point,
        route: list[tuple[str, int, Point, ChannelRecord | None]],
    ) -> float:
        total = 0.0
        current = start
        for _, _, point, _ in route:
            total += current.distance_to(point)
            current = point
        return total
    @staticmethod
    def _particle_signal(
        particle: tuple[Point, float, float | None],
        detector: Point
    ) -> bool:
        source, receive_radius, direction = particle
        if source.distance_to(detector) > receive_radius:
            return False
        if direction is None:
            return True
        detector_angle = degrees(
            atan2(detector.y - source.y, detector.x - source.x)
        ) % 360.0
        difference = (detector_angle - direction + 180.0) % 360.0 - 180.0
        return abs(difference) <= 90.0
    def _rollout_route_score(
        self,
        route: list[tuple[str, int, Point, ChannelRecord | None]],
        detected_count: int,
    ) -> float:
        distance_score = self._route_length_from(self._rolling_current, route)
        if self.rollout_information_weight <= 0.0:
            return distance_score
        cover_order = [point for kind, _, point, _ in route if kind == "cover"]
        if not cover_order:
            return distance_score
        consistent = [
            particle
            for particle in self._discovery_particles
            if not any(
                self._particle_signal(particle, point)
                for point in self._visited_cover_points
            )
        ]
        if not consistent:
            return distance_score
        counts = []
        for particle in consistent:
            count = len(cover_order)
            for index, point in enumerate(cover_order, 1):
                if self._particle_signal(particle, point):
                    count = index
                    break
            counts.append(count)
        expected_cover_count = sum(counts) / len(counts)
        if detected_count < 10:
            expected_remaining_sources = 13.0 - detected_count
        else:
            expected_remaining_sources = max(0.0, (16.0 - detected_count) / 2.0)
        # 一次频道检测约5秒，大多数还有1秒换频；按5m/s折算30米路程。
        information_cost_m = 30.0 * expected_remaining_sources * expected_cover_count
        return distance_score + self.rollout_information_weight * information_cost_m
    def _exact_open_route(
        self,
        current: Point,
        tasks: list[tuple[str, int, Point, ChannelRecord | None]],
    ) -> list[tuple[str, int, Point, ChannelRecord | None]]:
        """用Held--Karp动态规划求小规模开放TSP，终点不限定。"""
        count = len(tasks)
        if count <= 1:
            return list(tasks)
        costs: dict[tuple[int, int], float] = {}
        previous: dict[tuple[int, int], int | None] = {}
        for index, task in enumerate(tasks):
            state = (1 << index, index)
            costs[state] = current.distance_to(task[2])
            previous[state] = None
        for mask in range(1, 1 << count):
            for last in range(count):
                state = (mask, last)
                base = costs.get(state)
                if base is None:
                    continue
                for following in range(count):
                    if mask & (1 << following):
                        continue
                    next_mask = mask | (1 << following)
                    next_state = (next_mask, following)
                    value = base + tasks[last][2].distance_to(tasks[following][2])
                    old = costs.get(next_state)
                    if old is None or value + 1e-9 < old:
                        costs[next_state] = value
                        previous[next_state] = last
```

```python
        full = (1 << count) - 1
        last = min(
            range(count),
            key=lambda index: (costs[(full, index)], tasks[index][0], tasks[index][1]),
        )
        indices: list[int] = []
        mask = full
        while True:
            indices.append(last)
            parent = previous[(mask, last)]
            if parent is None:
                break
            mask ^= 1 << last
            last = parent
        indices.reverse()
        return [tasks[index] for index in indices]
    def _rolling_route(
        self,
        current: Point,
        remaining_cover: list[Point],
        detected: set[int],
    ) -> list[tuple[str, int, Point, ChannelRecord | None]]:
        tasks: list[tuple[str, int, Point, ChannelRecord | None]] = [
            ("cover", index, point, None)
            for index, point in enumerate(remaining_cover)
        ]
        self._rolling_current = current
        for channel in sorted(detected):
            record = self.records[channel]
            if record.cleared:
                continue
            if len(record.observations) >= 2:
                result = self._bounded_localization_result(record)
                if (
                    result.bounded
                    and result.minimum_circle is not None
                    and result.minimum_circle.radius <= self.maximum_refinement_circle_m
                ):
                    tasks.append(
                        ("service", channel, self._refinement_probe(result.polygon), record)
                    )
                elif not remaining_cover:
                    # 覆盖已结束时，近退化或过大定位区域也必须进入
                    # 服务队列；_refine_after_two_bearings会切换到有限扇形保底。
                    tasks.append(
                        ("service", channel, self._predicted_source(record), record)
                    )
            elif not remaining_cover:
                entries = self._symmetric_second_points(record)
                entry = min(entries, key=lambda point: current.distance_to(point))
                tasks.append(("service", channel, entry, record))
        if 0 < len(tasks) <= self.exact_route_limit:
            return self._exact_open_route(current, tasks)
        remaining_source_factor = max(0, 16 - len(detected)) / 16.0
        information_lookup: dict[tuple[float, float], tuple[bool, ...]] = {}
        consistent_particles: tuple[tuple[Point, float, float | None], ...] = ()
        if self.rollout_information_weight > 0.0 and remaining_cover:
            consistent_particles = tuple(
                particle
                for particle in self._discovery_particles
                if not any(
                    self._particle_signal(particle, visited)
                    for visited in self._visited_cover_points
                )
            )
            for point in remaining_cover:
                key = (round(point.x, 9), round(point.y, 9))
                information_lookup[key] = tuple(
                    self._particle_signal(particle, point)
                    for particle in consistent_particles
                )
        if len(detected) < 10:
            expected_undetected_sources = 13.0 - len(detected)
        else:
            expected_undetected_sources = max(
                0.0, (16.0 - len(detected)) / 2.0
            )
        def information_cost_m(
            route: list[tuple[str, int, Point, ChannelRecord | None]],
        ) -> float:
            if not consistent_particles or expected_undetected_sources <= 0.0:
                return 0.0
            cover_order = [
                point for kind, _, point, _ in route if kind == "cover"
            ]
            if not cover_order:
                return 0.0
            total_first_index = 0
            for particle_index in range(len(consistent_particles)):
                first_index = len(cover_order)
                for index, point in enumerate(cover_order, 1):
                    key = (round(point.x, 9), round(point.y, 9))
                    if information_lookup[key][particle_index]:
                        first_index = index
                        break
                total_first_index += first_index
            expected_first_index = total_first_index / len(consistent_particles)
            # 一次检测约5秒，典型换频1秒；以5m/s折成30米等价路长。
            return 30.0 * expected_undetected_sources * expected_first_index
        def objective(
            route: list[tuple[str, int, Point, ChannelRecord | None]],
        ) -> float:
            value = self._route_length_from(current, route)
            if self.cover_priority_m > 0.0 and remaining_source_factor > 0.0:
                latency = 0.0
                for position, (kind, _, point, _) in enumerate(route):
                    if kind != "cover":
                        continue
                    radius = sqrt(point.x * point.x + point.y * point.y)
                    utility = max(0.0, 1.0 - (radius / 2800.0) ** 2)
                    latency += position * utility
                value += self.cover_priority_m * remaining_source_factor * latency
            if self.rollout_information_weight > 0.0:
                value += (
                    self.rollout_information_weight * information_cost_m(route)
                )
            return value
        def improve(
            route: list[tuple[str, int, Point, ChannelRecord | None]],
        ) -> list[tuple[str, int, Point, ChannelRecord | None]]:
            improved = True
            while improved:
                improved = False
                best = objective(route)
                for left in range(len(route) - 1):
                    for right in range(left + 1, len(route)):
                        candidate = (
                            route[:left]
                            + list(reversed(route[left : right + 1]))
                            + route[right + 1 :]
                        )
                        value = objective(candidate)
                        if value + 1e-8 < best:
                            route = candidate
                            best = value
                            improved = True
            improved = True
            while improved:
                improved = False
                best = objective(route)
                for source_index in range(len(route)):
                    node = route[source_index]
                    shortened = route[:source_index] + route[source_index + 1 :]
                    for destination in range(len(shortened) + 1):
                        candidate = (
                            shortened[:destination]
                            + [node]
                            + shortened[destination:]
                        )
                        value = objective(candidate)
                        if value + 1e-8 < best:
                            route = candidate
                            best = value
                            improved = True
            return route
        first_candidates = sorted(
            tasks,
            key=lambda item: (
                current.distance_to(item[2]), item[0] != "service", item[1]
            ),
        )[: self.route_multistart]
        routes: list[list[tuple[str, int, Point, ChannelRecord | None]]] = []
        for first in first_candidates:
            remaining = list(tasks)
            remaining.remove(first)
            seed_route = [first]
            cursor = first[2]
            while remaining:
                selected = min(
                    remaining,
                    key=lambda item: (
                        cursor.distance_to(item[2]),
                        item[0] != "service",
```

```
                        item[1],
                    ),
                )
                seed_route.append(selected)
                remaining.remove(selected)
                cursor = selected[2]
            routes.append(improve(seed_route))
        route = min(
            routes,
            key=objective,
        )
        if self.single_service_insertion_m is not None and remaining_cover:
            single_records = [
                self.records[channel]
                for channel in sorted(detected)
                if not self.records[channel].cleared
                and len(self.records[channel].observations) == 1
            ]
            for record in single_records:
                predicted = self._predicted_source(record)
                entries = self._symmetric_second_points(record)
                best_insertion: tuple[float, int] | None = None
                for position in range(len(route) + 1):
                    previous = current if position == 0 else route[position - 1][2]
                    following = None if position == len(route) else route[position][2]
                    for entry in entries:
                        delta = previous.distance_to(entry) + entry.distance_to(predicted)
                        if following is not None:
                            delta += predicted.distance_to(following)
                            delta -= previous.distance_to(following)
                        candidate = (delta, position)
                        if best_insertion is None or candidate < best_insertion:
                            best_insertion = candidate
                if (
                    best_insertion is not None
                    and best_insertion[0] <= self.single_service_insertion_m
                ):
                    route.insert(
                        best_insertion[1],
                        ("service", record.channel, predicted, record),
                    )
        return route
    def run(self, robot: RobotInterface) -> Problem4Report:
        robot.enter()
        unknown = set(range(1, 21))
        detected: set[int] = set()
        current_channel = 1
        current_position = Point(0.0, 0.0)
        remaining_cover = list(self.cover_points)
        while remaining_cover or any(
            not self.records[channel].cleared for channel in detected
        ):
            route = self._rolling_route(
                current_position, remaining_cover, detected
            )
            if not route:
                break
            kind, _, point, record = route[0]
            if kind == "service":
                assert record is not None
                if len(record.observations) >= 2:
                    current_position = self._refine_after_two_bearings(
                        robot, record, current_position
                    )
                else:
                    entries = self._symmetric_second_points(record)
                    entry = min(
                        entries,
                        key=lambda item: current_position.distance_to(item),
                    )
                    current_position = self._localize_or_cover(
                        robot, record, entry, current_position
                    )
                continue
            remaining_cover.remove(point)
            ordered_channels = sorted(
                unknown, key=lambda channel: channel != current_channel
            )
            for channel in ordered_channels:
                response = robot.measure(point.x, point.y, channel)
                current_position = point
                current_channel = channel
                result = response["measure_result"]
                if result == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] == "success":
                        self.records[channel].cleared = True
                        detected.add(channel)
                        unknown.remove(channel)
                elif result == "direction":
                    self.records[channel].observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
                    detected.add(channel)
                    unknown.remove(channel)
            candidates: list[tuple[float, int, ChannelRecord]] = []
            for channel in sorted(detected):
                candidate_record = self.records[channel]
                if candidate_record.cleared or len(candidate_record.observations) != 1:
                    continue
                if self._opportunity_counts.get(channel, 0) >= self.opportunity_attempts:
                    continue
                predicted = self._predicted_source(candidate_record)
                if point.distance_to(predicted) > self.opportunity_distance_m:
                    continue
                angle = self._crossing_angle(
                    candidate_record.observations[0].station, point, predicted
                )
                if angle < self.opportunity_min_angle_deg:
                    continue
                candidates.append((-angle, channel, candidate_record))
            for _, channel, candidate_record in sorted(candidates):
                self._opportunity_counts[channel] = (
                    self._opportunity_counts.get(channel, 0) + 1
                )
                response = robot.measure(point.x, point.y, channel)
                current_channel = channel
                if response["measure_result"] == "near":
                    clear_response = robot.clear(point.x, point.y, channel)
                    if clear_response["clear_result"] == "success":
                        candidate_record.cleared = True
                elif response["measure_result"] == "direction":
                    candidate_record.observations.append(
                        BearingObservation(point, float(response["svd_deg"]))
                    )
            self._visited_cover_points.append(point)
            if len(detected) == 16:
                remaining_cover.clear()
        response = robot.exit()
        cleared = tuple(
            channel for channel, record in self.records.items() if record.cleared
        )
        return Problem4Report(
            detected_channels=tuple(sorted(detected)),
            cleared_channels=cleared,
            fallback_channels=tuple(self.fallback_channels),
            exit_response=response,
        )
```

solve_problem1.py

```
"""问题一通用求解入口。
输入JSON格式:
{
  "error_deg": 1.0,
  "measurements": [
    {"x": -500, "y": 0, "bearing_deg": 0},
    {"x": 0, "y": -500, "bearing_deg": 90}
  ]
}
"""
from __future__ import annotations
import argparse
import json
from pathlib import Path
from b_model.geometry import Point, build_localization_polygon, polygon_area
def _point_dict(point: Point) -> dict[str, float]:
    return {"x": point.x, "y": point.y}
def solve(payload: dict) -> dict:
    error_deg = float(payload.get("error_deg", 1.0))
    raw_measurements = payload.get("measurements", [])
    if len(raw_measurements) < 2:
        raise ValueError("至少需要两次测量")
    measurements = [
        (
            Point(float(item["x"]), float(item["y"])),
            float(item["bearing_deg"]),
        )
        for item in raw_measurements
    ]
    result = build_localization_polygon(measurements, error_deg=error_deg)
```

```
    if not result.polygon:
        return {
            "status": "empty" if result.bounded else "unbounded",
            "message": (
                "测量约束不相容，可行域为空"
                if result.bounded
                else "可行域无界，需要增加测量或改善测站布局"
            ),
        }
    if not result.bounded:
        return {
            "status": "unbounded",
            "message": "可行域无界，需要增加测量或改善测站布局",
            "known_vertices": [_point_dict(p) for p in result.polygon],
        }
    assert result.diameter is not None
    assert result.diameter_pair is not None
    assert result.minimum_circle is not None
    return {
        "status": "bounded",
        "vertex_count": len(result.polygon),
        "vertices_counterclockwise": [_point_dict(p) for p in result.polygon],
        "area": polygon_area(result.polygon),
        "diameter": result.diameter,
        "diameter_pair": [
            _point_dict(result.diameter_pair[0]),
            _point_dict(result.diameter_pair[1]),
        ],
        "minimum_enclosing_circle": {
            "center": _point_dict(result.minimum_circle.center),
            "radius": result.minimum_circle.radius,
        },
        "selected_diameter_pair_circle_covers": (
            result.selected_diameter_pair_circle_covers
        ),
        "any_diameter_circle_covers": result.diameter_circle_covers,
    }
def main() -> None:
    parser = argparse.ArgumentParser(description="求解B题问题一的定位可行域")
    parser.add_argument("input", type=Path, help="输入JSON文件")
    parser.add_argument("-o", "--output", type=Path, help="输出JSON文件；省略时打印")
    args = parser.parse_args()
    payload = json.loads(args.input.read_text(encoding="utf-8"))
    answer = solve(payload)
    rendered = json.dumps(answer, ensure_ascii=False, indent=2)
    if args.output is None:
        print(rendered)
    else:
        args.output.write_text(rendered + "\n", encoding="utf-8")
if __name__ == "__main__":
    main()
```

run_official_problem3_practice.py

```
"""连接官方模拟器运行问题3演练。
本脚本不会在缺少显式确认参数时调用 /enter，防止误启动当前测试。
必须先在官方模拟器界面选择“问题3演练测试”，等待接口就绪后再运行。
"""
from __future__ import annotations
import argparse
import json
import sys
from pathlib import Path
from b_model.problem3_planner import (
    BayesianProbeProblem3Planner,
    JointHomingProblem3Planner,
    Problem3Planner,
    RefinedProbeProblem3Planner,
)
from b_model.simulator_client import SimulatorClient
CONFIRMATION_TEXT = "START-P3-PRACTICE"
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
if hasattr(sys.stderr, "reconfigure"):
    sys.stderr.reconfigure(encoding="utf-8")
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="在已经就绪的官方模拟器问题3演练局中运行优化规划器。"
    )
    parser.add_argument(
        "--robot-id",
        required=True,
        help="当前登录模拟器的参赛队号。",
    )
    parser.add_argument(
        "--base-url",
        default="http://127.0.0.1:2026",
        help="官方模拟器本机接口地址；修改过端口时同步修改。",
    )
    parser.add_argument(
        "--confirm",
        required=True,
        help=f"必须填写 {CONFIRMATION_TEXT} 才会调用 /enter。",
    )
    parser.add_argument(
        "--log-path",
        type=Path,
        default=Path("logs") / "official_problem3_practice.jsonl",
        help="本地请求响应日志路径。",
    )
    parser.add_argument(
        "--planner",
        choices=("stable", "fast249", "fast240", "refined"),
        default="stable",
        help=(
            "stable为原稳健版；fast249为七点联合竞速版；"
            "fast240为粒子后验首测点版；refined为当前最快的完整清除版。"
        ),
    )
    return parser.parse_args()
def main() -> int:
    args = parse_args()
    if args.confirm != CONFIRMATION_TEXT:
        print(
            "确认文字不匹配，未连接模拟器，也未调用 /enter。\n"
            "请先在模拟器中启动“问题3演练测试”并等待接口就绪，"
            f"然后把 --confirm 设置为 {CONFIRMATION_TEXT}。",
            file=sys.stderr,
        )
        return 2
    client = SimulatorClient(
        args.robot_id,
        base_url=args.base_url,
        log_path=args.log_path,
        timeout_s=10.0,
        max_network_retries=4,
    )
    if args.planner == "refined":
        planner = RefinedProbeProblem3Planner()
    elif args.planner == "fast240":
        planner = BayesianProbeProblem3Planner(probe_uncertainty_weight=0.6)
    elif args.planner == "fast249":
        planner = JointHomingProblem3Planner(
            homing_prediction_scale=1.1,
            speculative_clear_radius_m=100.0,
            speculative_clear_at_centroid=True,
            aggressive_opportunity_distance_m=1000.0,
            aggressive_opportunity_min_angle_deg=15.0,
            max_aggressive_opportunity_failures=1,
            cover_point_design="seven_guaranteed",
        )
    else:
        planner = Problem3Planner()
    try:
        report = planner.run(client)
    except Exception as error:
        print(f"演练运行失败：{error}", file=sys.stderr)
        if client.state.entered and not client.state.exited:
            try:
                exit_response = client.exit()
                print("已尝试安全退出：", json.dumps(exit_response, ensure_ascii=False))
            except Exception as exit_error:  # 接口关闭时只能由界面查看结束原因。
                print(f"安全退出未成功：{exit_error}", file=sys.stderr)
        print(f"请求响应日志：{args.log_path.resolve()}", file=sys.stderr)
        return 1
    summary = {
        "detected_channels": report.detected_channels,
        "cleared_channels": report.cleared_channels,
        "unknown_channels_after_cover": report.unknown_channels_after_cover,
        "virtual_time_s": client.state.virtual_time_s,
        "exit_response": report.exit_response,
        "log_path": str(args.log_path.resolve()),
    }
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```

run_official_problem4_practice.py

```
"""Connect to the official simulator and run a Problem 4 practice case.
The simulator UI must already be in a ready Problem 4 PRACTICE case.  The
explicit command-line token prevents this runner from being started by an
accidental double click.
"""
from __future__ import annotations
```

```python
import argparse
import json
import sys
from pathlib import Path
from b_model.problem4_planner import (
    OptimizedProblem4Planner,
    RollingOptimizedProblem4Planner,
)
from b_model.simulator_client import SimulatorClient
CONFIRMATION_TEXT = "START-P4-PRACTICE"
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
if hasattr(sys.stderr, "reconfigure"):
    sys.stderr.reconfigure(encoding="utf-8")
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Run the Problem 4 planner in a ready official practice case."
    )
    parser.add_argument("--robot-id", required=True)
    parser.add_argument("--base-url", default="http://127.0.0.1:2026")
    parser.add_argument("--confirm", required=True)
    parser.add_argument(
        "--log-path",
        type=Path,
        default=Path("logs") / "official_problem4_practice.jsonl",
    )
    parser.add_argument(
        "--planner",
        choices=("rolling", "conservative"),
        default="rolling",
        help="rolling=current fastest robust candidate; conservative=slower fallback",
    )
    return parser.parse_args()
def main() -> int:
    args = parse_args()
    if args.confirm != CONFIRMATION_TEXT:
        print("Confirmation token mismatch; /enter was not called.", file=sys.stderr)
        return 2
    args.log_path.parent.mkdir(parents=True, exist_ok=True)
    client = SimulatorClient(
        args.robot_id,
        base_url=args.base_url,
        log_path=args.log_path,
        timeout_s=8.0,
        max_network_retries=2,
    )
    if args.planner == "rolling":
        planner = RollingOptimizedProblem4Planner(
            exact_route_limit=12,
            rollout_information_weight=0.0,
            sector_search_mode="outward",
            polygon_search_mode="legacy",
        )
    else:
        planner = OptimizedProblem4Planner()
    try:
        report = planner.run(client)
    except Exception as error:
        print(f"Problem 4 practice failed: {error}", file=sys.stderr)
        if client.state.entered and not client.state.exited:
            try:
                exit_response = client.exit()
                print("Safe exit response:", json.dumps(exit_response, ensure_ascii=False))
            except Exception as exit_error:
                print(f"Safe exit failed: {exit_error}", file=sys.stderr)
        print(f"Request log: {args.log_path.resolve()}", file=sys.stderr)
        return 1
    summary = {
        "planner": args.planner,
        "detected_channels": report.detected_channels,
        "cleared_channels": report.cleared_channels,
        "detected_count": len(report.detected_channels),
        "cleared_count": len(report.cleared_channels),
        "fallback_channels": report.fallback_channels,
        "virtual_time_s": client.state.virtual_time_s,
        "exit_response": report.exit_response,
        "log_path": str(args.log_path.resolve()),
    }
    print(json.dumps(summary, ensure_ascii=False, indent=2))
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```